

Metaheuristic Search with Inequalities and Target Objectives for Mixed Binary Optimization Part I: Exploiting Proximity

Fred Glover, OptTek Systems, Inc., USA

Saïd Hanafi, University Lille Nord de France, France

ABSTRACT

Recent adaptive memory and evolutionary metaheuristics for mixed integer programming have included proposals for introducing inequalities and target objectives to guide the search. These guidance approaches are useful in intensification and diversification strategies related to fixing subsets of variables at particular values, and in strategies that use linear programming to generate trial solutions whose variables are induced to receive integer values. In Part I (the present article), we show how to improve such approaches by new inequalities that dominate those previously proposed and by associated target objectives that underlie the creation of both inequalities and trial solutions. Part I focuses on exploiting inequalities in target solution strategies by including partial vectors and more general target objectives. We also propose procedures for generating target objectives and solutions by exploiting proximity in original space or projected space. Part II of this study (to appear in a subsequent issue) focuses on supplementary linear programming models that exploit the new inequalities for intensification and diversification, and introduce additional inequalities from sets of elite solutions that enlarge the scope of these models. Part II indicates more advanced approaches for generating the target objective based on exploiting the mutually reinforcing notions of reaction and resistance. Our work in the concluding segment, building on the foundation laid in Part I, examines ways our framework can be exploited in generating target objectives, employing both older adaptive memory ideas of tabu search and newer ones proposed here for the first time.

Keywords: Adaptive Search, Parametric Tabu Search, Valid Inequalities, Zero-One Mixed Integer Programming

1. NOTATION AND PROBLEM FORMULATION

$$(MIP) \begin{cases} \text{Minimize} & x_0 = fx + gy \\ \text{subject to} & (x, y) \in Z = \{(x, y) : Ax + Dy \geq b\} \\ & x \text{ integer} \end{cases}$$

We represent the mixed integer programming problem in the form

We assume that $Ax + Dy \geq b$ includes the inequalities $U_j \geq x_j \geq 0, j \in N = \{1, \dots, N\}$, where

DOI: 10.4018/jamc.2010102601

some components of U_j may be infinite. The linear programming relaxation of (MIP) that results by dropping the integer requirement on x is denoted by (LP). We further assume $Ax + Dy \geq b$ includes an objective function constraint $x_0 \leq U_0$, where the bound U_0 is manipulated as part of a search strategy for solving (MIP), subject to maintaining $U_0 < x_0^*$, where x_0^* is the x_0 value for the currently best known solution x^* to (MIP).

The current paper focuses on the zero-one version of (MIP) denoted by (MIP:0-1), in which $U_j = 1$ for all $j \in N$. We refer to the LP relaxation of (MIP:0-1) likewise as (LP), since the identity of (LP) will be clear from the context,

In the following we make reference to two types of search strategies: those that fix subsets of variables to particular values within approaches for exploiting strongly determined and consistent variables, and those that make use of solution targeting procedures. As developed here, the latter solve a linear programming problem $LP(x', c')$ that includes the constraints of (LP) (and additional bounding constraints in the general (MIP) case) while replacing the objective function x_0 by a linear function $v_0 = c'x$. The vector x' is called a *target solution*, and the vector c' consists of integer coefficients c'_j that seek to induce assignments $x_j = x'_j$ for different variables with varying degrees of emphasis.

We adopt the convention that each instance of $LP(x', c')$ implicitly includes the (LP) objective of minimizing the function $x_0 = fx + gy$ as a secondary objective, dominated by the objective of minimizing $v_0 = c'x$, so that the true objective function consists of minimizing $\omega_0 = Mv_0 + x_0$, where M is a large positive number. As an alternative to working with ω_0 in the form specified, it can be advantageous to solve $LP(x', c')$ in two stages. The first stage minimizes $v_0 = c'x$ to yield an optimal solution $x = x''$ (with objective function value $v_0'' = c'x''$), and the second stage enforces $v_0 = v_0''$ to solve the residual problem of minimizing $x_0 = fx + gy$.²

A second convention involves an interpretation of the problem constraints. Selected instances of inequalities generated by approaches of the following sections will be understood

to be included among the constraints $Ax + Dy \geq b$ of (LP). In our definition of $LP(x', c')$ and other linear programs related to (LP), we take the liberty of representing the currently updated form of the constraints $Ax + Dy \geq b$ by the compact representation $x \in X = \{x: (x, y) \in Z\}$, recognizing that this involves a slight distortion in view of the fact that we implicitly minimize a function of y as well as x in these linear programs.³

To launch our investigation of the problem (MIP:0-1) we first review previous ideas for generating guiding inequalities for this problem in Section 2 and associated target objective strategies using partial vectors and more general target objectives in Section 3. We then present new inequalities in Section 4 that improve on those previously proposed. The fundamental issue of creating the target objectives that can be used to generate the new inequalities and that lead to trial solutions for (MIP: 0-1) by exploiting proximity is addressed in Section 5. Concluding remarks are given in Section 6.

2. EXPLOITING INEQUALITIES IN TARGET SOLUTION STRATEGIES

Let x' denote an arbitrary solution, and define the associated index sets

$$\begin{aligned} N(x', v) &= \{j \in N: x'_j = v\} \text{ for } v \in \{0, 1\}, \\ N(x') &= \{j \in N: x'_j \in \{0, 1\}\} \text{ and} \\ N^*(x') &= \{j \in N: x'_j \in]0, 1[\}, \end{aligned}$$

we have $N = N(x') \cup N^*(x')$. For any real number z , $\lceil z \rceil$ and $\lfloor z \rfloor$ respectively identify the least integer $\geq z$ and the greatest integer $\leq z$.

Define

$$\delta(x', x) = \sum_{j \in N} x_j (1 - x'_j) + x'_j (1 - x_j) \quad (1)$$

Proposition 1. Let x' denote an arbitrary binary solution. Then the inequality

$$\delta(x', x) \geq 1 \quad (1.1)$$

eliminates the assignment $x = x'$ as a feasible solution, but admits all other binary x vectors.

Proof: It is evident that $\delta(x', x) = \|x - x'\|_1 = \|x - x'\|_2$, so for all $x \neq x'$, we have $\delta(x', x) > 0$. The proposition follows from the fact that the value $\delta(x', x)$ is integer. $\square$

Remark 1: The inequality (1.1) has been used, for example, to produce 0-1 "short hot starts" for branch and bound by Spielberg and Guignard (2000) and Guignard and Spielberg (2003).

The constraint (1.1) is called a *canonical cut* on the unit hypercube by Balas and Jeroslow (1972). The constraint (1.1) has also been used by Soyster et al. (1978), Hanafi and Wilbaut (2006) and Wilbaut and Hanafi (2006).

Proposition 1 has the following consequence.

Corollary 1. Let x' denote an arbitrary binary solution. Then the inequality

$$\delta(x', x) \leq n - 1 \quad (1.2)$$

eliminates the assignment $x = e - x'$ (the complement of x') as a feasible solution, but admits all other binary x vectors.

Proof: Immediate from the proof on Proposition 1, by using $e - x'$. $\square$

We make use of solutions such as x' by assigning them the role of *target solutions*. In this approach, instead of imposing the inequality (1.1) we adopt the strategy of first seeing how close we can get to satisfying $x = x'$ by solving the LP problem⁴

LP(x'): Minimize $\{\delta(x', x): x \in X\}$

whereas earlier, $X = \{x: (x, y) \in Z\}$. We call x' the *target solution* for this problem. Let x'' denote an optimal solution to LP(x'). If the target solution x' is feasible for LP(x') then it is also uniquely optimal for LP(x') and hence $x'' = x'$, yielding $\delta(x', x'') = 0$. In such a case, upon testing x' for feasibility in (MIP:0-1) we can impose the inequality (1.1) as indicated earlier in order to avoid examining the solution again. However, in the case where x' is not feasible for LP(x'), an optimal solution x'' will yield $\delta(x', x'') > 0$ and since the distance $\delta(x, x')$ is an integer value we may impose the valid inequality

$$\delta(x, x') \geq \lceil \delta(x', x'') \rceil \quad (2.1)$$

The fact that $\delta(x', x'') > 0$ discloses that (2.1) is at least as strong as (1.1). In addition, if the solution x'' is a binary vector that differs from x' , we can also test x'' for feasibility in (MIP:0-1) and then redefine $x' = x''$, to additionally append the constraint (1.1) for this new x' . Consequently, regardless of whether x'' is binary, we eliminate x'' from the collection of feasible solutions as well as obtaining an inequality (2.1) when $\delta(x', x'')$ is fractional that dominates the original inequality (1.1).

Upon generating the inequality (2.1) (and an associated new form of (1.1) if x'' is binary), we continue to follow the policy of incorporating newly generated inequalities among the constraints defining X , and hence those defining Z of (MIP:0-1). Consequently, we assure that X excludes both the original x' and the solution x'' . This allows the problem LP(x') to be re-solved, either for x' as initially defined or for a new target vector (which can be also be x'' if the latter is binary), to obtain another solution x'' and a new (2.1).

Remark 2: The same observations can be made to eliminate the complement of x' , i.e. $(e - x')$, by solving the following LP problem:

LP*(x'): Maximize $\{\delta(x', x): x \in X\}$

Let $x^{+''}$ denote an optimal solution to $LP^+(x')$. If the complement of the target solution x' is feasible for $LP^+(x')$ then it is also uniquely optimal for $LP^+(x')$ and hence $x^{+''} = e - x'$, yielding $\delta(x', x^{+''}) = n$. In such a case, upon testing $e - x'$ for feasibility in (MIP:0-1) we can impose the inequality (1.2) as indicated earlier in order to avoid examining the solution again. However, in the case where $e - x'$ is not feasible for $LP^+(x')$, an optimal solution $x^{+''}$ will yield $\delta(x', x^{+''}) < n$ and we may impose the valid inequality

$$\delta(x', x) \leq \lfloor \delta(x', x^{+''}) \rfloor \quad (2.2)$$

The fact that $\delta(x', x^{+''}) < n$ discloses that (2.2) is at least as strong as (1.2).

It is worthwhile to use simple forms of tabu search memory based on recency and frequency in such processes to decide when to drop previously introduced inequalities, in order to prevent the collection of constraints from becoming unduly large. Such approaches can be organized in a natural fashion to encourage the removal of older constraints and to discourage the removal of constraints that have more recently or frequently been binding in the solutions to the $LP(x')$ problems produced (see Glover & Laguna, 1997; Glover & Hanafi, 2002). Older constraints can also be replaced by one or several surrogate constraints.

The strategy for generating a succession of target vectors x' plays a critical role in exploiting such a process. The feasibility pump approach of Fischetti, Glover and Lodi (2005) applies a randomized variant of nearest neighbor rounding to each non-binary solution x'' to generate the next x' , but does not make use of associated inequalities such as (1.x) and (2.x). In subsequent sections we show how to identify more effective inequalities and associated target objectives to help drive such processes.

3. GENERALIZATION TO INCLUDE PARTIAL VECTORS AND MORE GENERAL TARGET OBJECTIVES

We extend the preceding ideas in two ways, drawing on ideas of parametric branch and bound and parametric tabu search (Glover, 1978, 2006a). First we consider *partial x vectors* that may not have all components x_j determined, in the sense of being fixed by assignment or by the imposition of bounds. Such vectors are relevant in approaches where some variables are compelled or induced to receive particular values, while others remain free or are subject to imposed bounds that are not binding.

Let x' denote an arbitrary solution and $J \subseteq N(x')$ define the associated set

$$F(J, x') = \{x \in [0, 1]^n : x_j = x'_j \text{ for } j \in J\}$$

Let x, x' two arbitrary binary solutions and $J \subseteq N$, define

$$\delta(J, x', x) = \sum_{j \in J} x_j (1 - x'_j) + x'_j (1 - x_j) \quad (3)$$

Proposition 2. Let x' denote an arbitrary binary solution and $J' \subseteq N(x')$. Then the inequality

$$\delta(J', x', x) \geq 1 \quad (3.1)$$

eliminates all solutions in $F(J', x')$ as a feasible solution, but admits all other binary x vectors.

Proof: It is evident that for all $x \in F(J', x')$, we have $\delta(J', x', x) = 0$. $\square$

Proposition 2 has the following consequence.

Corollary 2. Let x' denote an arbitrary binary solution and $J' \subseteq N(e - x')$. Then the inequality

$$\delta(J', x', x) \leq |J'| - 1 \quad (3.2)$$

eliminates all solutions in $F(J', e - x')$ as a feasible solution, but admits all other binary x vectors.

Proof: Immediate from the proof on Proposition 2, by using $e - x'$.

We couple the target solution x' with the associated set $J' \subseteq N(x')$ to yield the problem

LP(x', J'): Minimize $(\delta(J', x', x): x \in X)$.

An optimal solution to LP(x', J'), as a generalization of LP(x'), will likewise be denoted by x'' . We obtain the inequality

$$\delta(J', x', x) \geq \lceil \delta(J', x', x'') \rceil \quad (4.1)$$

By an analysis similar to the derivation of (2.1), we observe that (4.1) is a valid inequality, i.e., it is satisfied by all binary vectors that are feasible for (MIP:0-1) (and more specifically by all such vectors that are feasible for LP(x', J')), with the exception of those ruled out by previous examination.

Remark 3: The same observations can be made to eliminate all solutions in $F(J', e - x')$ as a feasible solution by solving the following LP problem:

LP⁺(x', J'): Maximize $(\delta(J', x, x'): x \in X)$.

We obtain the inequality

$$\delta(J', x', x) \leq \lfloor \delta(J', x', x'') \rfloor \quad (4.2)$$

where x''' is an optimal solution to LP⁺(x', J').

In the special case where $J' = N(x')$, we have the following properties. Let $x' \in [0, 1]^n$ define the associated set

$$F(x') = F(x', N(x')) = \{x \in [0, 1]^n: x_j = x'_j \text{ for } j \in N(x')\}.$$

Let k be an integer satisfying $0 \leq k \leq n - |N(x')|$, the canonical hyperplane associated to the solution x' , denoted $H(x', k)$ is defined by

$$H(x', k) = \{x \in [0, 1]^n: \delta(N(x'), x', x) = k\}.$$

Proposition 3. $x \in H(x', k) \cap \{0, 1\}^n \Leftrightarrow \delta(x, F(x')) \cap \{0, 1\}^n = k$

where $\delta(x, F) = \min\{\delta(x, y): y \in F\}$

Proof: i) Necessity: if $x \in H(x', k) \cap \{0, 1\}^n$ this imply that $\delta(N(x'), x', x) = k$. Moreover if $y \in F(x') \cap \{0, 1\}^n$ thus $\delta(N(x'), x', y) = 0$ which imply that $y(N(x')) = x'(N(x'))$ where $x(J) = (x_j)_{j \in J}$. Hence, we have

$$\begin{aligned} \delta(x, y) &= \delta(N(x'), x, y) + \delta(N - N(x'), x, y) \\ &= \delta(N(x'), x, x') + \delta(N - N(x'), x, y) \\ &= k + \delta(N - N(x'), x, y) \geq k. \end{aligned}$$

Let $y' \in \{0, 1\}^n$ such that $y'(N(x')) = x'(N(x'))$ and $y'(N - N(x')) = x(N - N(x'))$. Then we have $y' \in F(x')$ and $\delta(x, y') = k$. Hence, $\delta(x, F(x') \cap \{0, 1\}^n) = \min\{\delta(x, y): y \in F(x') \cap \{0, 1\}^n\} = k$.

ii) Sufficiency: Let $y' \in F(x') \cap \{0, 1\}^n$ such that $\delta(x, y') = \delta(x, F(x') \cap \{0, 1\}^n) = k$. To simplify the notion let $F' = F(x') \cap \{0, 1\}^n = \{x \in \{0, 1\}^n: x_j = x'_j \text{ for } j \in N(x')\}$. Hence, we have $\delta(N - N(x'), x, F') = 0$ which implies that $\delta(N - N(x'), x, y') = 0$. Moreover if $y' \in F(x')$ we have $y'(N(x')) = x'(N(x'))$. Thus $\delta(N(x'), x, x') = k$. This implies that $x \in H(x', k) \cap \{0, 1\}^n$. which completes the proof of this proposition. $\square$

In the next proposition, we state relation between half-spaces associated to the canonical hyperplanes. Let $H(x', k)$ be the half-space associated with the canonical hyperplane $H(x', k)$ defined by

$$H(x', k) = \{x \in [0, 1]^n: \delta(N(x'), x', x) \leq k\}$$

Proposition 4. Let x' and x'' be two arbitrary solutions. Then

$$H(x', k) \cup H(x'', k) \subseteq H((x' + x'')/2, k).$$

Proof: Immediate from the fact that $N(x') \subseteq N((x' + x'')/2)$ and $N(x'') \subseteq N((x' + x'')/2)$. $\square$

Proposition 5. $\text{Co}(H(x', k) \cap \{0, 1\}^n) = H(x', k)$, where $\text{Co}(X)$ is the convex hull of the set X .

Proof: The inclusion $\text{Co}(H(x', k) \cap \{0, 1\}^n) \subseteq H(x', k)$ is obvious for any solution x' and integer k . To prove the inclusion

$$H(x', k) \subseteq \text{Co}(H(x', k) \cap \{0, 1\}^n), \quad (5.1)$$

let $y \in H(x', k)$ and observe that $\delta(N(x'), x', y) = \delta(N(x') \cap N(y), x', y) + \delta(N(x') \cap N^*(y), x', y) = k$. Now, we show by induction the second inclusion (4.3) on $p = \delta(N(x') \cap N^*(y), x', y)$. The statement is evident for $p = 0$. We assume that the statement is true for $\delta(N(x') \cap N^*(y), x', y) = p$. To show that it is also true for $\delta(N(x') \cap N^*(y), x', y) = p+1$, consider the subset $J \subseteq N(x') \cap N^*(y)$ such that

$$\delta(J, x', y) = \sum_{j \in J} \delta(j, x', y) = 1 \quad (5.2)$$

Thus we have $\delta(N(x'), x', y) = \delta(J, x', y) + \delta((N-J) \cap (N(x') \cap N^*(y)), x', y) = k$. For all $j \in J$, define the vector y^j such that

$$\begin{aligned} y^j(N-J) &= y(N-J) \\ y^j(J - \{j\}) &= x'(J - \{j\}) \text{ and} \\ y^j(\{j\}) &= 1 - x'_j. \end{aligned} \quad (5.3)$$

Now we show that

$$\sum_{j \in J} \delta(j, x', y) y^j = y. \quad (5.4)$$

From (5.3) and (5.2), for all $q \notin J$ we have

$$\begin{aligned} \sum_{j \in J} \delta(j, x', y) y_q^j &= \\ y_q \sum_{j \in J} \delta(j, x', y) y_q^j &= y_q. \end{aligned}$$

For all $q \in J$, from (5.3) and (5.2) and since $(1 - 2x'_q)^2 = 1$ and $x'_q(1 - x'_q) = 0$ for all $x'_q \in \{0, 1\}$, we have

$$\begin{aligned} \sum_{j \in J} \delta(j, x', y) y_q^j &= \sum_{j \in J - \{q\}} \delta(j, x', y) y_q^j + \\ &\quad \delta(q, x', y) y_q^q \\ &= \sum_{j \in J - \{q\}} \delta(j, x', y) x'_q + \\ &\quad \delta(q, x', y) (1 - x'_q) \\ &= \sum_{j \in J} \delta(j, x', y) x'_q + \\ &\quad \delta(q, x', y) (1 - 2x'_q) \\ &= x'_q \sum_{j \in J} \delta(j, x', y) + \\ &\quad \delta(q, x', y) (1 - 2x'_q) \\ &= x'_q + \delta(q, x', y) (1 - 2x'_q) \\ &= x'_q + ((1 - 2x'_q) y_q + x'_q) (1 - 2x'_q) \\ &= (1 - 2x'_q)^2 y_q + 2x'_q (1 - x'_q) \\ &= y_q \end{aligned}$$

Hence y is on the convex hull of the vector y^j for $j \in J$ (see 5.4) and it is easy to see that

$$\delta(N(x') \cap N^*(y^j), x', y) = p \text{ for all } j \in J.$$

By applying the hypothesis of the induction, we conclude that each vector y^j is also on the convex hull of binary solutions in $H(x', k)$. This completes the proof of the second inclusion (5.1). The proposition then follows from the two inclusions. $\square$

Proposition 5 is related to Theorem 1 of Balas and Jeroslow.

Let x' denote an arbitrary solution and $c \in \mathbb{I}N^n$ define the associated set

$$\begin{aligned} F(x', c) &= \{x \in [0, 1]^n : c_j(x_j - x'_j) = \\ &0 \text{ for } j \in N(x')\} \end{aligned}$$

Let x, x' be two arbitrary binary solutions and let c be an integer vector ($c \in \mathbb{I}N^n$). Define

$$\delta(c, x', x) = \sum_{j \in N} c_j \left(x_j (1 - x'_j) + x'_j (1 - x_j) \right) \quad \delta(c, x', x) \geq \lceil \delta(c, x', x'') \rceil \quad (7.1)$$

$$\delta(c, x', x) \leq \lfloor \delta(c, x', x'') \rfloor \quad (7.2)$$

Remark 4: $\delta(e, x', x) = \|x - x'\|_1 = \|x - x''\|_1$

$$B(c) = \{x \in [0, 1]^n : c_j x_j (1 - x_j) = 0\}.$$

Remark 5: $\delta(c, x', x) = \delta(J', x', x)$ if $c_j = 1$ if $j \in J'$ otherwise $c_j = 0$.

Remark 6: $B(e) = \{0, 1\}^n$ and $B(0) = [0, 1]^n$
 $C(x) = \{c \in \mathbb{R}_+^n : c_j x_j (1 - x_j) = 0\}.$

Proposition 6. Let x' denote an arbitrary solution and $c \in C(x')$. Then the inequality

$$\delta(c, x', x) \geq 1 \quad (6.1)$$

eliminates the solutions in $F(x', c)$ as a feasible solution, but admits all other binary x vectors. The inequality

$$\delta(c, x', x) \leq ce - 1 \quad (6.2)$$

eliminates the solutions in $F(e - x', c)$ as a feasible solution, but admits all other binary x vectors.

Proof: Immediate from the proof on Proposition 2 and Corollary 2 by setting $J' = \{c \in N : c_j \neq 0\}$. $\square$

We couple the target solution x' with the associated vector $c \in C(x')$ to yield the two problems

LP(x' , c): Minimize $(\delta(c, x', x) : x \in X)$.

LP⁺(x' , c): Maximize $(\delta(c, x', x) : x \in X)$.

An optimal solution to LP(x' , c) (resp. LP⁺(x' , c)), as a generalization of LP(x') (resp. LP⁺(x')), will likewise be denoted by x'' (resp. x'''). Finally, we obtain the inequalities

4. STRONGER INEQUALITIES AND ADDITIONAL VALID INEQUALITIES FROM BASIC FEASIBLE LP SOLUTIONS

Our approach to generate inequalities that dominate those of (7) is also able to produce additional valid inequalities from related basic feasible solution to the LP problem LP(x' , c), expanding the range of solution strategies for exploiting the use of target solutions. We refer specifically to the class of basic feasible solutions that may be called y -optimal solutions, which are dual feasible in the continuous variables y (including in y any continuous slack variables that may be added to the formulation), disregarding dual feasibility relative to the x variables. Such y -optimal solutions can be easily generated in the vicinity of an optimal LP solution by pivoting to bring one or more non-basic x variables into the basis, and then applying a restricted version of the primal simplex method that re-optimizes (if necessary) to establish dual feasibility relative only to the continuous variables, ignoring pivots that would bring x variables into the basis. By this means, instead of generating a single valid inequality from a given LP formulation such as LP(x' , c), we can generate a collection of such inequalities from a series of basic feasible y -optimal solutions produced by a series of pivots to visit some number of such solutions in the vicinity of an optimal solution.

As a foundation for these results, we assume x'' (or more precisely, (x'', y'')) has been obtained as a y -optimal basic feasible solution to LP(x' , c) by the bounded variable simplex method (see, e.g., Dantzig, 1963). By reference to the linear programming basis that produces x'' , which we will call the x'' basis, define $B = \{j \in N : x''_j \text{ is basic}\}$ and $NB = \{j \in N : x''_j \text{ is non-basic}\}$. We subdivide NB to identify the two subsets $NB(0) = \{j \in NB : x''_j = 0\}$, $NB(1) = \{j \in NB : x''_j = 1\}$. These sets have no necessary relation

to the sets $N'(x'', 0)$ and $N'(x'', 1)$, though in the case where x'' is an optimal basic solution⁵ to $LP(x', c)$, we would normally expect from the definition of c in relation to the target vector x' that there would be some overlap between $N'(x'', 0)$ and $N'(x'', 1)$ and similarly between $NB(1)$ and $N(1)$.

To simplify the notation, we find it convenient to give $\delta(c, x', x)$ an alternative representation.

$$\delta(c, x', x) = c'x + cx' \text{ with } c'_j = c_j(1 - 2x'_j), j \in N$$

The new inequality that dominates (6) results by taking account of the reduced costs derived from the x'' basis. Letting rc denote the reduced cost to an arbitrary y -optimal basic feasible solution x'' for $LP(x', c)$. Finally, to identify the new inequality, define the vector d' by

$$d' = \lceil c - rc \rceil$$

We then express the inequality as

$$\delta(d', x', x) \geq \lfloor \delta(d', x', x'') \rfloor \quad (8)$$

We first show that (8) is valid when generated from an arbitrary y -optimal basic feasible solution, and then demonstrate in addition that it dominates (7) in the case where (8) is a valid inequality (i.e., where (8) is derived from an optimal basic feasible solution). By our previously stated convention, it is understood that X (and (MIP:0-1)) may be modified by incorporating previously generated inequalities that exclude some binary solutions originally admitted as feasible.

Our results concerning (8) are based on identifying properties of basic solutions in reference to the problem

LP(x', d'): Minimize $(\delta(d', x', x): x \in X)$

Proposition 7. *The inequality (8) derived from an arbitrary y -optimal basic feasible solution x'' for $LP(x', c)$ is satisfied by all binary vectors*

$x \in X$, and excludes the solution $x = x''$ when $\delta(c, x', x'')$ is fractional.

Proof: *We first show that the basic solution x'' for $LP(x', c)$ is an optimal solution to $LP(x', d')$. Let rd denote the reduced cost for the objective function $\delta(d', x', x)$ for $LP(x', d')$ relative to the x'' basis. Assume $X = \{x: Ax \geq b, x \geq 0\}$ and let B the basis associated to the basic solution x'' . From the definitions the reduced cost $rc = c' - c'_B(A^B)^{-1}A$, and of $d' = \lfloor c' - rc \rfloor$, it follows that*

$$d' = \lfloor c'_B(A^B)^{-1}A \rfloor \text{ and } d'_B = c'_B \quad (8.1)$$

thus the reduced costs rd is null; i.e.,

$$rd' = d' - d'_B(A^B)^{-1}A = \lfloor c'_B(A^B)^{-1}A \rfloor - d'_B(A^B)^{-1}A = \lfloor c'_B(A^B)^{-1}A \rfloor - c'_B(A^B)^{-1}A \leq 0.$$

This establishes the optimality of x'' for $LP(x', d')$. Since the d'_j coefficients are all integers, we therefore obtain the valid inequality $\delta(d', x', x) \geq \lceil \delta(d', x', x'') \rceil$.

The definition of d' yields

$$\delta(d', x', x'') = \delta(c, x', x'') + \delta(\lfloor -rc \rfloor, x', x'').$$

The $\delta(\lfloor -rc \rfloor, x', x'')$ value is integer, since $x'' \in B(\lfloor -rc \rfloor)$. Thus, $\delta(d', x', x'')$ is fractional if and only if $\delta(c', x', x'')$ is fractional, and we also have

$$\lceil \delta(d', x', x'') \rceil = \lceil \delta(c, x', x'') \rceil + \delta(\lfloor -rc \rfloor, x', x'').$$

The proposition then follows from the definitions of (7) and (8).

Proposition 7 has the following novel consequence.

Corollary 3. *The inequality (8) is independent of the c_j values for the non-basic x variables. In particular, for any y -feasible basic solution and specified values c_j for $j \in B$, the coefficients d'_j*

of d' are identical for every choice of the integer coefficients $c_j, j \in NB$.

$$\delta(\lfloor -rc \rfloor, x', x'') - \delta(\lfloor -rc \rfloor, x', x) = \lfloor -rc \rfloor x'' - \lfloor -rc \rfloor x \geq 0 \quad (8.3)$$

Proof: The Corollary follows from the arguments of the Proof of Proposition 7 (see 8.1) thus showing that these changes cancel out, to produce the same final d'_0 and d' after implementing the changes that existed previously.

Moreover we have

$$\delta(d', x', x'') = \delta(c, x', x'') + \delta(\lfloor -rc \rfloor, x', x''). \quad (8.4)$$

$$\delta(d', x', x) = \delta(c, x', x) + \delta(\lfloor -rc \rfloor, x', x). \quad (8.5)$$

In effect, since Corollary 1 applies to the situation where $c_j = 0$ for $j \in NB$, it also allows each d'_j coefficient for $j \in NB$ to be identified by reference to the quantity that results by multiplying the vector of optimal dual values by the corresponding column A^j of the matrix A defining the constraints of (MIP), excluding rows of A corresponding to the inequalities $1 \geq x_j \geq 0$. (We continue to assume this matrix is enlarged by reference to additional inequalities such as (7) or (8) that may currently be included in defining $x \in X$.)

Now we establish the result that (8) is at least as strong as (7).

Hence by substituting (8.4) and (8.5) in the inequality (8) we obtain

$$\delta(c, x', x) + \delta(\lfloor -rc \rfloor, x', x) \geq \lceil \delta(c, x', x'') + \delta(\lfloor -rc \rfloor, x', x'') \rceil = \delta(c, x', x'') + \delta(\lfloor -rc \rfloor, x', x'').$$

Thus by using (8.3) we obtain (7). Consequently, this establishes that (8) implies (7).

Corollary 4. If the basic solution x'' for $LP(x', c)$ is optimal then

$$\delta(d', x', x) - \lceil \delta(d', x', x'') \rceil = \delta(c, x', x) - \lceil \delta(c, x', x'') \rceil.$$

Proposition 8. If the basic solution x'' for $LP(x', c)$ is optimal, and thus yields a valid inequality (7), then the inequality (8) dominates (7).

Proof: We use the fact that x'' is optimal for $LP(x', d')$ as established by Proposition 7. When x'' is optimal for $LP(x', c)$ from the optimal condition of the corresponding dual we have

$$rc x'' \leq rc x \text{ for all } x \geq 0$$

Thus we have

$$\lfloor -rc \rfloor x'' \geq \lfloor -rc \rfloor x \text{ for all } x \geq 0 \quad (8.2)$$

Since

$$\begin{aligned} \delta(\lfloor -rc \rfloor, x', x'') &= \lfloor -rc \rfloor x'' + \lfloor -rc \rfloor x' \\ \delta(\lfloor -rc \rfloor, x', x) &= \lfloor -rc \rfloor x + \lfloor -rc \rfloor x' \end{aligned}$$

This with (8.2) implies that

Proof: If the basic solution x'' for $LP(x', c)$ is optimal then $\delta(\lfloor -rc \rfloor, x', x'')$ is integer so $\delta(\lfloor -rc \rfloor, x', x'') = \lceil \delta(\lfloor -rc \rfloor, x', x'') \rceil$ which implies that $\delta(d', x', x) - \lceil \delta(d', x', x'') \rceil = \delta(c, x', x) - \lceil \delta(c, x', x'') \rceil$.

As in the use of the inequality (7), if a basic solution x'' that generates (8) is a binary vector that differs from x' , then we can also test x'' for feasibility in (MIP:0-1) and then redefine $x' = x''$, to additionally append the constraint (1.1) for this new x' .

The combined arguments of the proofs of Propositions 7 and 8 lead to a still stronger conclusion. Consider a linear program $LP(x', h')$ given by

$LP(x', h')$: Minimize $(\delta(h', x', x): x \in X)$.
where the coefficients $h'_j = d'_j$ (and hence $= c_j$) for $j \in B$ and, as before, B is defined relative to a given y -optimal basic feasible solution x'' . Subject to this condition, the only restriction

on the h'_j coefficients for $j \in NB$ is that they be integers. Then we can state the following result.

Corollary 5. *The x'' basis is an optimal LP basis for $LP(x', h')$ if and only if*

$$h'_j \geq d'_j \text{ for } j \in NB$$

and the inequality (8) dominates the corresponding inequality derived by reference to $LP(x', h')$.

Proof: *Immediate from the proofs on Propositions 7 and 8.*

The importance of Corollary 5 is the demonstration that (8) is the strongest possible valid inequality from those that can be generated by reference to a given y -optimal basic solution x'' and an objective function that shares the same coefficients for the basic variables.

It is to be noted that if (MIP:0-1) contains an integer valued slack variable s_i upon converting the associated inequality $A_i x + D_i y \geq b_i$ of the system $Ax + Dy \geq b$ into an equation – hence if A_i and b_i consist only of integers and D_i is the 0 vector – then s_i may be treated as one of the components of the vector x in deriving (8), and this inclusion serves to sharpen the resulting inequality. In the special case where all slack variables have this form, i.e., where (MIP:0-1) is a pure integer problem having no continuous variables and all data are integers, then it can be shown that the inclusion of the slack variables within x yields an instance of (8) that is equivalent to a fractional Gomory cut, and a stronger inequality can be derived by means of the foundation-penalty cuts of Glover and Sherali (2003). Consequently, the primary relevance of (8) comes from the fact that it applies to mixed integer as well as pure integer problems, and more particularly provides a useful means for enhancing target objective strategies for these problems. As an instance of this, we now examine methods that take advantage of (8) in additional ways by

extension of ideas proposed with parametric tabu search.

5. GENERATING TARGET OBJECTIVES AND SOLUTIONS BY EXPLOITING PROXIMITY

We now examine the issue of creating the target solution x' and associated target objective $\delta(c, x', x)$ that underlies the inequalities of the preceding sections. This is a key determinant of the effectiveness of targeting strategies, since it determines how quickly and effectively such a strategy can lead to new integer feasible solutions.

In this section, we propose a relatively simple approach for generating the vector c of the target objective by exploiting proximity. The proximity procedure for generating target solutions x' and associated target objectives $\delta(c, x', x)$ begins by solving the initial problem (LP), and then solves a succession of problems $LP(x', c)$ by progressively modifying x' and c . Beginning from the linear programming solution x'' to (LP) (and subsequently to $LP(x', c)$), the new target solution x' is derived from x'' simply by setting $x'_j = \langle x''_j \rangle$, $j \in N$, where $\langle v \rangle$ denotes the nearest integer neighbour of v . (The value $\langle v \rangle$ can be either 0 or 1, by employing an arbitrary tie-breaking rule.)

Since the resulting vector x' of nearest integer neighbors is unlikely to be feasible for (MIP:0-1), the critical element is to generate the target objective $\delta(c, x', x)$ so that the solutions x'' to successively generated problems $LP(x', c)$ will become progressively closer to satisfying integer feasibility.

If one or more integer feasible solutions is obtained during this approach, each such solution qualifies as a new best solution x^* , due to the incorporation of the objective function constraint $x_o \leq U_o < x_o^*$.

The criterion of the proximity procedure that selects the target solution x' as a nearest integer neighbor of x'' is evidently myopic. Consequently, the procedure is intended to be executed for only a limited number of iterations.

However, the possibility exists that for some problems the target objectives of this approach may quickly lead to new integer solutions without invoking more advanced rules. To accommodate this eventuality, we include the option of allowing the procedure to continue its execution as long as it finds progressively improved solutions.

The proximity procedure is based on the principle that some variables x_j should be more strongly induced to receive their nearest neighbor target values x'_j than other variables. In the absence of other information, we may tentatively suppose that a variable whose LP solution value x''_j is already an integer or is close to being an integer is more likely to receive that integer value in a feasible integer solution. Consequently, we are motivated to choose a target objective $\delta(c, x', x)$ that will more strongly encourage such a variable to receive its associated value x'_j . However, the relevance of being close to an integer value needs to be considered from more than one perspective.

5.1 Batwing Function for Proximity

The targeting of $x_j = x'_j$ for variables whose values x''_j already equal or almost equal x'_j does not exert a great deal of influence on the solution of the new LP(x', c), in the sense that such a targeting does not drive this solution to differ substantially from the solution to the previous LP(x', c). A more influential targeting occurs by emphasizing the variables x_j whose x''_j values are more “highly fractional,” and hence which differ from their integer neighbours x'_j by a greater amount. There are evidently trade-offs to be considered in the pursuit of influence, since a variable whose x''_j value lies close to .5, and hence whose integer target may be more influential, has the deficiency that the likelihood of this integer target being the “right” target is less certain. A compromise targeting criterion is therefore to give greater emphasis to driving x_j to an integer value if x''_j lies “moderately” (but not exceedingly) close to an integer value. Such a criterion affords an improved chance that the targeted value will

be appropriate, without abandoning the quest to identify targets that exert a useful degree of influence. Consequently, we select values λ_0 and $\lambda_1 = 1 - \lambda_0$ that lie moderately (but not exceedingly) close to 0 and 1, such as $\lambda_0 = 1/5$ and $\lambda_1 = 4/5$, or $\lambda_0 = 1/4$ and $\lambda_1 = 3/4$, and generate c_j coefficients that give greater emphasis to driving variables to 0 and 1 whose x''_j values lie close to λ_0 and λ_1 .

The following rule creates a target objective $\delta(c, x', x)$ based on this compromise criterion, arbitrarily choosing a range of 1 to 21 for the coefficient c_j . (From the standpoint of solving the problem LP(x', c), this range is equivalent to any other range over positive values from v to $21/v$, except for the necessity to round the c_j coefficients to integers.)

Proximity Rule for Generating c_j :

Choose λ_0 from the range $.1 \leq \lambda_0 \leq .4$, and let $\lambda_1 = 1 - \lambda_0$.

If $x'_j = 0$ (hence $x''_j \leq .5$) then

If $x''_j \leq \lambda_0$, set $c_j = 1 + 20x''_j / \lambda_0$

Else set $c_j = 1 + 20(.5 - x''_j) / (.5 - \lambda_0)$

Else if $x'_j = 1$ (hence $x''_j \geq .5$) then

If $x''_j \leq \lambda_1$, set $c_j = 1 + 20(x''_j - .5) / (\lambda_1 - .5)$

Else set $c_j = 1 + 20(1 - x''_j) / (1 - \lambda_1)$

End if

Finally, replace the specified value of c_j by its nearest integer neighbour $\langle c_j \rangle$

Remark 7: $c_j = 1$ if $x'_j = x''_j$.

The values of c_j coefficients produced by the preceding rule describe what may be called a *batwing* function—a piecewise linear function resembling the wings of a bat, with shoulders at $x''_j = .5$, wing tips at $x''_j = 0$ and $x''_j = 1$, and the angular joints of the wings at $x''_j = \lambda_0$ and $x''_j = \lambda_1$. Over the x''_j domain from the left wing tip at 0 to the first joint at λ_0 , the function ranges from 1 to 21, and then from this joint to the left shoulder at .5 the function ranges from 21 back

to 1. Similarly, from right shoulder, also at .5, to the second joint at λ_1 , the function ranges from 1 to 21, and then from this joint to the right wing tip at 1 the function ranges likewise from 21 to 1. (The coefficient c_j takes the negative of these absolute values from the right shoulder to the right wing tip.).

In general, if we let *Tip*, *Joint* and *Shoulder* denote the c_j values to be assigned at these junctures (where typically *Joint* > *Tip*, *Shoulder*), then the generic form of a batwing function results by replacing the four successive c_j values in the preceding method by

$$\begin{aligned} c_j &= \text{Tip} + (\text{Joint} - \text{Tip})x_j'' / \lambda_0, \\ c_j &= \text{Shoulder} + (\text{Joint} - \text{Shoulder})(.5 - x_j'') / (.5 - \lambda_0), \\ c_j &= \text{Shoulder} + (\text{Joint} - \text{Shoulder})(x_j'' - .5) / (\lambda_1 - .5) \\ c_j &= \text{Tip} + (\text{Joint} - \text{Tip})(1 - x_j'') / (1 - \lambda_1) \end{aligned}$$

The values of c_j coefficients called a *batwing* function can also be expressed as follows:

$$\begin{aligned} c_j &= \text{Tip} + (\text{Joint} - \text{Tip}) \delta(j, x', x'') / \lambda_0, \text{ if } x_j'' \notin [\lambda_0, 1 - \lambda_0] \\ c_j &= \text{Shoulder} + (\text{Joint} - \text{Shoulder})(.5 - \delta(j, x', x'')) / (.5 - \lambda_0), \text{ otherwise} \end{aligned}$$

The image of such a function more nearly resembles a bat in flight as the value of *Tip* is increased in relation to the value of *Shoulder*, and more nearly resembles a bat at rest in the opposite case. The function can be turned into a piecewise convex function that more strongly targets the values λ_0 and λ_1 by raising the absolute value of c_j to a power $p > 1$ (affixing a negative sign to yield c_j over the range from the right shoulder to the right wing tip). Such a function (e.g., a quadratic function) more strongly resembles a bat wing than the linear function.⁶

5.2 Design of the Proximity Procedure

We allow the proximity procedure that incorporates the foregoing rule for generating c_j the option of choosing a single fixed λ_0 value, or of choosing different values from the specified interval to generate a greater variety of outcomes. A subinterval for λ_0 centred around .2 or .25 is anticipated to lead to the best outcomes, but it can be useful to periodically choose values outside this range for diversification purposes.

We employ a stopping criterion for the proximity procedure that limits the total number of iterations or the number of iterations since finding the last feasible integer solution. In each instance where a feasible integer solution is obtained, the method re-solves the problem (LP), which is updated to incorporate both the objective function constraint $x_o \leq U_o < x_o^*$ and inequalities such as (8) that are generated in the course of solving various problems $\text{LP}(x', c)$. The instruction "Update the Problem Inequalities" is included within the proximity procedure to refer to this process of adding inequalities to $\text{LP}(x', c)$ and (LP), and to the associated process of dropping inequalities by criteria indicated in Section 2.

Proximity Procedure

1. Solve (LP). (If the solution x'' to the first instance of (LP) is integer feasible, the method stops with an optimal solution for (MIP:0-1).)
2. Construct the target solution x' derived from x'' by setting $x_j' = x_j''$, for $j \in N$. Apply the Rule for Generating c_j to each $j \in N$, to produce the vector c .
3. Solve $\text{LP}(x', c)$, yielding the solution x'' . Update the Problem Inequalities.
4. If x'' is integer feasible: update the best solution $(x^*, y^*) = (x'', y'')$, update $U_o < x_o^*$, and return to Step 1. Otherwise, return to Step 2.

A preferred variant of the proximity procedure does not change all the components of

c each time a new target objective is produced, but changes only a subset consisting of k of these components, for a value k somewhat smaller than n . For example, a reasonable default value for k is given by $k = 5$. Alternatively, the procedure may begin with $k = n$ and gradually reduce k to its default value.

This variant results by the following modification. Let c° identify the form of c produced by the Proximity Rule for Generating c_j , as applied in Step 2 of the Proximity Procedure. Re-index the x_j variables so that $c_1^\circ \geq c_2^\circ \geq \dots \geq c_n^\circ$, and let $J(k) = \{1, \dots, k\}$, thus identifying the variables x_j , $j \in J(k)$, as those having the k largest c_j° values. Then proximity procedure is amended by setting $c = 0$ in Step 1 and then setting $c_j = c_j^\circ$ for $j \in J(k)$ in Step 2, without modifying the c_j values for $j \in N - J(k)$. Relevant issues for research involve the determination of whether it is better to begin with k restricted or to gradually reduce it throughout the search, or to allow it to oscillate around a preferred value. Different classes of problems will undoubtedly afford different answers to such questions, and may be susceptible to exploitation by different forms of the batwing function (allowing different magnitudes for the Tip, Joint and Shoulder, and possibly allowing the location of the shoulders to be different than the .5 midpoint, with the locations of the joints likewise asymmetric).

6. CONCLUSION

Branch-and-bound ($B\&B$) and branch-and-cut ($B\&C$) methods have long been considered the methods of choice for solving mixed integer programming problems. This orientation has resulted in eliciting contributions to these classical methods from many researchers, and has led to successive improvements in these methods extending over a period of several decades. In recent years, these efforts to create improved $B\&B$ and $B\&C$ solution approaches have intensified and have produced significant benefits, as evidenced by the existence of MIP

procedures that are appreciably more effective than their predecessors.

It remains true, however, that many MIP problems resist solution by the best current $B\&B$ and $B\&C$ methods. It is not uncommon to encounter problems that confound the leading commercial solvers, resulting in situations where these solvers are unable to find even moderately good feasible solutions after hours, days, or weeks of computational effort. As a consequence, metaheuristic methods have attracted attention as possible alternatives or supplements to the more classical approaches. Yet to date, the amount of effort devoted to developing good metaheuristics for MIP problems is almost negligible compared to the effort being devoted to developing refined versions of the classical methods.

The view adopted in this paper is that metaheuristic approaches can benefit from a change of perspective in order to perform at their best in the MIP setting. Drawing on lessons learned from applying classical methods, we anticipate that metaheuristics can likewise profit from generating inequalities to supplement their basic functions. However, we propose that these inequalities be used in ways not employed in classical MIP methods, and indicate two principal avenues for doing this: first by generating the inequalities in reference to strategically created target solutions and target objectives, as in the current Part I, and second by embedding these inequalities in special intensification and diversification processes, as described in Part II.

ACKNOWLEDGMENT

The present research work has been supported by International Campus on Safety and Intermodality in Transportation, the Nord-Pas-de-Calais Region, the European Community, the Regional Delegation for Research and Technology, the Ministry of Higher Education and Research, the National Center for Scientific Research, and by a "Chaire d'excellence" from "Pays de la Loire" Region (France). A restricted

(preliminary) version of this work appeared in Glover (2008).

REFERENCES

- Balas, E., & Jeroslow, R. (1972). Canonical cuts on the unit hypercube. *SIAM Journal on Applied Mathematics*, 23(1), 60–69. doi:10.1137/0123007
- Dantzig, G. (1963). *Linear programming and extensions*. NJ: Princeton University Press.
- Fischetti, M., Glover, F., & Lodi, A. (2005). Feasibility pump. *Mathematical Programming - Series A*, 104, 91–104. doi:10.1007/s10107-004-0570-3
- Glover, F. (1978). Parametric branch and bound. *OMEGA: The International Journal of Management Science*, 6(2), 145–152.
- Glover, F. (2005). Adaptive memory projection methods for integer programming. In C. Rego and B. Alidaee (Eds.), *Metaheuristic optimization via memory and evolution: Tabu search and scatter search* (pp. 425–440). Dordrecht, The Netherlands: Kluwer Academic Publishers.
- Glover, F. (2006a). Parametric tabu search for mixed integer programs. *Computers & Operations Research*, 33(9), 2449–2494. doi:10.1016/j.cor.2005.07.009
- Glover, F. (2006b). *Satisfiability data mining for binary data classification problems*. Boulder: University of Colorado.
- Glover, F. (2007). Infeasible/feasible search trajectories and directional rounding in integer programming. *Journal of Heuristics*, 13(6), 505–542. doi:10.1007/s10732-007-9029-z
- Glover, F. (2008). Inequalities and target objectives for metaheuristic search – part I: Mixed binary optimization. In P. Siarry and Z. Michalewicz (Eds.), *Advances in metaheuristics for hard optimization* (pp. 439–474). New York: Springer.
- Glover, F., & Greenberg, H. (1989). New approaches for heuristic search: A bilateral linkage with artificial intelligence. *European Journal of Operational Research*, 39(2), 119–130. doi:10.1016/0377-2217(89)90185-9
- Glover, F., & Hanafi, S. (2002). Tabu search and finite convergence. *Discrete Applied Mathematics*, 119, 3–36. doi:10.1016/S0166-218X(01)00263-3
- Glover, F., & Laguna, M. (1997). *Tabu search*. Dordrecht, The Netherlands: Kluwer Academic Publishers.
- Glover, F., & Sherali, H. D. (2003). Foundation-penalty cuts for mixed-integer programs. *Operations Research Letters*, 31, 245–253. doi:10.1016/S0167-6377(03)00014-2
- Guignard, M., & Spielberg, K. (2003). *Double contraction, double probing, short starts and bb-probing cuts for mixed (0, 1) programming*. Philadelphia: Wharton School of the University of Pennsylvania.
- Hanafi, S., & Wilbaut, C. (2006). Improved convergent heuristics for the 0-1 multidimensional knapsack problem. *Annals of Operations Research*. doi:doi:10.1007/s10479-009-0546-z
- Hvattum, L. M., Lokketangen, A., & Glover, F. (2004). Adaptive memory search for boolean optimization problems. *Discrete Applied Mathematics*, 142, 99–109. doi:10.1016/j.dam.2003.06.006
- Nowicki, E., & Smutnicki, C. (1996). A fast taboo search algorithm for the job shop problem. *Management Science*, 42(6), 797–813. doi:10.1287/mnsc.42.6.797
- Soyster, A. L., Lev, B., & Slivka, W. (1978). Zero-one programming with many variables and few constraints. *European Journal of Operational Research*, 2(3), 195–201. doi:10.1016/0377-2217(78)90093-0
- Spielberg, K., & Guignard, M. (2000). *A sequential (quasi) hot start method for bb (0,1) mixed integer programming*. Paper presented at the Mathematical Programming Symposium, Atlanta.
- Ursulenko, A. (2006). *Notes on the global equilibrium search*. College Station, TX: Texas A&M University.
- Wilibaut, C., & Hanafi, S. (2009). New convergent heuristics for 0-1 mixed integer programming. *European Journal of Operational Research*, 195, 62–74. doi:10.1016/j.ejor.2008.01.044

ENDNOTES

- ¹ The vector c' depends on x' . As will be seen, we define several different linear programs that are treated as described here in reference to the problem $LP(x', c')$.

- ² An effective way to enforce $v_o = v_o''$ is to fix all non-basic variables having non-zero reduced costs to compel these variables to receive their optimal first stage values throughout the second stage. This can be implemented by masking the columns for these variables in the optimal first stage basis, and then to continue the second stage from this starting basis while ignoring the masked variables and their columns. (The masked non-basic variables may incorporate components of both x and y , and will generally include slack variables for some of the inequalities embodied in $Ax + Dy \geq b$.) The resulting residual problem for the second stage can be significantly smaller than the first stage problem, allowing the problem for the second stage to be solved very efficiently.
- ³ In some problem settings, the inclusion of the secondary objective x_o in $v_{oo} = Mv_o + x_o$ is unimportant, and in these cases our notation is accurate in referring to the explicit minimization of $v_o = c'x$.

⁴ This strategy is utilized in the parametric branch and bound approach of Glover (1978) and in the feasibility pump approach of Fischetti, Glover and Lodi (2005).

⁵ We continue to apply the convention of referring to just the x -component x'' of a solution (x'', y'') , understanding the y component to be implicit.

⁶ Calibration to determine a batwing structure, either piecewise linear or nonlinear, that proves more effective than other alternatives within Phase 1 would provide an interesting study.

Fred Glover is the chief technology officer in charge of algorithmic design and strategic planning initiatives for OptTek Systems, Inc., heading the development of commercial computer software systems currently serving more than 80,000 users in the United States and abroad. He also holds the title of distinguished professor at the University of Colorado, Boulder, where he is affiliated with the Leeds School of Business and the Department of Electrical and Computer Engineering. Dr. Glover is widely known for his work in the applications of computer decision support systems, including industrial planning, financial analysis, systems design, energy and natural resources planning, logistics, transportation and large-scale allocation models. He has authored or co-authored more than 370 published articles and eight books in the fields of mathematical optimization, computer science and artificial intelligence. He is also the originator of Tabu Search (Adaptive Memory Programming), an optimization search methodology of which more than 200,000 Web pages can be found with a simple Google search. Dr. Glover is the recipient of the highest honor of the Institute of Operations Research and Management Science, the von Neumann Theory Prize, and is an elected member of the National Academy of Engineering. He has also received numerous other awards and honorary fellowships, including those from the American Association for the Advancement of Science (AAAS), the NATO Division of Scientific Affairs, the Institute of Operations Research and Management Science (INFORMS), the Decision Sciences Institute (DSI), the U.S. Defense Communications Agency (DCA), the Energy Research Institute (ERI), the American Assembly of Collegiate Schools of Business (AACSB), Alpha Iota Delta, the Institute of Cybernetics of the Ukrainian Academy of Science, and the Miller Institute for Basic Research in Science.

Saïd Hanafi holds a full professor position in computing science at Institute of Techniques and Sciences, University of Valenciennes and is currently in charge of the team Operations Research and Decision Support. His research lies in the design of effective heuristic and metaheuristic algorithms for solving large-scale combinatorial search problems. He is interested in theoretical as well as algorithmic modelling and application aspects of integer programming and combinatorial optimisation and has published over 30 articles on the topic. His current interests revolve around the integration of tools from hybrid methods mixing exact and heuristics for solving hard problems.

A New Multiagent Algorithm for Dynamic Continuous Optimization

Julien Lepagnot, Université de Paris 12, France

Amir Nakib, Université de Paris 12, France

Hamouche Oulhadj, Université de Paris 12, France

Patrick Siarry, Université de Paris 12, France

ABSTRACT

Many real-world problems are dynamic and require an optimization algorithm that is able to continuously track a changing optimum over time. In this article, a new multiagent algorithm is proposed to solve dynamic problems. This algorithm is based on multiple trajectory searches and saving the optima found to use them when a change is detected in the environment. The proposed algorithm is analyzed using the Moving Peaks Benchmark, and its performances are compared to competing dynamic optimization algorithms on several instances of this benchmark. The obtained results show the efficiency of the proposed algorithm, even in multimodal environments.

Keywords: *Dynamic, Metaheuristic, Moving Peaks Benchmark, Multiagent, Multimodal Environment, Non-Stationary, Optimization, Time-Varying*

1. INTRODUCTION

Recently, optimization in dynamic environments has attracted a growing interest, due to its practical relevance. Many real-world problems are dynamic optimization problems (DOPs), i.e. their objective function changes over time: typical examples are in vehicle routing (Larsen, 2000), inventory management (Minner, 2003) and scheduling (Branke & Mattfeld, 2005). For dynamic environments, the goal is not only to

locate the optimum, but to follow it as closely as possible. A dynamic optimization problem can be expressed by:

$$\begin{aligned} \max \quad & f(\vec{x}, t) \\ \text{s.t.} \quad & h_j(\vec{x}, t) = 0 \text{ for } j = 1, 2, \dots, p \\ & g_k(\vec{x}, t) \leq 0 \text{ for } k = 1, 2, \dots, l \\ & \vec{x} = [x_1, x_2, \dots, x_n] \end{aligned} \quad (1)$$

where $f(\vec{x}, t)$ is the objective function of the problem, $h_j(\vec{x}, t)$ denotes the j^{th} equality

DOI: 10.4018/ijamc.2010102602

constraint and $g_k(\vec{x}, t)$ denotes the k^{th} inequality constraint. Both of them may change over time, denoted by t . In this article, we focus on a dynamic optimization problem with time constant constraints.

The main approaches to deal with DOPs can be classified into the following five groups (Jin & Branke, 2005):

1. **Generated diversity after a change:** When a change in the environment is detected, explicit actions are taken to increase diversity and to facilitate the shift to the new optimum.
2. **Maintained diversity throughout the run:** Convergence is avoided all the time and it is hoped that a spread-out population can adapt to change more efficiently.
3. **Memory-based approaches:** These methods are supplied with a memory to be able to recall useful information from the past. In practice, they store good solutions in order to reuse them when a change is detected.
4. **Multipopulation approaches:** Dividing up the population into several subpopulations, distributed on different optima, allows tracking of multiple optima simultaneously and increases the probability to find new ones.
5. **Future prediction:** Recently, another kind of methods, trying to predict future changes, has attracted much attention (Rossi, Abderrahim, & Diaz, 2008), (Rossi, Barrientos, & Cerro, 2007), (Simoes & Costa, 2008). This approach is based on the fact that in a real problem, changes can follow some pattern that could be learned.

We focus on population-based metaheuristics, which are global search, generally bio-inspired, algorithms. We can roughly classify them in four categories: evolutionary algorithms (EAs), particle swarm optimization (PSO), ant colony optimization (ACO) and hybrid methods. We will now describe dynamic metaheuristics that have been proposed in the literature in each of these categories.

A lot of existing dynamic optimization methods are EAs. EAs can be indeed well suited for optimization in changing environments, since they are inspired by the principles of natural evolution, and natural evolution deals very well with environmental changes. In (Rossi, Abderrahim, & Diaz, 2008), the algorithm incorporates a motion prediction technique based on Kalman filter, in order to improve the speed of optimum tracking. This kind of prediction could be suited for many problems, but it increases the complexity of the algorithm and the number of parameters. In (Tinos & Yang, 2007), the authors propose to maintain diversity in a genetic algorithm (GA) by replacing the worst individual and some others with randomly generated individuals, and maintaining their offspring in a subpopulation. However, this method is only designed for discrete problems (DDOPs). Another GA is described in (Yang, 2006), that is based on immune system and also devoted to DDOPs. In (Huang & Rocha, 2005), an agent-based coevolutionary GA is proposed. This algorithm makes coevolve the way the genotype of an agent is read along with its genotype. Thus, the chromosomes are transcribed into their "edited" counterparts, using the "editors" which coevolve with them, then crossed-over and mutated. In (Mendes & Mohais, 2005), a multipopulation differential evolution (DE) algorithm is proposed, in which some techniques are added in order to increase diversity. DE is a population-based approach, its strategy consists in generating a new position for an individual according to the differences calculated between other randomly selected individuals. This algorithm is based on two main parameters, that must be correctly fitted. However in (Mendes & Mohais, 2005), these parameters are randomly generated in order to make DE easier to use.

Another widely used class of algorithms for dynamic optimization is PSO. PSO is a population-based approach, similar in some respects to EAs, except that potential solutions (particles) move, rather than evolve, throughout the search space (Blackwell & Branke, 2004). The move rules, or particle dynamics, are inspired by

models of swarming and flocking (Kennedy & Eberhart, 1995). In this class of methods, we can cite (Blackwell & Branke, 2004), where the authors propose two multi-swarm algorithms based on an atomic model. The first algorithm uses multiple swarms, composed of a sub-swarm of mutually repelling particles, orbiting around another sub-swarm of neutral, or conventional PSO particles. The second algorithm is based on a quantum model of the atom, where the charged particles (electrons) will not follow a classical trajectory, but will be rather randomized within a ball centered on the swarm attractor. Both of these algorithms place their swarms on each of localized optima, thus letting each swarm track a different optimum. However, these methods have many parameters that must be suitably fitted. Moreover, the obtained results with these methods are not good enough. This approach of using charged particles is also used in (Blackwell & Branke, 2006) and (Li, Branke, & Blackwell, 2006), with other techniques to increase diversity and to track optima. Another PSO algorithm is proposed in (Du & Li, 2008), that uses two populations of particles. The first one is for diversification, and the second is for intensification.

Another class of algorithms, ACO, has been used for dynamic optimization. ACO is also a population-based approach using swarm intelligence, inspired from the way ants find the shortest path between the nest and a source of food (Dorigo & Gambardella, 2002). In this class of methods, we can cite (Dréo & Siarry, 2006), that proposes to hybridize an ACO algorithm with the Nelder-Mead simplex method (Nelder & Mead, 1965) for local search. A modified Nelder-Mead simplex method is also developed in this article for dynamic optimization. However, this method does not perform well on functions having a lot of local optima. In (Tfaily & Siarry, 2008), charged ants are used in order to maintain diversity. Moreover, the attribution of an electrostatic charge to each ant prevents them from converging to same local optima. The performances of this method are not good enough, according to the benchmark defined in (Dréo & Siarry, 2006).

In order to perform better on dynamic problems, some authors have tried hybrid methods, like in (Lung & Dumitrescu, 2007) and (Lung & Dumitrescu, 2008), that propose hybrid PSO/EAs algorithms. However, the results of those methods are not so different from PSO or EAs methods, described previously.

An algorithm proposed in (Moser & Hendtlass, 2007) is based on extremal optimization (EO). EO does not use a population of solutions, but improves a single solution using mutation. This algorithm uses a “stepwise” sampling scheme that samples every dimension of the search space in equal distances. Then, the algorithm takes the best candidate as the next solution, afterwards it proceeds to a hill-climbing phase, in order to fine-tune the solution. Then, the solution is stored in memory, and the method is applied again on another randomly generated solution. This algorithm is simple and efficient, and allows to obtain good results on a specific test called the “Moving Peaks Benchmark” (MPB) (Branke, 1999). However, this method is especially developed and fitted for this benchmark, and it seems not applicable in real world problems.

In this article, a new method is proposed to solve dynamic optimization problems. Our algorithm, called “MADO”, for “MultiAgent Dynamic Optimization”, has been developed in order to be efficient for solving a wide range of DOPs. It is a multiagent method, that makes use of a population of agents to explore the search space. The proposed algorithm is based on the following considerations: when optimizing in a multimodal environment, we need to keep track of each local optimum, to overcome the case where the global optimum “jumps” from one of them to another. The found optima are archived in a memory. Then, this memory can be used when a change is detected.

It is common that real world problems have time costly evaluation of fitness functions. Hence, the computational cost of the proposed algorithm should be made as short as possible and expressed in terms of number of evaluations. The proposed algorithm makes use of the following inspirations:

- Keeping information about the previous positions allows to prevent wasting evaluations in unpromising zones of the search space;
- Using a sampling of candidate solutions that optimally cover the local landscape may reduce the number of evaluations per trajectory step, without decreasing performance. For this goal, we maximize the distances between all the candidate solutions, whereas keeping them constricted into the current local landscape. More precisely, it is done by evaluating the candidate solutions on the surface of an hypersphere centered on the current position;
- The sampling of candidate solutions is adaptable to the local landscape. This is done by using an adaptable radius for the above-mentioned hypersphere;
- An exclusion radius for the current solution of each search trajectory is used to avoid the case where some search trajectories explore the same area of the search space.

The rest of this article is organized as follows. Section 2 presents test problems used in the literature and introduces the benchmark set that will be used here. Section 3 describes the proposed algorithm. Experimental results are discussed in Section 4. Conclusion and works in progress are in Section 5.

2. BENCHMARK SET FOR DYNAMIC OPTIMIZATION

Table 1 presents a summary of competing methods available in the literature and gives the test problems used by each one. The test problems gathered in the first column of Table 1 are those used by the competing methods. These competing methods are described in the references listed in the third column. From Table 1, one can remark that the most commonly used testbed is the Moving Peaks Benchmark (MPB) (Branke, 1999). This benchmark is becoming

the standard for testing dynamic optimization algorithms, and is claimed to be representative of real world problems (Branke, 1999). To compare our algorithm to the competing ones, this testbed was adopted.

MPB consists of a number of peaks that vary their shape, position and height randomly upon time. At any time, one of the local optima becomes the new global optimum. MPB generates DOPs consisting of a set of peaks that periodically move in a random direction, by a fixed amount s (the change “severity”). The movements are autocorrelated by a coefficient $0 \leq \lambda \leq 1$, where 0 means uncorrelated and 1 means highly autocorrelated. The peaks change position every α iterations (function evaluations), and α is called time span. The fitness function for the landscape of MPB is formulated as follows:

$$F(\vec{x}, t) = \max_{i=1, \dots, m} \left(H_i(t) - \left(W_i(t) \sqrt{\sum_{j=1}^d (x_j - X_{ij}(t))^2} \right) \right) \quad (2)$$

where m is the number of peaks, d is the number of dimensions, $H_i(t)$ is the height of the i^{th} peak at the time t , $W_i(t)$ is the width of the i^{th} peak at the time t and $\vec{X}_i(t)$ is the position of the i^{th} peak at the time t .

Figure 1 illustrates an MPB landscape before and after a change (after one time span).

In order to evaluate the performance, the “offline error” is used. Offline error (oe) is defined as the average of the errors of the best points evaluated during each time span. It is defined by:

$$oe = \frac{1}{Nc} \sum_{j=1}^{Nc} \left(\frac{1}{Ne(j)} \sum_{i=1}^{Ne(j)} (f_j^* - f_{ji}^*) \right) \quad (3)$$

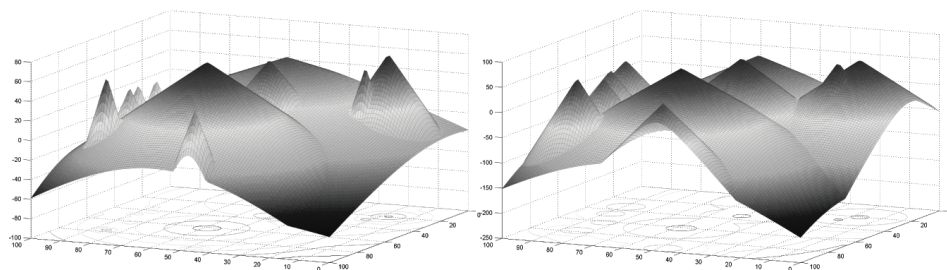
where Nc is the total number of fitness landscape changes within a single experiment, $Ne(j)$ is the number of iterations performed for the j^{th} state of the landscape, f_j^* is the value

Table 1. Summary of recent DOP algorithms

Test problems	Base algorithm	Authors
MPB ¹ and the dynamic Rastrigin function	PSO ²	Du & Li, 2008
various continuous dynamic problems (Dréo & Siarry, 2006)	ACO ³	Tfaily & Siarry, 2008
a computer vision problem	EAs ⁴	Rossi, Abderrahim, & Diaz, 2008
MPB	Hybrid PSO/EAs	Lung & Dumitrescu, 2008 (<i>ESCA</i>)
MPB	Hybrid PSO/EAs	Lung & Dumitrescu, 2007 (<i>CESO</i>)
MPB	EO ⁵	Moser & Hendtlass, 2007
a set of discrete dynamic problems (Yang, 2003), (Yang & Yao, 2005)	GA ⁶	Tinos & Yang, 2007
a set of discrete dynamic problems (Yang, 2003), (Yang & Yao, 2005)	GA	Yang, 2006
various continuous dynamic problems (Dréo & Siarry, 2006)	ACO	Dréo & Siarry, 2006
MPB	PSO	Li, Branke, & Blackwell, 2006
MPB	PSO	Blackwell & Branke, 2006 (<i>2nd version</i>)
MPB	DE ⁷	Mendes & Mohais, 2005
a testbed proposed by the authors	EAs	Huang & Rocha, 2005
MPB	PSO	Blackwell & Branke, 2004 (<i>1st version</i>)

¹ The Moving Peaks Benchmark (Branke, 1999). ² Particle Swarm Optimization. ³ Ant Colony Optimization. ⁴ Evolutionary Algorithms. ⁵ Extremal Optimization. ⁶ Genetic Algorithm. ⁷ Differential Evolution.

Figure 1. An MPB landscape before and after a change



of the optimal solution for the j^{th} landscape and f_{ji}^* is the current best fitness value found for the j^{th} landscape.

We can remark that this measure has some weaknesses: it is sensitive to the overall height of the landscape, and to the number of peaks. It is important for an algorithm to find the global optimum quickly, to minimize the offline error. Hence, the most successful strategy is a multi-solution approach that keeps track of every local peak (Moser & Hendtlass, 2007).

3. THE PROPOSED MADO ALGORITHM

3.1 Overall Scheme

MADO is a multiagent algorithm that consists in the following three modules:

1. **Memory module:** In case of a multimodal environment, a dynamic optimization method needs to keep track of each local optimum found, since one of them can become the new global optimum after a change. We propose to use a memory to archive the found optima.
2. **Agent manager:** It contains all the agents, and manages their execution, creation and deletion. Agents are nearsighted (they have only a local vision of the search space). More precisely, agents are only performing local search, they jump from their current position to a better one, in their neighborhood, until they cannot improve their current solution, reaching thus a local optimum.
3. **Coordinator:** It counterbalances the nearsightedness of the agents. The coordinator has indeed a global vision of the search performed by the agents, and it is able to prevent them from searching in unpromising zones of the search space. The coordinator supervises the whole search, and

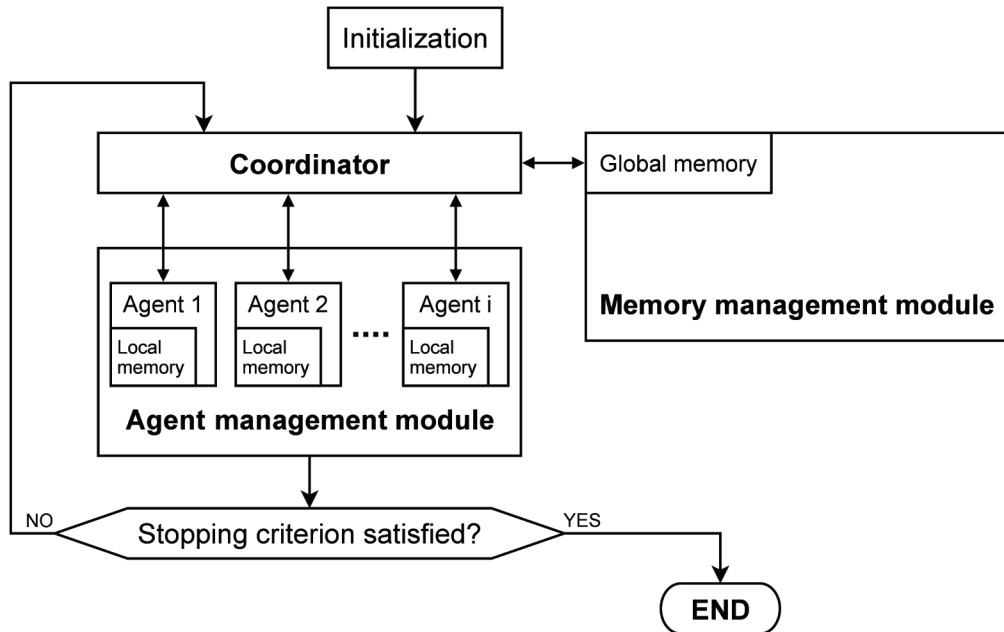
manages the interactions between memory and agents modules.

In the following subsections, a detailed description of the overall scheme of the method is presented. Subsequently, the three modules that compose the MADO algorithm are described. Afterwards, the fitting of the parameters of the algorithm is presented, and their values, further used in the experimental section, are given.

The overall scheme of the MADO algorithm is illustrated in Figure 2. As it is shown, all the interactions between the memory manager and the agent manager are through the coordinator. Moreover, all global decisions are taken by the coordinator. The memory manager maintains the archive of local optima, that are provided by the coordinator. The agent manager informs the coordinator about the found optima, and receives its instructions for creating, deleting, and repositioning agents.

Before explaining in detail these modules, we need to describe how the distances are calculated in MADO. As we are considering an Euclidian space of d dimensions as the search space, the Euclidian distance will be used. However, the search space may not have the same bounds on each dimension. Then, we will use a “normalized” basis. We denote by Δ_i the size of each interval that defines the search space, with $i \in [1, d]$. Then, the unit vectors $\vec{e}_i$ in the direction of each axis of the Cartesian coordinates system are “scaled” in order to produce modified unit vectors u_i , that make all the Δ_i equal to 1 in the basis defined by these modified unit vectors. This is done by using $\{\vec{u}_1 = \frac{\vec{e}_1}{\Delta_1}, \vec{u}_2 = \frac{\vec{e}_2}{\Delta_2}, \dots, \vec{u}_d = \frac{\vec{e}_d}{\Delta_d}\}$ as the basis of the Euclidian space where the search space is defined. Consequently, when an hypersphere inside the search space is considered, it will correspond to an ellipsoid in the canonical basis.

Figure 2. Overall scheme of MADO algorithm



3.2 Agent Management Module

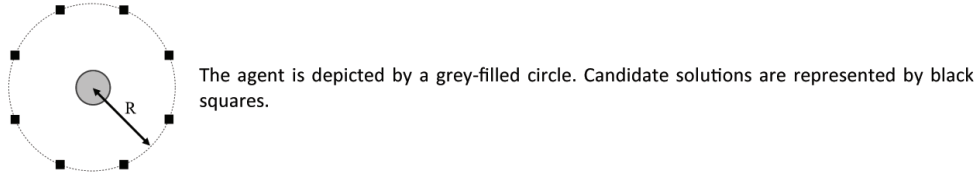
3.2.1 The Exploration Strategy of the Agents

Agents explore the search space step-by-step, moving from their current position to a better one in their neighborhood, until they reach a local optimum. Hence, to precisely describe the behavior of the agents, we first explain the used kind of neighborhood, and how the agents make use of it. As agents are nearsighted, they can only test candidate solutions for their next move in a delimited zone of the search space, centered on their current position. This zone must be bounded. Without any information about the local landscape of an agent, a good choice is to make it isotropic. Then, the most adapted topology is a ball. To keep small the number of fitness function evaluations, we need a sampling that optimally covers the local landscape. Afterwards, we maximize the distances between all candidate solutions inside this

ball. It leads to sample them on the boundary of the ball (on a hypersphere centered on the current solution of an agent). This delimited zone of the search space must be adaptive to the local landscape, to increase the efficiency of the agents. It is done by using an adaptive radius for the ball that defines this zone. From all these considerations, we define the neighborhood of the agents as a set of N candidate solutions placed on the hypersphere of radius R centered on the current position of an agent (N is a parameter of the algorithm). Figure 3 illustrates this neighborhood in dimension 2. Another set of N points are generated and stored at the beginning of the MADO algorithm. These points are calculated in order to meet the following constraints:

- The smallest distance between them is maximized;
- The distances between them and the origin of the space (the point $(0,0,...,0)$) are equal to 1 ;

Figure 3. The sampling of candidate solutions of an agent, when $d = 2$ and $N = 8$



- The number of dimensions of the space in which they are defined match the number of dimensions of the search space.

This computation is done using the well known electrostatic repulsion heuristic (Conway & A., 1998), that considers points as charged particles repelling each other. Starting from an arbitrary distribution, it uses point repulsion, where all points are considered to repel each other according to a $1/r^2$ force law, with r the distance between two points, and dynamics are simulated. The algorithm runs until the satisfaction of a stopping criterion, and the resulting set of points is returned. We use a stopping criterion that is satisfied when no point can be moved by a distance greater than ε , typically equal to 10^{-4} . Then, this set of points is used by agents to get the positions of the candidate

solutions of their current neighborhood. The procedure used to get the exact positions of candidate solutions is presented in Algorithm 1; where S' is the computed set of an agent's candidate solutions, d is the number of dimensions of the search space, S is the precalculated set of uniformly spaced N points on the unit hypersphere and $\vec{P}_c$ is the current position of the agent. We can note that, for $d = 1$, the only possible set of points for S is $\{(-1), (1)\}$ and agents can only move forward or backward by steps of length equal to R . In this particular case, the generated unit vector $\vec{V}$ can only be (-1) or (1) . The use of the Householder reflection (Householder, 1958) is required here in order to sample candidate solutions on the whole surface of the unit hypersphere. It allows generating from S new sets of uniformly spaced

Algorithm 1. Calculates the set of candidate solutions of an agent

```

 $S' \leftarrow \emptyset$ 
 $\vec{V} \leftarrow$  a random unit vector uniformly distributed on the hypersphere of radius
1, of  $d$  dimensions and centered on the origin
foreach point  $\vec{P}_i \in S$  do
     $\vec{P}_i^r \leftarrow$  the Householder reflection of  $\vec{P}_i$  which uses  $\vec{V}$  as the Householder vector
    (to randomly reflect  $\vec{P}_i$ )
     $\vec{P}_i^{rs} \leftarrow \vec{P}_i^r \cdot R$  (to scale  $\vec{P}_i^r$  by  $R$ )
     $\vec{P}_i^{rst} \leftarrow \vec{P}_i^{rs} + \vec{P}_c$  (to translate  $\vec{P}_i^{rs}$  on the agent's local landscape hypersphere)
    if  $\vec{P}_i^{rst}$  is inside the search space then (if it is not beyond the boundary of
    the search space)
         $S' \leftarrow S' \cup \{\vec{P}_i^{rst}\}$ 
    end
end
return  $S'$ 

```

points on this hypersphere, rather than directly using the predefined set S . Moreover, the use of the precalculated set S allows to perform the electrostatic repulsion heuristic only one time (at the beginning of the algorithm), rather than each time a new set of candidate solutions is generated. This way, it saves a significant amount of computation time.

3.2.2 The Step Size Adaptation Strategy

The adaptation of the radius R makes use of trajectory information gathered along the steps of an agent. We propose to use the “cumulative path length control” described in (Hansen & Ostermeier, 2001), with much simpler calculations. The Figure 4 illustrates the two possible kinds of “bad” trajectories that an agent may follow. When one of these cases is detected, an increase or a decrease of the step size R is performed. The first case (Figure 4(a)) may mean that the agent is turning around an optimum, without being able to reach it directly. This is due to a too large step size, leading to back and forth displacements, which cancel each other out. Thus, in this first case, a decrease in R is needed. On the contrary, the displacements on Figure 4(b) are oriented in the same direction, which may

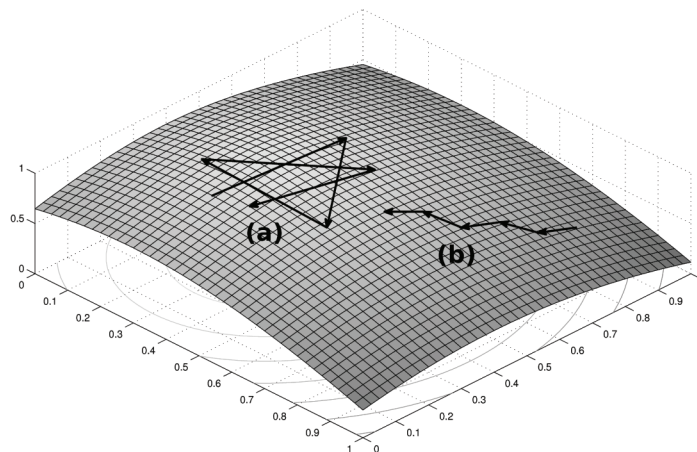
mean that the agent is hill climbing a large peak (a peak is considered to be large when, climbed by an agent, the statement $U_n > s_u$ becomes true (see equation (5) and Algorithm 2)). Then, to avoid too slow moves to the top of this peak, an increase in R is performed.

Information about the successive moves of an agent is collected by using a “cumulative dot product” of old successive displacement vectors. It is done using the following formula:

$$U_n = \begin{cases} \sum_{i=1}^{n-2} \left(c_u \right)^{n-i-2} \frac{\vec{D}_{i,i+1} \cdot \vec{D}_{i+1,i+2}}{\|\vec{D}_{i,i+1}\| \|\vec{D}_{i+1,i+2}\|} & \text{if } n > 2 \\ 0 & \text{otherwise} \end{cases} \quad (4)$$

where U_n is the cumulative dot product of an agent’s search trajectory, n is the number of successive positions of this trajectory, $\vec{D}_{i,i+1}$ is the displacement vector from the i^{th} position to the $(i+1)^{th}$ position in this trajectory, and c_u is a constant in $[0, 1]$, where 0 gives no weight to displacements before the last one, whereas 1 applies the same weight to the last displacement and to its previous ones. In practice, U_n

Figure 4. The two kinds of trajectories that lead to a step size adaptation



(a) Agent turning around an optimum. (b) Agent hill climbing a large peak.

is computed using a recurrence relation on U_{n-1} and on the two last displacements (see equation (5)). There is no need to record all the displacement vectors of the agents, but only their two last ones.

$$U_n = \begin{cases} c_u U_{n-1} + \frac{\vec{D}_{n-2,n-1} \cdot \vec{D}_{n-1,n}}{\|\vec{D}_{n-2,n-1}\| \|\vec{D}_{n-1,n}\|} & \text{if } n > 2 \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

Then, U_n is used to determine the decision for the step size adaptation. The first case Figure 4(a) tends to produce negative dot products, whereas the second case tends to produce positive ones. Hence, when the value of U_n falls beyond a negative threshold, the step size of the agent will decrease, and when the value of U_n exceeds a positive threshold, the step size of the agent will increase (Algorithm 2).

Algorithm 2. Adaptation of the step size using the cumulative dot product

```

if  $U_n < -s_u$  then
     $R \leftarrow c_r R$ 
     $U_n \leftarrow 0$ 
else if  $U_n > s_u$  then
     $R \leftarrow \frac{1}{c_r} R$ 
     $U_n \leftarrow 0$ 
end
    
```

where c_r is constant in $]0,1]$, and s_u is a constant threshold, calculated as follows:

$$s_u = \cos\left(\frac{\pi}{3}\right) (1 + c_u) \quad (6)$$

This formula can be explained by the following considerations. In the case of $c_u = 0$, the value of U_n (see equation (5)) will be equal

to the cosinus of the angle, denoted a , between the last two displacement vectors of an agent. As a can take any value in $[0, \pi]$, this interval has been uniformly partitioned and linked to the three following cases:

1. The agent follows a trajectory as in Figure 4(a). This case produces the highest values of a , and is associated to the interval $\left[\frac{2\pi}{3}, \pi\right]$;
2. The temporary situation, where we cannot take a decision about the agent. Hence, no step size adaptation is made in this case. It produces values of a around $\frac{\pi}{2}$, and it is related to the interval $\left[\frac{\pi}{3}, \frac{2\pi}{3}\right]$;
3. The agent follows a trajectory as in Figure 4(b). This leads to the lowest values of a , and this last case corresponds to the interval $\left[0, \frac{\pi}{3}\right]$.

This case ($c_u = 0$) corresponds to the case where the threshold value s_u is equal to $\cos\left(\frac{\pi}{3}\right)$. Generally, c_u is not equal to 0, but can take any value in the interval $]0,1]$. We propose to adjust this threshold using equation (6).

The agents step size adaptation assumes that the agents are always able to move to a better position from their current one. This is a condition to be able to compute U_n from U_{n-1} and the last displacement. If an agent cannot find a better solution in its local landscape, the agent stays on its current position. Then, a second adaptation method is proposed in this particular case. This method is based on the following assumption: if the agent cannot find a better solution in its local landscape, then, it has probably finished hill climbing a peak, and an optimum may lay at a distance lower than the current radius of its neighborhood (the hypersphere of radius R). Then, reducing this

radius may let the agent continue its search towards the nearby optimum. Briefly, the second adaptation method consists in decreasing R in order to let the agent converge to the found optimum ($R \leftarrow c_r R$).

3.2.3 The Convergence Process and the Stagnation Criterion

We consider that an agent converged to a local optimum, when none of its last δ_t steps improved significantly the fitness value of its current solution (if $|f_n - f_{n-1}| \leq \delta_p$ for $n = 1, 2, \dots, \delta_t$ where f_n is the fitness of the solution found by the agent n trajectory steps ago). This is a well known stagnation criterion for stopping a local search. We use the parameter δ_t to prevent unnecessary fitness evaluations. An adaptation method for the number of neighbor solutions, that have to be evaluated in order to find the new position of an agent, is based on the following considerations. When an agent is close to a local optimum, it has a lower probability to get out of this local optimum than an agent that is still exploring the landscape. The agent will not be likely to escape from the basin of attraction (the region of the search space, around an optimum, that leads to trajectories towards this optimum) it has converged to, even if it performs a lot of fitness evaluations. However, for an agent that is still exploring the landscape, it is important to maintain a good trajectory, in order to hill climb towards a high quality solution. The convergence level of an agent can be expressed by the number of its last d_t steps that did not improve its fitness more than δ_p . The Algorithm 3 summarizes the procedure to find the next position of an agent.

Algorithm 3. The search for a better solution among the neighboring ones

```

 $v \leftarrow v_c$ 
 $\vec{P}_{best} \leftarrow \vec{P}_c$ 

```

```

foreach point  $\vec{P}_i^{rst} \in S'$  do
   $v_i \leftarrow \text{evaluate } \vec{P}_i^{rst}$ 
  if  $v_i > v$  then
     $v \leftarrow v_i$ 
     $\vec{P}_{best} \leftarrow \vec{P}_i^{rst}$ 
  if  $\text{rand}(\delta_t) < d_t$  then
    return( $\vec{P}_{best}, v$ )
end
end
return( $\vec{P}_{best}, v$ )

```

where v_c is the current fitness of the agent, $\vec{P}_c$ is its current position, S' is the set of all neighbor candidate solutions of the agent, $\vec{P}_i^{rst}$ is a reflected scaled translated position of a given solution, $\text{rand}(\delta_t)$ generates a uniform random number in $[0, \delta_t]$, d_t is the number of steps that the agent has made without improving its fitness more than δ_p , and $\vec{P}_{best}$ and v are the new solution of the agent in its search trajectory and its new fitness value, respectively.

When an agent has finally found a local optimum, the agent manager sends this optimum to the coordinator, that will transmit it to the memory manager. Then, the coordinator shows to the agent manager where this agent will be repositioned, in order to perform a new trajectory search. A new value is given to the step size (R) of this agent. The way the new step size value is calculated is described in subsection 3.4. One can remark that in a flat landscape, agents will not move from their initial position (their trajectory will consist in a succession of the same initial point), and stop their local search after δ_t steps. Thus, in a flat landscape, the local optima transmitted to the memory manager will be always the initial positions of the agents. In this particular case, the only adaptation of the step size of an agent is a decrease in R , since no displacement vectors can be computed (the agent does not move).

3.2.4 The Diversity Maintaining Strategy

To prevent several agents from exploring the same zone of the search space, and to prevent them from converging to the same local optimum, an exclusion radius r_e is attributed to each agent. Hence, when an agent detects one or several other agents at a distance lower than r_e , only the agent with the best fitness, among the detected agents and the agent having detected them, is allowed to continue its search. All the other agents have to start a new search elsewhere (see subsection 3.4 to see how they are repositioned). The value of r_e is dynamically adjusted each time a new local optimum is memorized, and its new value is calculated by the memory manager. To prevent that r_e becomes too small, a given radius r_l needs to be introduced. This radius will be used as a lower bound for r_e . It is required, in order to avoid the use of near zero values for r_e and as initial step size. Too small values for these radius will slow the adaptation processes. The update process of r_e is administrated by the coordinator, and executed by the memory manager.

3.2.5 The Flowchart of an Agent

Agents perform their search by running their local search algorithm independently of each other. The flowchart of the agents search algorithm is illustrated in Figure 5. One can remark that a special state named "SYNCHRONIZATION" appears two times in this flowchart. This state marks the end of one "loop" of the algorithm of an agent. Hence, when this state is reached, the agent manager halts the execution of this agent until all other agents have reached a SYNCHRONIZATION state. Then, the execution of the agents is resumed at the SYNCHRONIZATION state they previously halted on (not at another SYNCHRONIZATION state). This special state allows then the parallel execution of the agents.

3.3 Memory Management Module

3.3.1 Local Optima Archiving Strategy

The memory manager maintains the archive of local optima found by the agents. This archive must be bounded, its size is fixed by a predefined number n_m of entries. Thus, all the found optima cannot be included into this archive, only the n_m best ones will be stored. When the archive is full, we propose the following condition to update the archive:

- If the new optimum is better than the worst optimum of the archive, or its fitness value is at least equal to the one of this worst optimum, then this worst optimum is replaced by the new one;
- If there is one or several other optima in the archive that are "too close" to the new optimum, then it is possible that all these optima close to each other are in fact scattered around the top of one peak. Thus, this subset of solutions is replaced by the best optimum among them. Hence, if there are other optima in the archive that are "too close" to the newly found one, this new optimum will replace them, only if it is better, or if its fitness is at least equal to the fitness of the best one. The process of detecting possible "too close" optima is presented in Algorithm 4.

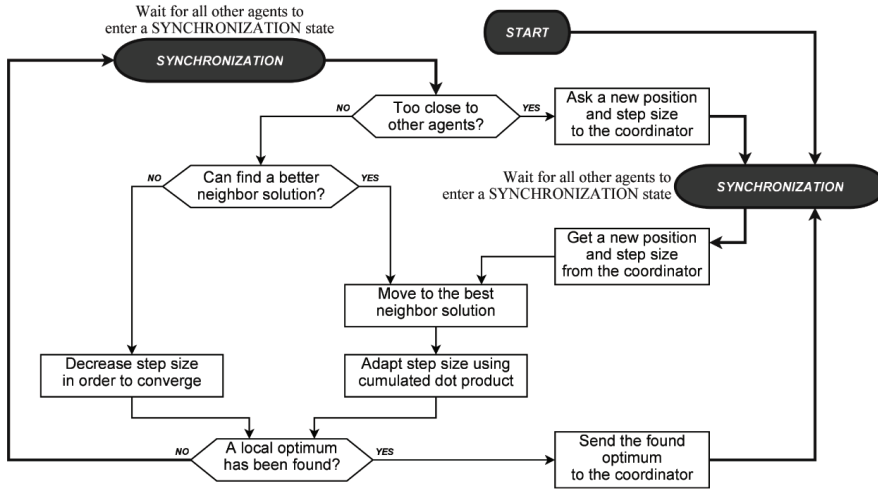
Algorithm 4. The replacement of suboptimal stored solutions

```

 $S_{sub} \leftarrow \{O_c\}$ 
 $O_{best} \leftarrow O_c$ 
foreach  $O_i \in S_m$  do
  if  $dist(O_i, O_c) \leq r_o$  then
     $S_{sub} \leftarrow S_{sub} \cup \{O_i\}$ 

```

Figure 5. The main algorithm flowchart of a MADO agent



```

if fitness( $O_i$ ) > fitness( $O_{best}$ ) then
     $O_{best} \leftarrow O_i$ 
end
end
 $S_{sub} \leftarrow S_{sub} - \{O_{best}\}$ 
 $S_m \leftarrow S_m - S_{sub}$ 
 $S_m \leftarrow S_m \cup \{O_{best}\}$ 

```

where O_c is the newly found optimum, S_m is the archive of stored optima, $dist(O_i, O_c)$ computes the distance between O_i and O_c , $fitness(O_i)$ is the fitness value of O_i and r_o is a threshold that defines the boundary of the suboptimal zone around an optimum, where there is not any other optimum.

r_o does not need to be lower than the lowest initial step size r_l of an agent, because even if there was another optimum O' at a distance from O_{best} lower than r_l , an agent starting a new search from O_{best} would detect O' . This is due to the fact that O' will lie in the local landscape of the agent (inside the hypersphere of radius R with $R \geq r_l$). Then, an agent starting

a new search from O_{best} would be able to sample candidate solutions around O' . Moreover, r_o must be high enough in order, for most stored suboptimal solutions, to be detected and removed. One can remark that, in the case where the landscape has thin peaks close to each other, a high value of r_o will remove some of them. The value of r_e is adjusted in order to prevent agents from hill climbing a same peak, and this value should be indeed a good upper bound for r_o . From these considerations, a good choice for the value of r_o appeared to be the geometric average of r_l and r_e . The value of r_o is thus calculated according to (7).

$$r_o = \sqrt{r_l r_e} \quad (7)$$

3.3.2 The Computation of the Exclusion Radius

Another task assigned to the memory manager is updating the value of r_e . At the beginning, an initial value of r_e , denoted by r_e^{max} , is calculated according to (8).

$$r_e^{max} = \frac{1}{2 n_a} \quad (8)$$

where n_a is the predefined number of agents created in the search space at the beginning of the MADDO algorithm. As it will be shown in subsection 3.4, the number of agents may vary temporarily, but the average number of agents along the whole search process tends to be equal to n_a . According to (8), when n_a increases, r_e^{max} decreases. We have to take into account that the increase in the number of agents will increase the probability to have several agents converging to the same optimum. Thus, if r_e^{max} does not decrease according to the number of agents, then for a high number of agents, most of them will never start a new local search, because of the presence of other agents in their exclusion radius. The formula (8) has also been chosen because it gives the largest radius that an agent can have. It allows to place all the n_a agents in a one-dimensional

search space, without having their exclusion radius overlapping another one.

When a new optimum is found, it is transmitted to the memory manager, and if this optimum is accepted, then the memory manager will update the value of r_e using this last stored optimum. The process of updating r_e is presented in Algorithm 5; where S'_m is the subset of optima in the archive without the last stored optimum, S_m is the set of stored optima, O_c is the last stored optimum, $dist(O_i, O_c)$ computes the distance between O_i and O_c and $card(S'_m)$ is the number of optima in the set S'_m . To update r_e , we compute the average value of the lowest distances from each stored optimum to the others, denoted by δ_{mean} . This is done using (9).

$$\delta_{mean} = \frac{\sum_{O_i \in S'_m} \left(\min_{O_j \in S_m - \{O_i\}} dist(O_j, O_i) \right)}{card(S'_m)} \quad (9)$$

Algorithm 5. The updating of the exclusion radius

```

 $S'_m \leftarrow S_m - \{O_c\}$ 
if  $card(S'_m) > 0$  then (since we need to compute distances, there must be at least two optima
in the archive)
     $\delta_{min} \leftarrow \infty$ 
    foreach  $O_i \in S'_m$  do (computes the lowest distance between the new optimum and the
    previously stored ones)
         $\delta_i \leftarrow dist(O_i, O_c)$ 
        if  $\delta_i < \delta_{min}$  then
             $\delta_{min} \leftarrow \delta_i$ 
        end
    end
    (since a too high  $r_e$  might prevent the convergence to optima close to each other,
    if  $\frac{\delta_{min}}{2} < r_e^{max}$  then  $r_e^{max}$  is used as an upper bound for  $\frac{\delta_{min}}{2}$ )
     $r_{mean} \leftarrow \frac{card(S'_m) r_e + \frac{\delta_{min}}{2}}{card(S'_m) + 1}$ 
     $r_e^{new} \leftarrow \frac{card(S'_m) r_{mean} + (n_m - card(S'_m)) r_e}{n_m}$ 
     $r_e \leftarrow r_e^{new}$ 
end
end

```

Assuming that r_e tends to be equal to the average value of the lowest distances from each optimum of S'_m to the others, then we can replace the sum of these lowest distances in (9) by $2 \text{card}(S'_m) r_e$. This leads to the formula (10).

$$\delta_{mean} \approx \frac{2 \text{card}(S_m - \{O_c\}) r_e + \min_{O_j \in S_m - \{O_c\}} \text{dist}(O_j, O_c)}{\text{card}(S_m)} \quad (10)$$

Since the exclusion radius r_e should match the average “radius” of the attractive zone of a peak, it is estimated by half the average lowest distance between the top of two peaks. This value, denoted by r_{mean} , is calculated from δ_{mean} using (11):

$$r_{mean} = \frac{\delta_{mean}}{2} \quad (11)$$

When S'_m is nearly empty, the updating process does not have enough optima to estimate a fitted average lowest distance between peaks. Then, it should not modify the value of r_e so much. However, when S'_m will be filled out, this estimation will be adapted to the search space landscape, and r_e will be made close to r_{mean} . This is the idea behind the expression of r_e^{new} in Algorithm 5.

3.4 Coordinator

The coordinator administrates all the main operations of the search process, by giving instructions to memory and agent management modules, and receiving information from them. It is in charge of the creation of the agents at the beginning of the algorithm. The number of agents to be created is given by the parameter n_a . The locations of these agents at the start of the search process are not randomly generated, but are computed in order to maximize

distances between them. This is done by using the electrostatic repulsion heuristic described in section 3.2.1. Rather than constraining the repulsing points to stay on the surface of the unit hypersphere, they are constrained to stay inside the hyperrectangle, denoted by T , that is centered in the search space (Figure 6). As illustrated in Figure 6, T is the Cartesian product of intervals $T_i = [r_e^{max}, 1 - r_e^{max}]$ with $i = 1, 2, \dots, d$. Thus, at the end of this heuristic, we get a set of initial positions for the initial set of agents, which are optimally covering the search space, i.e. the lowest distance between two of them is maximized inside T .

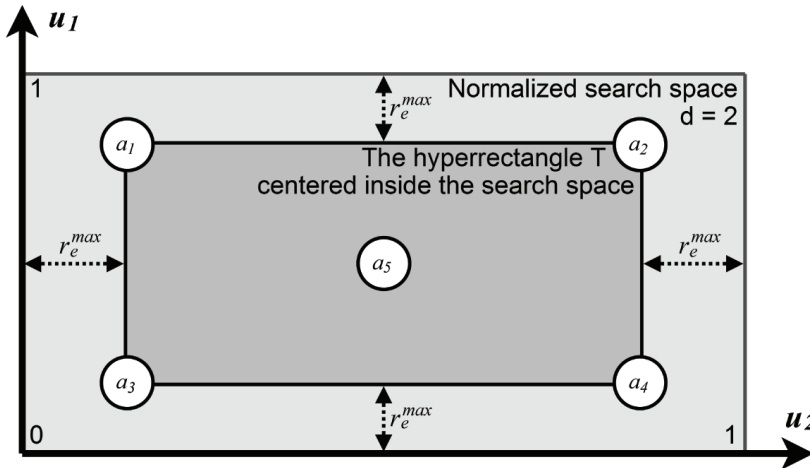
The instruction to create the initial set of agents is given to the agent management module, then the MADO algorithm can start the search process. The exploration of the search space is done by the agents, using trajectory local searches. Once an agent has found an optimum, this optimum is transmitted to the coordinator, that transmits it to the memory management module. Afterwards, the coordinator transmits a new location and a new step size to the agent. This new location is randomly generated and uniformly distributed in the search space, under the constraint that the lowest distance between this new location and the current position of any agent is greater or equal to the exclusion radius r_e . After five attempts to generate a new location at a random position outside the exclusion radii of agents without success, the coordinator sends a delete instruction to the agent manager. We did so because of the high probability that the search space is saturated with existing agents. If a random position satisfying the constraint is found, it is given as the new position of the agent, and its new step size is calculated as in (12):

$$R = \max(r_e, \text{rand}(1)) \quad (12)$$

where $\text{rand}(1)$ generates a uniform random number in $[0, 1]$.

The coordinator detects also the changes in the environment. This detection is performed when all the agents have finished one loop in

Figure 6. Computation of an initial set of 5 agents in a two-dimensional search space



Agents are depicted by white-filled circles denoted by a_i with $i \in \{1,2,\dots,5\}$

their search algorithm, i.e. when all the agents have moved one step ahead in their trajectory and/or performed one adaptation of their step size.

Changes in the environment are detected by re-evaluating the fitness of the best optimum of the archive. When any optimum has been found yet, we re-evaluate the current fitness of an agent randomly chosen, and compare it to its previous value. If these values are different, a change is supposed to have occurred, and a tracking of the stored optima is performed (Algorithm 6); where S_a is the set of all agents, S_m is the

archive of stored optima and r_l is the lowest possible initial step size of an agent, i.e. the lowest allowed value for r_e . The use of such a low initial step size for these new agents is needed in order to track the optima, i.e. to hill climb the peaks of the previously stored optima to find their new position. A large initial step size allows these agents to leave the peaks of the tracked optima. Thus, they will explore other zones of the search space rather than staying on the peaks they have to hill climb. Hence, the best value for r_l is the value that takes

Algorithm 6. The tracking of stored optima after a change

```

foreach  $a_i \in S_a$  do
    Re-evaluate the fitness of  $a_i$ 
end
foreach  $O_i \in S_m$  do
    Re-evaluate the fitness of  $O_i$ 
    Create an agent located on  $O_i$  and with an initial step size of  $r_l$ 
end
 $S_m \leftarrow \emptyset$ 

```

into account the move of the peaks through the search space.

As we can see, the number of agents varies along the search process, and can be lower or greater than the initial number n_a . However, to prevent the number of agents to increase more and more at every detection of a change in the environment, the coordinator will send a delete instruction if the number of agents is higher than n_a , rather than letting an agent start a new search, when its trajectory search is finished. This way, the number of agents will be temporarily higher than n_a (after a change has been detected), and will decrease to n_a , because the excess agents will just track the previously stored optima and send back their new positions.

3.5 Parameter Setting of MADO

Table 2 summarizes the eight parameters of MADO that the user has to define. In this table, the values given are suitable for MPB and they were fixed experimentally. These values will be used to perform the experiments reported in Section 4.

We fixed the number of agents equal to 4, because the convergence needs to be fast. Having too many agents exploring the search space will slow down the individual convergence of each agent and lead to a waste of fitness function evaluations. The precision parameter δ_p and the lowest step size r_l need to be correctly adapted to the change severity and to the change frequency of the environment. A low value for δ_p makes the agents unable to track optima in a fast changing environment, since they will spend too many iterations on fine-tuning their current solution, and might never be able to send it to the coordinator for archiving, before a new change in the environment occurs. Hence, the lower is the number of iterations between two changes, the higher should be the value of δ_p . This means that the precision of the found optima in fast changing environments might be worse than in slow changing ones.

The value of r_l needs also to be well suited to the severity of the changes, since a low value of r_l in a fast changing environment requires a lot of adaptations of the step size, and a waste of many fitness function evaluations. On the opposite, a high value of r_l may make the tracking agent leave the peak of the optimum it has to track, and make it begin exploring the search space elsewhere. The other parameters of the algorithm are much less critical, and might be left at their default values (given in Table 2).

4. RESULTS AND DISCUSSION

We used the Moving Peaks Benchmark in order to test the MADO algorithm. In (Branke, 1999), three sets of parameters, called scenarios, were proposed. These scenarios are given as a set of standard settings, that can be used in order to produce comparable results. Authors have to specify the scenario they used to test their methods, preventing the use of custom settings that may differ from one paper to another. It appears that the most commonly used set of parameters for MPB is scenario 2 (see Table 3), hence, it will be used in this article. In this scenario, the correlation coefficient λ is equal to 0, which means that when a change occurs, the peaks move in random directions that are not correlated to their previous moves. Hence, the direction of the peaks' moves cannot be predicted from the direction of their previous moves. This leads to a harder scenario than if we used a value of λ strictly greater than 0. The rest of this section is organized as follows. Firstly, an experimental analysis of the MADO algorithm is made, justifying the use of each component of the algorithm. This analysis is followed by a comparison to other methods on scenario 2, for various numbers of peaks and dimensions.

4.1 Empirical Analysis of the MADO Algorithm

An evaluation of all the different components of the MADO algorithm is made, in order to

Table 2. MADO parameter setting

Name	Interval	Default value	Short description
n_a	N	4	initial (and average) number of agents
n_m	N	10	capacity of the archive of found optima
N	N	$\text{round}(0.28 \cdot d + 2)$	number of candidate solutions per agent's move, where $\text{round}(v)$ gives the nearest integer to v
c_u	$[0, 1]$	0.5	coefficient which defines the weight of the cumulative dot product in the step size adaptation process
c_r	$]0, 1]$	0.8	coefficient which specifies how much the step size is decreased (or increased) during its adaptation
r_l	$]0, 1]$	0.0025	lowest possible step size (should match the change severity of the search space)
δ_t	N	8	maximum number of moves an agent can make without improving its fitness more than δ_p
δ_p	R	0.005	the precision parameter of the stagnation criterion of the agents trajectory searches

justify their requirement for obtaining quality results. This evaluation is performed on scenario 2 of MPB, and the resulting offline errors and standard deviations averaged on 100 runs are summarized in Table 4, for different variants of MADO algorithm. The maximum number of iterations is fixed to $5 \cdot 10^5$, that corresponds to 100 changes occurred in the environment during each run.

As one can see, the standard MADO is better than all the simpler variants tested here. Among these variants, we can note that $MADO - r_e$ does not perform a detection of other agents inside the exclusion radius r_e of an agent, i.e. an agent has not to start a new search elsewhere when it is too close to another agent, and the coordinator has not to generate a new starting position, for an agent outside the exclusion radius of the other agents. Thus, all agents can explore the same zone of the search space. However, r_e is still used in $MADO - r_e$ for the other procedures, as in equation (12) and in Algorithm 4. In $MADO - r_e^{adapt}$, the only

difference with the standard version is that r_e is constant and equal to r_e^{max} , i.e. the Algorithm 5 is not performed. In $MADO - S_a^{max}$,

the electrostatic repulsion heuristic is not used to produce the initial set of agents, since their positions are randomly generated inside the search space and outside the exclusion radius of existing agents. In $MADO - S$, the electrostatic repulsion heuristic is not used to produce the set S of candidate solutions of an agent, and the candidate solutions are randomly generated and uniformly distributed inside the hypersphere of radius R centered on the agent's current position. In $MADO - CDPA$, the cumulative dot product adaptation of the step size is not used, and the only adaptation process used is the reduction of R (when no better candidate solution can be found in the local landscape of an agent). Finally, we can also note that in $MADO - r_l^{track}$, the initial step size of an agent, created by the coordinator when a change is

Table 3. MPB parameters in scenario 2

Parameter	Scenario 2
Number of peaks	10
Dimensions	5
Peak heights	[30, 70]
Peak widths	[1, 12]
Change cycle	5000
Change severity	1
Height severity	7
Width severity	1
Correlation coefficient λ	0

detected in the environment (on the location of a previously found optimum), is generated using equation (12).

From Table 4, one can conclude that the most important components of the MADO algorithm, in order to obtain good results, are the archiving of found local optima, the use of maximally spaced candidate solutions on the surface of the local landscape hypersphere of agents, and the use of r_i as initial step size of the agents created (in place of stored optima, when a change is detected).

To give an idea about the computational cost of the MADO algorithm, the average time on 100 runs needed to compute $5 \cdot 10^5$ fitness function evaluations on the scenario 2 of MPB, using a 2.26 GHz Intel Core 2 Duo P8400 processor, is equal to 363 milliseconds, including the time of evaluation of the fitness function of MPB.

4.2 Comparison with Competing Methods

The comparison, on MPB, of MADO with the other leading optimization algorithms in dynamic environments is summarized in

Table 5. The offline errors and the standard deviations are given, and the algorithms are sorted from the best to the worst. Results are averaged on 100 runs or 50 runs of the tested algorithms, and the maximum number of fitness evaluations per run is fixed to $5 \cdot 10^5$, i.e. 100 changes per run.

Results gathered in Table 5 for competing algorithms are those given in the references listed in the first column. As we can see, MADO is the second best algorithm in this classification, and its confidence interval overlaps the confidence interval of the best rated algorithm. Although MADO is rated behind MMEO (Moser & Hendtlass, 2007) on scenario 2 of MPB, MMEO produces worse results than MADO on some other instances of MPB, i.e. those with a higher number of peaks or dimensions. An extended comparison, averaged on 100 runs with 100 changes per run, between MMEO and MADO, is given in Table 6. Scenario 2 is still used, with a modified value of one parameter in each column, and the offline error is also used, with its standard deviation.

This comparison shows that MADO is more robust and can be applied to highly multimodal DOPs and DOPs with high dimensionality.

Table 4. Offline error on MPB for each simplified variant of the MADO algorithm

Variant	Offline error	Short description
<i>MADO</i>	0.80 ± 0.19	the standard MADO algorithm
$MADO_{n_a=1}$	1.26 ± 0.48	with only one exploring agent
$MADO_{n_m=0}$	5.96 ± 0.37	without archiving local optima
$MADO_{r_e}$	1.23 ± 0.38	no exclusion radius for the agents
$MADO_{r_e^{adapt}}$	0.82 ± 0.24	no adaptation of the exclusion radius
$MADO_{S_a^{max}}$	0.84 ± 0.26	no lowest distance maximization for the initial set of agents
$MADO_S$	3.78 ± 0.48	no lowest distance maximization for candidate solutions
$MADO_{CDPA}$	1.06 ± 0.33	no cumulative dot product adaptation of the step size of the agents
$MADO_{R^{min}}$	0.91 ± 0.33	using r_l instead of r_e as lower bound of the initial step size of an agent (see (12))
$MADO_{r_l^{track}}$	2.82 ± 0.24	without using r_l as initial step size for agents created in place of stored optima

5. CONCLUSION

A new multiagent algorithm has been designed for continuous dynamic optimization problems. In this algorithm, a multiple trajectory local search strategy with an adaptive step size has been implemented. These local searches are conducted by the agents of the algorithm, and a memory was added for archiving the found optima. This way, when a change is detected in the environment, the archived optima can be re-used in order to track them through the search space. The agents and the memory are also encapsulated in two different modules, that are themselves coordinated by a third module. This third module, named the coordinator, takes all the global decisions and actions over the whole search process, like deleting or creating agents, and detecting changes in the

environment. This implementation has been specifically designed for dynamic continuous optimization, and proved its efficiency on the Moving Peaks Benchmark. This algorithm can also be expected to produce good results, when solving static objective functions.

The major drawback of MADO is that its critical parameters are not automatically adjusted along with the search process, and need to be fine-tuned by the user. In works in progress, we would like to make these critical parameters adaptive. We will also attempt to make this algorithm simpler and lighter. Some of its components may be subject to replacement or deletion, and the number of parameters may be reduced. Finally, as a lot of real world DOPs are multi-objective, or have a lot of constraints that can be dynamic, the proposed algorithm may also be modified to make it

Table 5. Comparison with competing algorithms on MPB (scenario 2)

Algorithm	number of runs	offline error $\pm$ standard deviation
Moser & Hendtlass, 2007	100	0.66 ± 0.20
MADO	100	0.80 ± 0.19
	50	0.78 ± 0.22
Lung & Dumitrescu, 2007	50	1.38 ± 0.02
Lung & Dumitrescu, 2008	50	1.53 ± 0.01
Blackwell & Branke, 2006	50	1.72 ± 0.06
Mendes & Mohais, 2005	50	1.75 ± 0.03
Li, Branke, & Blackwell, 2006	50	1.93 ± 0.06
Blackwell & Branke, 2004	50	2.16 ± 0.06
Du & Li, 2008	50	4.02 ± 0.56

Table 6. Offline error of MADO and MMEO on several instances of MPB

Algorithm	on 100 peaks	in 7 dimensions
MADO	1.12 ± 0.08	1.61 ± 0.13
MMEO	1.47*	1.73 ± 0.55

multi-objective and able to handle hard and dynamic constraints.

REFERENCES

Blackwell, T., & Branke, J. (2004). Multi-swarm optimization in dynamic environments (LNCS 3005, pp. 489-500).

Blackwell, T., & Branke, J. (2006). Multi-swarms, exclusion and anti-convergence in dynamic environments. *IEEE Transactions on Evolutionary Computation*, 10, 459–472. doi:10.1109/TEVC.2005.857074

Branke, J. (1999). Memory enhanced evolutionary algorithms for changing optimization problems. In *Proceedings of the IEEE Congress on Evolutionary Computation (CEC 1999)* (pp. 1875-1882). Washington, DC: IEEE Computer Society.

Branke, J. (1999). *The moving peaks benchmark*. Retrieved from <http://www.aifb.uni-karlsruhe.de/~jbr/MovPeaks>

Branke, J., & Mattfeld, D. (2005). Anticipation and flexibility in dynamic scheduling. *International Journal of Production Research*, 43, 3103–3129. doi:10.1080/00207540500077140

Conway, J. H., & Alexander, N. J. (1998). *Sphere packings, lattices and groups* (3rd ed.). New York: Springer.

Dorigo, M., & Gambardella, L. M. (2002). Guest editorial special on ant colony optimization. *IEEE Transactions on Evolutionary Computation*, 6, 317–319. doi:10.1109/TEVC.2002.802446

Dréo, J., & Siarry, P. (2006). An ant colony algorithm aimed at dynamic continuous optimization. *Applied Mathematics and Computation*, 181, 457–467. doi:10.1016/j.amc.2005.12.051

- Du, W., & Li, B. (2008). Multi-strategy ensemble particle swarm optimization for dynamic optimization. *Information Sciences*, 178, 3096–3109. doi:10.1016/j.ins.2008.01.020
- Hansen, N., & Ostermeier, A. (2001). Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9, 159–195. doi:10.1162/106365601750190398
- Householder, A. (1958). Unitary triangularization of a nonsymmetric matrix. *Journal of the ACM*, 5, 339–342. doi:10.1145/320941.320947
- Huang, C.-F., & Rocha, L. M. (2005). Tracking extrema in dynamic environments using a coevolutionary agent-based model of genotype edition. In *Proceedings of the 2005 Conference on Genetic and Evolutionary Computation* (pp. 545–552). ACM Publishing.
- Jin, Y., & Branke, J. (2005). Evolutionary optimization in uncertain environments - a survey. *IEEE Transactions on Evolutionary Computation*, 9, 303–317. doi:10.1109/TEVC.2005.846356
- Kennedy, J., & Eberhart, R. C. (1995). Particle swarm optimization. In *Proceedings of the IEEE International Conference on Neural Networks* (pp. 1942–1948). Washington, DC: IEEE Computer Society.
- Larsen, A. (2000). *The dynamic vehicle routing problem*. Copenhagen, Denmark: Technical University of Denmark.
- Li, X., Branke, J., & Blackwell, T. (2006). Particle swarm with speciation and adaptation in a dynamic environment. In *Proceedings of the 8th Annual Conference on Genetic and Evolutionary Computation* (pp. 51–58). ACM Publishing.
- Lung, R. I., & Dumitrescu, D. (2007). Collaborative evolutionary swarm optimization with a Gauss Chaotic Sequence Generator. *Innovations in Hybrid Intelligent Systems*, 44, 207–214. doi:10.1007/978-3-540-74972-1_28
- Lung, R. I., & Dumitrescu, D. (2008). ESCA: A new evolutionary-swarm cooperative algorithm. *Studies in Computational Intelligence*, 129, 105–114. doi:10.1007/978-3-540-78987-1_10
- Mendes, R., & Mohais, A. (2005). DynDE: A differential evolution for dynamic optimization problems. In *Proceedings of the 2005 IEEE Congress on Evolutionary Computation* (pp. 2808–2815). Washington, DC: IEEE Computer Society.
- Minner, S. (2003). Multiple-supplier inventory models in supply chain management: A review. *International Journal of Production Economics*, 81, 265–279. doi:10.1016/S0925-5273(02)00288-8
- Moser, I., & Hendtlass, T. (2007). A simple and efficient multi-component algorithm for solving dynamic function optimisation problems. In *Proceedings of the 2005 IEEE Congress on Evolutionary Computation* (pp. 252–259). Washington, DC: IEEE Computer Society.
- Nelder, J., & Mead, R. (1965). A simplex method for function minimization. *The Computer Journal*, 7, 308–313.
- Rossi, C., Abderrahim, M., & Diaz, J. C. (2008). Tracking moving optima using Kalman-based predictions. *Evolutionary Computation*, 16, 1–30. doi:10.1162/evco.2008.16.1.1
- Rossi, C., Barrientos, A., & Cerro, J. D. (2007). Two adaptive mutation operators for optima tracking in dynamic optimization problems with evolution strategies. In *Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation* (pp. 697–704). ACM Publishing.
- Simoes, A., & Costa, E. (2008). Evolutionary algorithms for dynamic environments: Prediction using linear regression and Markov chains. In *Parallel problem solving from nature* (pp. 306–315). Springer.
- Tfaily, W., & Siarry, P. (2008). A new charged ant colony algorithm for continuous dynamic optimization. *Applied Mathematics and Computation*, 197, 604–613. doi:10.1016/j.amc.2007.08.087
- Tinos, R., & Yang, S. (2007). A self-organizing random immigrants genetic algorithm for dynamic optimization problems. *Genetic Programming and Evolvable Machines*, 8, 255–286. doi:10.1007/s10710-007-9024-z
- Yang, S. (2003). Non-stationary problem optimization using the primal-dual genetic algorithm. In *Proceedings of the 2003 IEEE Congress on Evolutionary Computation* (pp. 2246–2253). Washington, DC: IEEE Computer Society.
- Yang, S. (2006). A comparative study of immune system based genetic algorithms in dynamic environments. In *Proceedings of the 8th Annual Conference on Genetic and Evolutionary Computation* (pp. 1377–1384). ACM Publishing.

Yang, S., & Yao, X. (2005). Experimental study on population-based incremental learning algorithms for dynamic optimization problems. *Soft Computing - A Fusion of Foundations . Methodologies and Applications*, 9, 815–834.

Julien Lepagnot was born in France in 1982. He received the master's degree in computer sciences in 2008 from the University Evry-val-d'Essonne. Now, he is pursuing a PhD degree in computer engineering at the University Paris 12. His main research interests include image segmentation and compression, stochastic global optimization heuristics and multi-agent systems.

Amir Nakib was born in Algeria in 1977. He received the magister in signal processing in 2003 and a second MS degree in image processing in 2004 from the University Paris 6. He received in December 2007 a PhD degree in computer engineering at University Paris 12 (France). His main research interests include image segmentation and compression, signal compression, and stochastic global optimization heuristics.

Hamouche Oulhadj was born in Algeria in 1956. He received the BS degree in electrical engineering at the Polytechnic School of Algiers, the DEA and the PhD degree in biomedical engineering from the University Paris 12 in 1985 and 1990 respectively. Now, he is an associate professor at the same University. His main research interests are pattern recognition, biomedical image segmentation and information extraction.

Patrick Siarry was born in France in 1952. He received the PhD degree from the University Paris 6, in 1986 and the doctorate of sciences (Habilitation) from the University Paris 11, in 1994. He was first involved in the development of analog and digital models of nuclear power plants at Electricité de France (E.D.F.). Since 1995 he is a professor in automatics and informatics. His main research interests are computer-aided design of electronic circuits, and the applications of new stochastic global optimization heuristics to various engineering fields. He is also interested in the fitting of process models to experimental data, the learning of fuzzy rule bases, and of neural networks.

A Reinforcement Learning: Great-Deluge Hyper-Heuristic for Examination Timetabling

Ender Özcan, University of Nottingham, UK

Mustafa Mısırlı, Yeditepe University, Turkey

Gabriela Ochoa, University of Nottingham, UK

Edmund K. Burke, University of Nottingham, UK

ABSTRACT

Hyper-heuristics can be identified as methodologies that search the space generated by a finite set of low level heuristics for solving search problems. An iterative hyper-heuristic framework can be thought of as requiring a single candidate solution and multiple perturbation low level heuristics. An initially generated complete solution goes through two successive processes (heuristic selection and move acceptance) until a set of termination criteria is satisfied. A motivating goal of hyper-heuristic research is to create automated techniques that are applicable to a wide range of problems with different characteristics. Some previous studies show that different combinations of heuristic selection and move acceptance as hyper-heuristic components might yield different performances. This study investigates whether learning heuristic selection can improve the performance of a great deluge based hyper-heuristic using an examination timetabling problem as a case study.

Keywords: Exam Timetabling, Great Deluge, Hyper-Heuristics, Meta-Heuristics, Reinforcement Learning

1 INTRODUCTION

Meta-heuristics have been widely and successfully applied to many different problems. However, significant development effort is often needed to produce fine tuned techniques for the particular problem or even instance that is under investigation. Hyper-heuristics represent an increasingly popular research direction in search and optimisation (Burke

et al., 2003a; Ross, 2005; Chakhlevitch et al., 2008; Özcan et al., 2008; Burke et al. 2009a, 2009b). One of the aims is to produce more general problem solving techniques, which can potentially be applied to different problems or instances with little development effort. The idea is that a hyper-heuristic approach should be able to intelligently choose an appropriate low-level heuristic (from a given repository of heuristics) to be applied at any given time. Thus, in hyper-heuristics, we are interested in adaptively finding solution methods, rather than

DOI: 10.4018/jamc.2010102603

directly producing a solution for whichever search problem we are studying.

Several hyper-heuristics approaches have been proposed in the literature. It is possible to consider methodologies based on *perturbation* low-level heuristics and those based on *construction* low-level heuristics. The latter type builds a solution incrementally, starting with a blank solution and using construction heuristics to gradually build a complete solution. They have been successfully investigated for several combinatorial optimisation problems such as: bin-packing (Tereshima-Marin et al., 2007), timetabling (Terashima-Marin et al., 1999; Burke et al., 2007; Qu et al., 2008), production scheduling (Vazquez-Rodriguez et al., 2007), and cutting stock (Terashima-Marin et al., 2005). On the other hand, approaches based on perturbation heuristics find a reasonable initial solution by some straightforward means (either randomly or using a simple construction heuristic) and then use heuristics, such as shift and swap to perturb solution components with the aim of finding improved solutions. In other words, they start from a complete solution and then search or select among a set of neighbourhoods for better solutions. A class of the most commonly used hyper-heuristics based on perturbation (improvement) low level heuristics is the *choice* hyper-heuristics (heuristic selection methodologies). They have been applied to real world problems, such as, personnel scheduling (Cowling et al., 2001; Burke et al., 2003b), timetabling (Burke et al., 2003b; Dowsland et al., 2007), and vehicle routing problems (Pisinger et al., 2007). In a choice hyper-heuristic framework based on perturbation low level heuristics, search is mostly performed using a single candidate solution. Such hyper-heuristics, iteratively, attempt to improve a given solution throughout two consecutive phases: *heuristic selection* and *move acceptance* as illustrated in Figure 1.

In Figure 1, a candidate solution (S_t) at a given time (t) is modified into a new solution (or solutions) using a chosen heuristic (or heuristics). Then, a move acceptance method is employed to decide whether to accept or

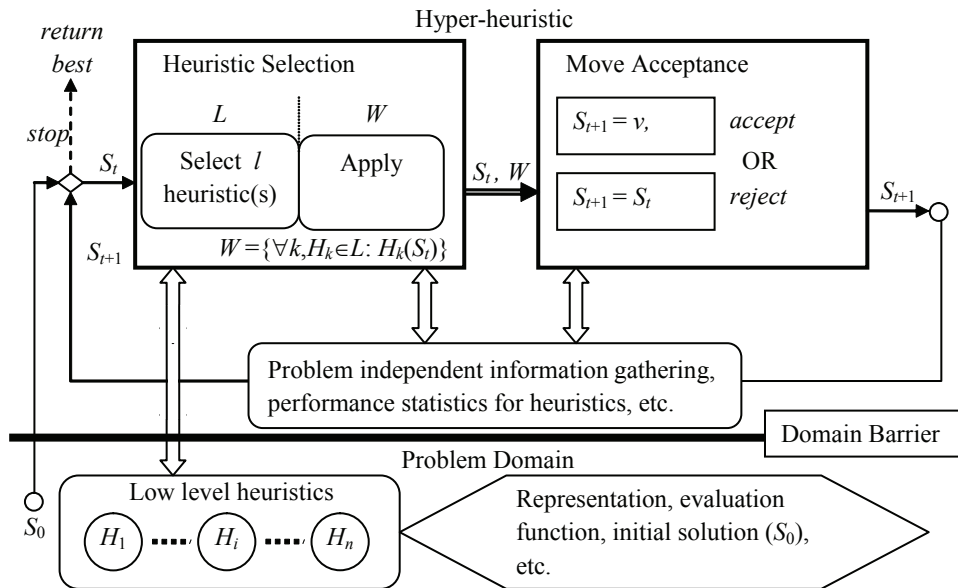
reject a resultant solution (R_t). This process is repeated until a predefined stopping condition is met. Only problem independent information flow is allowed between the problem domain and hyper-heuristic layers. Unless, we specifically say otherwise, a choice hyper-heuristic refers to a hyper-heuristic that operates on a set of perturbation low level heuristics from this point onwards. Moreover, such a hyper-heuristic will be denoted as *heuristic selection – move acceptance* based on its components.

Great deluge is a well-known acceptance strategy (Dueck, 1993; Burke et al., 2003). Bilgin et al. (2007) reported that hyper-heuristics formed by different combinations of heuristic selection and move acceptance methods might yield different performances. Moreover, simple random–great deluge delivered a similar performance to the best approach; namely, choice function–simulated annealing for examination timetabling. Obviously, simple random receives no feedback at all during the search to improve upon the heuristic selection process. Hence, in this study, *great-deluge* is preferred as the move acceptance component within a choice hyper-heuristic framework to investigate the effect of learning heuristic selection on its overall performance for solving the same examination timetabling problem as formulated in Bilgin et al. (2007). The learning mechanisms, inspired by the work by Nareyek (2003), are based on weight adaptation.

2 Hyper-Heuristics and Learning

Although hyper-heuristic as a term has been introduced recently (Denzinger et al., 1997), the origins of the idea date back to the early 1960s (Fisher et al., 1961). A hyper-heuristic operates at a high level by managing or generating low level heuristics which operate on the problem domain. Meta-heuristics have been commonly used as hyper-heuristics. A hyper-heuristic can conduct a single point or multi-point search. Population based meta-heuristics which perform multi-point search, such as learning classifier systems (Marin-Blázquez and Schulenburg, 2005), evolutionary algorithms (Cowling et al.,

Figure 1. A hyper-heuristic framework based on a single point search, where S_t denotes a candidate solution at time t , H_i is the i^{th} low level heuristic, R_i is the resultant solution after applying a set of selected low level heuristics that goes into the move acceptance process



2002; Han et al., 2003; Pillay and Banzhaf, 2008), genetic programming (Keller et al., 2007; Burke et al., 2009a), ant colony optimisation (Cuesta-Canada et al., 2005; Chen et al., 2007) have been applied to a variety of combinatorial optimisation problems as hyper-heuristics. Distributed computing methods can also be used to perform multi-point search (Rattadilok et al., 2004; Rattadilok et al., 2005; Ouelhadj et al., 2008). Özcan et al. (2008) presented different hyper-heuristic frameworks showing that a matching performance to memetic algorithms can be achieved. In this study, the choice hyper-heuristic framework as presented in Figure 1 is studied. The primary components of such hyper-heuristics are heuristic selection and move acceptance.

A major motivating feature of hyper-heuristic research is the aim to facilitate applicability to different problem instances having different characteristics as well as different problem domains. With this goal in mind, machine learning techniques are vital for hyper-heuristics

to make the right choices during the heuristic selection process. Learning can be achieved in an *offline* or *online* manner. An offline learning hyper-heuristic requires training over a set of problems, before it is used to solve the unseen problem instances. For example, Burke et al. (2006) use a case based reasoning system as a hyper-heuristic for solving course and examination timetabling problems. An online learning hyper-heuristic learns through the feedback obtained during the search process while solving a given problem. Most of the existing online learning hyper-heuristics incorporate *reinforcement learning* (Kaelbling et al., 1996; Sutton et al., 1998). A reinforcement learning system interacts with the environment and changes its state via a selected action in such a way as to increase some notion of long term reward. Hence, a learning hyper-heuristic maintains a utility value obtained through predetermined reward and punishment schemes for each low level heuristic. A heuristic is selected based on the utility values of the low level heuristics in

hand at each step. Remembering and forgetting represent core ingredients of learning. Remembering can be achieved through reward and punishment schemes. Forgetting can be achieved through the use of lower and upper bounds on the utility values. Some reinforcement learning methods use weighted average of the learnt utility values. A dynamic weighting scheme can be employed which favours the outcome of the most recent actions or choices. Reward and punishment schemes are allowed to use different adaptation rates in the case of an improving and worsening move, respectively. For example, the utility value of a selected heuristic can be increased at a constant rate linearly whenever there is an improvement after it is employed, otherwise the utility value can be decreased at a different rate, or it can even be kept constant. Initialisation of the utility values, lower and upper bounds for them along with a memory adjustment scheme (weighting) represent the remainder of the constituents for a reinforcement learning based hyper-heuristic.

Some previously studied heuristic selection methods are summarised in Table 1. Simple random, random gradient, random permutation gradient, greedy and choice function heuristic selection methods are presented in Cowling et al. (2001a). All these approaches can be considered to be learning heuristic selection methods, except simple random. In Cowling et al. (2001b), a parameter-free choice function was presented. As a problem domain, sales summit scheduling was used in both studies. Cowling and Chakhlevitch (2003) investigated peckish heuristic selection strategies that eliminated the selection and application of all low level heuristics as in greedy heuristic selection.

Nareyek (2003) investigated reinforcement learning using different reward/penalty schemes and heuristic selection strategies on the Orc Quest problem and in the logistics domain. Additive/subtractive adaptation rates combined with heuristic selection using the maximal utility generated better results as opposed to a fair random choice (softmax, roulette wheel). All heuristics were assigned to a utility value of 0 initially and raw utility values were main-

tained. Upper and lower bounds were defined for the utility values. In Burke et al. (2003b), reinforcement learning was combined with tabu search in a hyper-heuristic and applied to the personnel rostering and timetabling problems. The aim of this modification was to prevent the selection of some heuristics for a while by inserting them into a variable-length tabu list. A non-tabu heuristic with the highest utility value was chosen at each step.

Some studies concentrate on move acceptance in hyper-heuristics rather than upon heuristic selection methods, as accepting a move turns out to be an extremely important decision. In Cowling et al. (2001), heuristic selection methods are combined with either all moves accepted or with only an improving moves accepted strategy. On the other hand, Ayob and Kendall (2003) proposed three different Monte Carlo move acceptance strategies based on the objective value change due to the move, time (units), number of consecutive non-improving moves. Simple random was used as a heuristic selection within the hyper-heuristic for solving the component placement problem. The best move acceptance turned out to be *exponential Monte Carlo with counter*. One of the well known move acceptance strategies is *simulated annealing* (SA) (Kirkpatrick, 1983). The improving moves or the moves that generate an equal quality solution are accepted, while a worsening move is not rejected immediately. Acceptance of a given candidate solution is based on a probabilistic framework that depends on the objective value change and a temperature that decreases in time (cooling). The difference between exponential Monte Carlo with counter and the simulated annealing is that the latter one uses this *cooling schedule* while the former does not. Bai and Kendall (2003) investigated the performance of a simple random – simulated annealing hyper-heuristic on a shelf space allocation problem. Anagnostopoulos et al. (2006) applied a similar hyper-heuristic to a set of travelling tournament problem instances embedding a reheating scheme into the simulated annealing move acceptance. In Bai et al. (2007a), a reinforcement learning scheme is

Table 1. Description of a set of heuristic selection methods used within choice hyper-heuristics

Name	Description
Simple Random	Choose a low level heuristic randomly
Random Descent	Choose a low level heuristic randomly and employ the same heuristic as long as the candidate solution in hand is improved
Random Permutation Descent	Generate a random permutation of low level heuristics and form a cyclic list. Starting from the first heuristic, employ it repeatedly until a worsening move is hit, then go to the next heuristic in the list.
Greedy	Apply all low level heuristics to the same candidate solution separately and choose the heuristic that generates the best change in the objective value
Peckish	Apply a subset of all low level heuristics to the same candidate solution and choose the heuristic that generates the best change in the objective value
Choice Function	Dynamically score each heuristic weighing their individual performance, combined performance with the previously invoked heuristic and time passed since the last call to the heuristic at a given step. Then, a heuristic is chosen based on these scores.
Reinforcement Learning	Each heuristic carries a utility value and heuristic selection is performed based on these values. This value gets updated at each step based on the success of the chosen heuristic. An improving move is rewarded, while a worsening move is punished using a preselected adaptation rate.
Tabu Search	This method employs the same strategy as Reinforcement Learning and uses a tabu list to keep track of the heuristics causing worsening moves. A heuristic is selected which is not in the tabu list.

combined with simulated annealing with reheating as a hyper-heuristic and applied to three different problem domains: nurse rostering, course timetabling and 1D bin packing.

In (Dueck, 1993), two move acceptance strategies, namely *great deluge* (GD) and *record-to-record travel* that accept worsening moves based on a dynamic threshold value were presented. Kendall and Mohamad (2004) utilised a simple random – great deluge hyper-heuristic to solve a mobile telecommunication network problem. Great deluge uses a threshold that decreases in time at a given rate (e.g., linearly) to determine an acceptance range for the solution qualities based on three main parameters: (i) the maximum number of iterations (or total time), (ii) the number of iterations (or time) passed, and (iii) an expected range for the maximum fitness change between the initial and final objective value (e.g., lower bound). In the case of an improving move, it is accepted, while a worsening move is accepted only if

the objective value of the resultant candidate solution is less than the computed threshold at a given iteration. Kendall and Mohamad (2004) used an iteration based threshold formula with a maximum number of iterations as a termination criterion aiming a quadratic running time for the overall algorithm.

Bilgin et al. (2007) employed different heuristic selection and move acceptance mechanisms and used their combinations as hyper-heuristics. The results showed that a simple random – great deluge hyper-heuristic was the second best after choice function – simulated annealing, considering the average performance of all hyper-heuristics over a set of examination timetabling problems. Consequently, a hyper-heuristic without learning delivered a comparable performance to another one with a learning mechanism. Therefore, in this study, reinforcement learning is combined with great deluge to observe the effect of learning heuristic selection on the overall performance of the

hyper-heuristic for solving the same problem. All the runs during the experiments in (Bilgin et al., 2007) were restricted to 600 seconds; hence, the threshold is computed based on the CPU time within the great deluge move acceptance strategy. If a heuristic takes less time, then the threshold value will be lower as compared to the one that takes longer time. This hyper-heuristic differs from the one that Kendall and Hussin (2005) have investigated, as their hyper-heuristic embeds a tabu list approach to keep the chosen heuristic from getting selected again for a number of steps (tabu duration) into reinforcement learning as a heuristic selection. Moreover, the low level heuristics contained a mixture of thirteen different construction and perturbation low level heuristics.

Özcan et al. (2009) combined different heuristic selection methods with a late acceptance strategy, a new method that is initially presented as a local search for solving examination timetabling problems. Late acceptance requires a single parameter and it is a memory based approach. A trial solution is compared with a previously visited solution at a fixed distance apart from the current step in contrast to the conventional approaches that usually compare the trial solution with a current one. The trial solution is accepted, if there is an improvement over this previously visited solution. The results showed that reinforcement learning, reinforcement learning with tabu list or choice function heuristic selection methods did not improve the performance of the hyper-heuristic if late acceptance is used. Choosing a heuristic randomly at each step performed the best. More on hyper-heuristics can be found in Burke et al. (2003a), Ross (2005), Özcan et al. (2008), Chakhlevitch and Cowling (2008), Burke et al. (2009a, 2009b, 2009c).

3 THE EXAMINATION TIMETABLING PROBLEM

Examination timetabling is a challenging real world problem addressed by educational institutions. The goal is to find the best assign-

ment of available timeslots and possibly other resources, such as rooms for each examination subject to a range of constraints. There are two types of constraints: hard and soft constraints. Hard constraints must not be violated under any circumstances and a solution which satisfies them is called a feasible solution. For example, a student cannot take any pair of his/her examinations at the same time. Soft constraints reflect preferences and their violation is allowed, but the goal is to minimise it. For example, a number of timeslots might be preferred in between the examinations of a student scheduled to the same day.

3.1 Previous Work

Researchers have been studying various aspects of examination timetabling problems since the early 1960s (Cole, 1964; Broder, 1964). Examination timetabling problems are NP-complete (Even, 1976). Since the search space of candidate solutions grows exponentially with respect to the number examinations to be scheduled, many different non-traditional approaches (e.g., meta-heuristics) have been investigated for solving a variety of examination timetabling problems. Tables 2 and 3 provide some illustrative examples of these approaches.

Many examination timetabling problems are studied from a practical point of view, as they arise due to practical needs within institutions. It is worth pointing out that different institutions have very different requirements (Burke et al., 1996a). One consequence of this is that there is a variety of examination timetabling problems in the literature (Table 3; see Qu et al., 2009). Carter et al. (1996b) introduced one of the widely used examination timetabling data sets which was originally made up of 13 real world problems. Özcan et al. (2005) introduced an examination timetabling problem at Yeditepe University. In this initial study, different memetic algorithms were described. A type of violation directed hill climbing (Alkan and Özcan, 2003) was also investigated as a part of the memetic algorithm which turned out to be the best choice. A survey on examination timetabling is provided by

Table 2. Different approaches to examination timetabling

Approach	Representative Reference(s)
Decomposition and/or construction heuristics	Qu and Burke (2007);
Simulated annealing	Thompson and Dowsland (1998); Merlot et al. (2002);
Genetic algorithms and constraint satisfaction	Marin (1998)
Grouping genetic algorithm	Erben (2001)
Iterative greedy algorithm	Caramia et al. (2001)
Tabu search	Di Gaspero and Schaerf (2001); Burke et al. (2005)
Multiobjective evolutionary algorithm	Paquete and Fonseca (2001); Cheong et al. (2007)
Greedy randomised adaptive search procedure	Casey and Thompson (2003)
Adaptive heuristic ordering strategies	Burke and Newall, (2004)
Very large neighbourhood search	Abdullah et al. (2007)
Fuzzy reasoning	Petrovic et al. (2005)
Variable neighbourhood search	Qu and Burke (2005)
Ant colony optimisation	Dowsland and Thompson (2005)
Hybrid heuristics	Azimi (2005), Ersoy et al. (2007)
Neural network	Corr et al. (2006)
Case based reasoning based investigations	Petrovic et al. (2007)
Alternating stochastic-deterministic local search	Caramia and Dell'Olmo (2007)
Hyper-heuristics	Burke et al. (2006), Pillay and Banzhaf (2008)

Table 3. Some examination timetabling problems from different universities and the initial approaches proposed to solve them

Institution	Reference	Approach
University of Nottingham	Burke et al. (1995)	Memetic algorithm
Middle East Technical University	Ergul (1996)	Genetic algorithm
École de Technologie Supérieure	Wong et al. (2002)	Genetic algorithm
University of Melbourne	Merlot et al. (2002)	A multi-phase hybrid algorithm
University of Technology MARA	Kendall and Hussin (2005)	Hyper-heuristic
Yeditepe University	Özcan et al. (2005)	Memetic algorithm

Qu et al. (2009). Carter (1986) and Carter et al. (1996a, 1996b) provide earlier surveys on examination timetabling.

3.2 Problem Description

In this study, we deal with the examination timetabling problem at Yeditepe University.

This specific problem requires finding the best timeslots for a given set of examinations under four hard constraints and a soft constraint. The hard constraints are as follows:

- **Scheduled examination restriction:** Each examination must be assigned to a timeslot only once.

- **Unscheduled examination restriction:** All the examinations must be scheduled.
- **Examination clash restriction (C_1):** A student cannot enter into more than one examination at a given time.
- **Seating capacity restriction (C_2):** The number of students seated for all exams at a timeslot cannot be more than a given capacity.

The soft constraint is as follows:

- **Examination spread preference (C_3):** A student should have at least a single timeslot in between his/her examinations in the same day.

Let E represent the set of examinations $E = \{e_1, \dots, e_j, \dots, e_n\}$ and let S denotes the ordered list of timeslots to be assigned to the examinations, $S = \{t_1, \dots, t_k, \dots, t_p\}$. An array $A = \{a_1, \dots, a_j, \dots, a_n\}$ is used as a direct representation of a candidate solution, where each entry $a_j = t_k, t_k \in S$, indicates that e_j is assigned to a timeslot t_k in S . Hence, scheduled and unscheduled examination restrictions are resolved by using this direct and complete representation that encodes a timeslot for each given examination. The quality of a given timetable (TT) with respect to a set of students and the courses upon which they enrolled (SR) is determined by calculating the weighted average of constraint violations.

$$quality(TT) = \frac{-1}{1 + \sum_{v_i} violations(C_i, TT, SR) w_i} \quad (1)$$

where $i = \{1, 2, 3\}$ and $violations$ measures the violations due to a constraint C_i in TT considering SR .

The performances of a set of Reinforcement Learning – Great Deluge hyper-heuristics are investigated over the Yeditepe University and Toronto benchmarks (Carter et al., 1996b). Yeditepe University (Faculty of Engineering) data set contains real problem instances from

each semester in three consecutive years. Bilgin et al. (2007) modified the initial data set provided in Özcan et al. (2005) with new properties and also generated a variant of Toronto benchmarks that fits into the problem formulation. The Yeditepe University data sets and Toronto benchmarks can be obtained from <http://www.cs.nott.ac.uk/~exo/research/TTML/> and <http://www.cs.nott.ac.uk/~rxq/data.htm>, respectively. The number of exams determines the size of the search space to be explored, but the difficulty of a given problem might change with respect to some other characteristics, such as the number of students or conflict density (ratio of the number of examination pairs that should not clash to the total number of examination pairs) that might implicitly or explicitly restrict the search space containing feasible solutions. Such properties for each experimental data are provided in Table 4.

4 THE REINFORCEMENT LEARNING – GREAT DELUGE HYPER-HEURISTIC

Reinforcement Learning (RL) is a general term for a set of widely used approaches that provide a way to learn how to behave when an action comes or “*how to map situations to actions*” (Sutton and Barto, 1998) through *trial-and-error* interactions (Kaelbling et al., 1996). A choice hyper-heuristic combining reinforcement learning heuristic selection and great deluge move acceptance is implemented as shown in Figure 2. As suggested in Nareyek (2003), additive adaptation rate that increments the utility value of the low level heuristic is used in the case of an improvement as a reward at step 14. This value is tested against three different negative adaptation rates, namely subtractive, divisional and root, denoted as r_1 , r_2 and r_3 , respectively for the punishment of a heuristic causing a worsening move at step 17, where u_i is the utility value of the i th low level heuristic:

$$r_1 : u_i = u_i - 1 \quad (2)$$

Table 4. Properties of Yeditepe and modified Toronto benchmark problem instances

Data Set	Instance	Exams	Students	Enrolment	Conflict Density	Days	Capacity
Yeditepe	yue20011	126	559	3486	0.18	6	450
	yue20012	141	591	3708	0.18	6	450
	yue20013	26	234	447	0.25	2	150
	yue20021	162	826	5755	0.18	7	550
	yue20022	182	869	5687	0.17	7	550
	yue20023	38	420	790	0.20	2	150
	yue20031	174	1125	6714	0.15	6	550
	yue20032	210	1185	6833	0.14	6	550
Toronto	car91 I	682	16925	56877	0.13	17	1550
	car92 I	543	18419	55522	0.14	12	2000
	ear83 I	190	1125	8109	0.27	8	350
	hecs92 I	81	2823	10632	0.42	6	650
	kfu93	461	5349	25118	0.06	7	1955
	lse91	381	2726	10918	0.06	6	635
	pur93 I	2419	30029	120681	0.03	10	5000
	rye92	486	11483	45051	0.07	8	2055
	sta83 I	139	611	5751	0.14	4	3024
	tre92	261	4360	14901	0.18	10	655
	uta92 I	622	21266	58979	0.13	12	2800
	ute92	184	2749	11793	0.08	3	1240
	yor83 I	181	941	6034	0.29	7	300

$$r_2 : u_i = u_i / 2 \quad (3)$$

$$r_3 : u_i = \sqrt{u_i} \quad (4)$$

Memory length is implemented not only in terms of adaptation rates, but also by using a lower and an upper bound on the utility values. We experimented with four different ranges in $[0, \text{number_of_heuristics} \times (5i)]$, $i = \{1, 2, 3, 4\}$. It is assumed that these bounds are checked during steps 14 and 17. Optimistic initial utility values are utilised and all utilities are set to $\lfloor 0.75 \times \text{upper bound} \rfloor$ at step 3 to support exploration. As the environment might change dynamically, bounds on the utility values are essential in order to encourage exploration in further steps. Reinforcement learning is based

on the idea that heuristics obtaining large rewards should be more likely to be selected again, while heuristics getting small rewards should be less likely to be selected again. The reinforcement scheme used returns the same reward for all heuristic choices and we use the maximal utility value to select a heuristic. Note that selecting the heuristic with this strategy (denoted as *max*) is reported in (Nareyek, 2003) to be the best choice for step 9. If there are multiple low level heuristics under consideration, since their utility values are the same, then a random choice is made. Another approach to decide whether a given total reward is small or large can be achieved by comparing that value to a relative *reference reward*, such as the average of all utility values. In addition to the maximal

Figure 2. Pseudocode of the Reinforcement Learning – Great Deluge hyper-heuristic

RL-GD ALGORITHM**Input** – n : number of heuristics, u : array holding utility value for each heuristic, $totalTime$

```

1. // initialisation
2. Generate a random complete solution  $S_{current}$ ;
3. Initialise utility values;
4.  $f_{current} = f_0 = quality(S_{current})$ ;
5.  $startTime = t = time()$ ;  $level = f_{current}$ 
6. // main loop executes until total running time allowed is exceeded
7. while (  $t < totalTime$  ) {
8.   // heuristic selection
9.    $i = selectHeuristic(u)$ ; // select a heuristic using the utility values
10.   $S_{temp} = applyHeuristic(i)$ ;
11.   $f_{temp} = quality(S_{temp})$ ;
12.   $t = time() - startTime$ ;
13.  // move acceptance
14.  if ( $f_{temp} < f_{current}$ ) then {
15.     $u_i = reward(u_i)$ ; // improving move
16.     $S_{current} = S_{temp}$ ;
17.  } else {
18.     $u_i = punish(u_i)$ ; // worsening move
19.    if (  $f_{temp} < qualityLB + (f_0 - qualityLB)(1 - t/totalTime)$  ) then
20.       $S_{current} = S_{temp}$ ; // accept the move else reject the move
21.  }
22. }
```

utility, another heuristic selection scheme that chooses a low level heuristic randomly from the ones that are over (and equal to) the average, denoted as *overAvr* is implemented. The lower bound (*qualityLB*) is set to -1 at step 19 considering the evaluation function (Equation 1) during the experiments. The reinforcement learning heuristic selection methods using the negative adaptation rates r_1 , r_2 and r_3 are referred to as RL_1 , RL_2 and RL_3 , respectively.

In this study, we employed four low level heuristics (Bilgin et al., 2007). Three of them H_1 , H_2 and H_3 are associated with three constraints C_1 , C_2 and C_3 , respectively. They probe constraint based neighbourhoods using tournament selection to resolve violations of a corresponding constraint only. Each low level heuristic operates as follows:

1. **$H_1(H(x))$:** This heuristic chooses a number of examinations randomly that violate $x=C_1$ and this number is referred to as *toursize1*. Then, the examination causing the largest number of violations is selected. This examination is reassigned to a timeslot from a randomly selected set of timeslots (*tour-size2*) which generates the least number of $x=C_1$ violations.

2. **H_2 :** Using a tournament strategy, a number of timeslots (*toursize3*) with capacity constraint violations are selected. Examinations in the timeslot that has the largest number of violations are marked for further processing. The examination with the largest number of enrolled students is rescheduled. Then, this examination is reassigned to a timeslot from a randomly selected set of timeslots (*toursize4*) which generates the least amount of C_2 violations.
3. **H_3 :** This heuristic employs the same strategy as described in $H(x)$ with $x=C_3$.
4. **H_4 :** This heuristic makes a pass over all the examinations and reschedules the examination under consideration with a probability of $1/number_of_examinations$.

5 EXPERIMENTAL RESULTS

The experiments were performed on a Pentium IV 3 GHz LINUX (Fedora Core 8) PC with 2 GB memory. Each hyper-heuristic is tested on each instance for 50 trials and each trial is terminated after 600 CPU seconds. Initial experiments were performed for parameter tuning. Unless mentioned otherwise, the utility value upper bound is fixed as 40 and *max* is used as the utility based heuristic selection

strategy within the reinforcement learning hyper-heuristics. A sample run is performed for sta83 I using a reinforcement learning – great deluge hyper-heuristic. Figure 3 illustrates the change in utility for each low level heuristic and the improvement based on different negative adaptation rates for this run. If a low level heuristic worsens the solution after a number of successive improving moves, the best heuristic still gets a chance to operate on the candidate solution. The frequency of that chance is determined by the negative adaptation rate. For example, H_3 gets selected more frequently when the adaptation rate is subtractive(/divisional) as compared to divisional(/root) before the optimistic utility values of all heuristics reduces toward the lower bound (see Figure 3). The more severe (high) this rate is, the more exploration of different heuristics is favoured. All the low level heuristics get invoked within tens of steps while using divisional and root adaptation rates (Figure 3. (b) and (c)), whereas only two heuristics get invoked while using a subtractive adaptation rate (Figure 3. (a)).

The results show that all low level heuristics are valuable in improving a candidate solution. It seems that the quality of a solution is improved slowly whenever a slow negative adaptation rate is used. Naturally, there is still the chance of getting stuck at a local optimum in the long run. A low level heuristic at any step is chosen with a probability in $\{1.00, 0.50, 0.33, 0.25\}$. The reinforcement learning heuristic selection arranges these probabilities dynamically during the search process. As the search stagnates and a local optimum is found, the probability of 0.25 is used more frequently while selecting a low level heuristic.

In order to observe the effect of memory length via different combinations of negative adaptation {subtractive, divisional, root} and upper bound for the utility values {20, 40, 60, 80} experiments have been performed on modified Toronto problem instances. As a total twelve different choices are executed for each data and each choice is ranked from 1 (best) to 12 (worst) using the results from the runs. The average rank of a choice over all data and

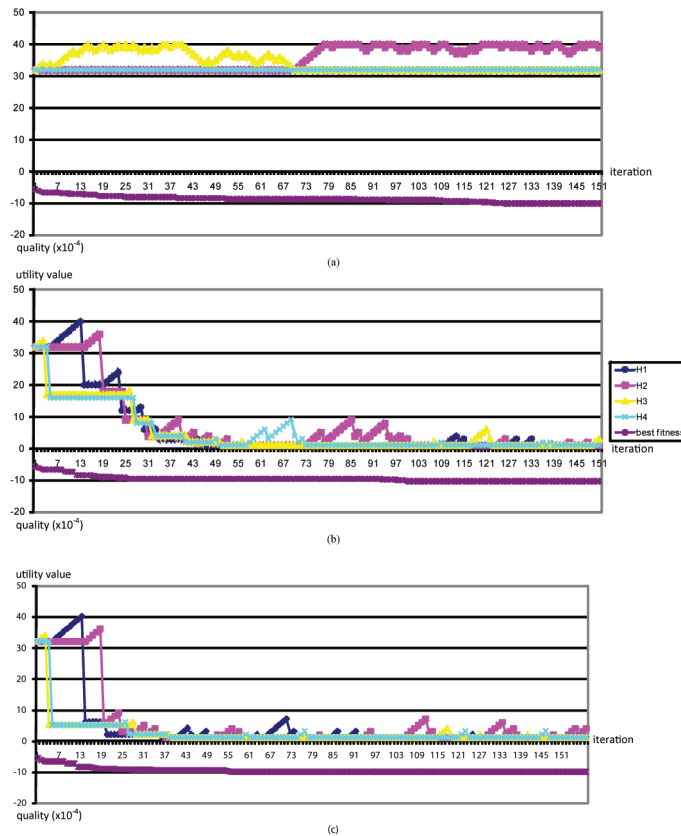
the related standard deviation are provided in Figure 4.

Determining the best adaptation rate, which is also vital to adjust the memory length, seems to be a key issue in fully utilising a reinforcement learning scheme within a hyper-heuristic. Different adaptation rates might yield different performances. The results show that the RL_1 heuristic selection method with a utility upper bound of 40 delivers the best average performance when combined with the great deluge method as a hyper-heuristic. Yet, this performance variation is not statistically better than the rest.

Using the best configuration from the previous set of experiments, another experiment is performed over modified Toronto problem instances to compare the average performances of utility based heuristic selection schemes; *max* and *overAvr*. Figure 5 summarises the experimental results. Maximal utility selection performs slightly better than *overAvr* with an average rank of 1.42 for the modified problem instances {car91 I, car92 I, kfu93, lse91, pur93 I, rye92, ute92}. There is a tie for sta83 I. Still, the performance difference between *max* and *overAvr* is not statistically significant. There might still be potential for a future use of this approach, as, in general, *overAvr* shows success in solving problem instances with relatively high conflict densities.

The reinforcement learning heuristic selection method, which is referred to as RL_1 , utilises additive reward and subtractive punishment schemes with a utility upper bound of 40 and *max*. RL_1 is combined with great deluge and tested against simple random – great deluge hyper-heuristic during the final set of experiments. The results are provided in Table 5. The percentage improvement in the table uses whichever approach generates a better result (average best of fifty trials) as the baseline for comparison. The simple random – great deluge hyper-heuristic generates better average performance for eight problem instances. It is especially successful in solving the Yeditepe problem instances which are smaller and have low conflict densities when

Figure 3. Plot of utility value for each low level heuristic and quality versus iteration on sta83 I using the reinforcement learning – great deluge hyper-heuristic based on (a) subtractive, (b) divisional and (c) root negative adaptation rates with max, utility upper bound=40, respectively



compared with the modified Toronto instances. Yet, RL_1 improves the performance of simple random hyper-heuristic with the great deluge move acceptance on eleven problem instances (out of twenty one problem instances), and for two problem instances there is a tie.

Finally, RL_1 – great deluge is compared to two previous studies. Bilgin et al. (2007) showed that the choice function – simulated annealing hyper-heuristic, out of thirty five approaches, performs the best for examination timetabling. In a recent study, Özcan et al. (2009) introduced a new move acceptance strategy that can be used in hyper-heuristics. The experiments resulted in the success of a simple random – late accep-

tance hyper-heuristic, performing even better than choice function – simulated annealing. Both of these approaches are compared to the RL_1 – great deluge in Table 6.

A hyper-heuristic learns how to make *good* moves through both heuristic selection and move acceptance. If a move is rejected, then the selected heuristic is annulled. Hence, if a hyper-heuristic uses a simple random heuristic selection, it does not imply that there is no learning within that hyper-heuristic. The late acceptance strategy uses a fixed length memory to hold the quality of some previously visited solutions. Simple random heuristic selection diversifies the search, while the late acceptance

Figure 4. Average rank of hyper-heuristics using different heuristic selection methods $\{RL_1, RL_2, RL_3\}$ with a utility upper bound of (a) 20, (b) 40, (c) 60 and (d) 80 over modified Toronto instances

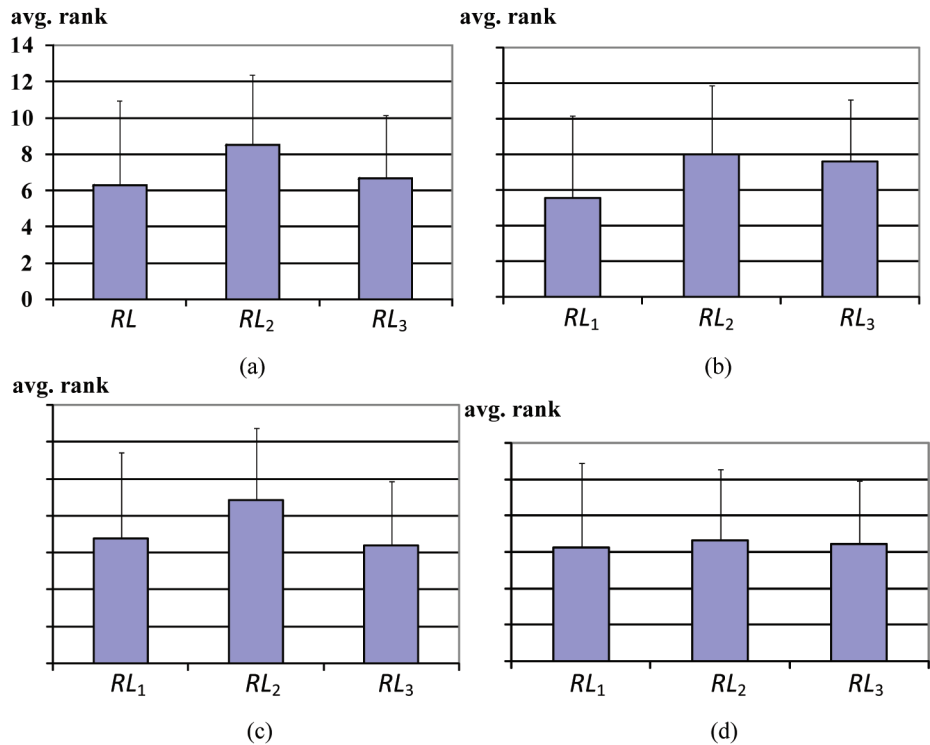


Figure 5. Comparison of utility value based heuristic selection schemes over modified Toronto instances based on their average ranks

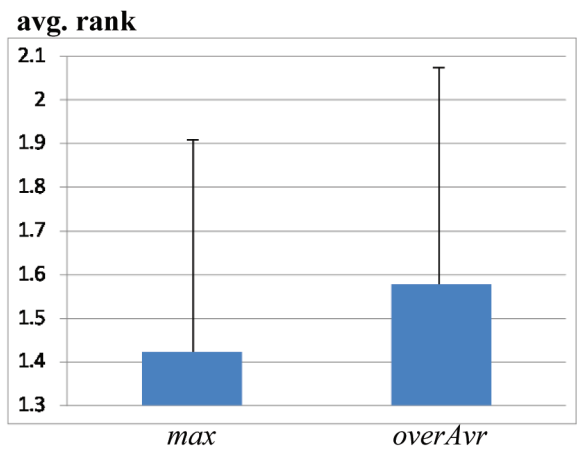


Table 5. Comparison of reinforcement learning and simple random heuristic selection within a hyper-heuristic using the great deluge acceptance move method. “ $\geq$ ” and “ $\approx$ ” indicate “is better than” and “delivers a similar performance”, respectively. ‘Percentage improvement’ uses the average best quality obtained in fifty runs for the better approach as the baseline for comparison.

Instance	Comparison	%-improv.
car91 I	$RL_1\text{-GD} \geq \text{SR-GD}$	1.70
car92 I	$RL_1\text{-GD} \geq \text{SR-GD}$	1.68
ear83 I	$RL_1\text{-GD} \geq \text{SR-GD}$	2.02
hecs92 I	$\text{SR-GD} \geq RL_1\text{-GD}$	11.85
kfu93	$RL_1\text{-GD} \geq \text{SR-GD}$	2.09
lse91	$RL_1\text{-GD} \geq \text{SR-GD}$	2.47
pur93 I	$\text{SR-GD} \geq RL_1\text{-GD}$	0.32
rye92	$RL_1\text{-GD} \geq \text{SR-GD}$	3.43
sta83 I	$RL_1\text{-GD} \geq \text{SR-GD}$	0.06
tre92	$\text{SR-GD} \geq RL_1\text{-GD}$	5.05
uta92 I	$\text{SR-GD} \geq RL_1\text{-GD}$	0.23
ute92	$RL_1\text{-GD} \geq \text{SR-GD}$	0.28
yor83 I	$RL_1\text{-GD} \geq \text{SR-GD}$	1.13
yue20011	$RL_1\text{-GD} \approx \text{SR-GD}$	0.00
yue20012	$RL_1\text{-GD} \geq \text{SR-GD}$	0.53
yue20013	$RL_1\text{-GD} \approx \text{SR-GD}$	0.00
yue20021	$\text{SR-GD} \geq RL_1\text{-GD}$	1.49
yue20022	$\text{SR-GD} \geq RL_1\text{-GD}$	0.83
yue20023	$\text{SR-GD} \geq RL_1\text{-GD}$	0.71
yue20031	$RL_1\text{-GD} \geq \text{SR-GD}$	5.26
yue20032	$\text{SR-GD} \geq RL_1\text{-GD}$	0.88

strategy intensifies the search process by approving the better moves based on its memory. This course of action can also be considered to be learning. Reinforcement learning not only improves on simple random heuristic selection when combined with great deluge but it also generates better results as compared to another learning hyper-heuristic; choice function – simulated annealing. Moreover, its performance is comparable to the performance of the simple random – late acceptance hyper-heuristic.

6 CONCLUSION

In this article, a hyper-heuristic can be thought of as a methodology that guides the search process at a high level by controlling a set of perturbation low level heuristics. A single point search hyper-heuristic framework that combines successive stages of heuristic selection and move acceptance is employed during the experiments. It has been observed in Bilgin et al. (2007) that simple random heuristic selection in a great deluge based hyper-heuristic performed well for solving an examination timetabling

Table 6. Comparison of RL_1 –great deluge to the previous studies; (1) choice function – simulated annealing (Bilgin et al., 2007), (2) simple random – late acceptance (Özcan et al., 2009). Each approach is ranked from 1 (best) to 3 (worst) for each modified Toronto problem instance.

Instance	RL_1 –GD	(1) CF–SA	(2) SR–LAS
car91 I	2	3	1
car92 I	1	3	2
ear83 I	1	2	3
hecs92 I	1	3	2
kfu93	1	3	2
lse91	2	3	1
pur93 I	2	3	1
rye92	2	3	1
sta83 I	1	3	2
tre92	1	3	2
uta92 I	2	3	1
ute92	3	1	2
yor83 I	2	3	1
avg.	1.62	2.77	1.62

problem encountered at Yeditepe University every semester. The same problem is used as a case study to investigate reinforcement learning with different components as heuristic selection methods in place of simple random.

There are different ways of implementing the great deluge move acceptance. In this study, a linear decreasing rate is adopted as suggested in previous studies (Bilgin et al., 2007; Kendal and Mohamad, 2004). Additionally, CPU time is used as a termination criterion and hence, CPU time is used as a part of the *level* decreasing scheme. This level sets a goal for the chosen heuristic in the case of a worsening move after its invocation. The resultant *bad* move is required to have a better quality than the goal, otherwise it is rejected. Using CPU time within the great deluge provides an additional side benefit. The running times of low level heuristics might be different. If a heuristic takes a short time to execute, then the expectation on the quality of the resultant move is lower as compared to a heuristic which takes a longer time to execute. This strategy seems to be viable and great del-

uge based on this strategy performs well as a hyper-heuristic component (Bilgin et al., 2007; Özcan et al., 2008).

A learning hyper-heuristic usually attempts to follow the best moves within a given period of time using some type of memory to make better future decisions. Bai et al. (2007b) observed that memory length is vital in learning and that the use of a learning mechanism with *short* term memory combined with a simulated annealing move acceptance generated the best results in their experiments over a set of course timetabling problems. They used weighted adaptation and tested various *learning rates* that adjust the influence of rewards compiled at different stages of search. In this study, other factors regarding the memory that affect the learning process, such as adaptation rate, lower and upper bounds on the utility values are identified and tested using relatively short memory lengths as suggested before. Furthermore, two different heuristic selection strategies, based on the utility values, are assessed. Considering only the adaptation rates, the results support the previous

findings in Nareyek (2003). The combination of slow adaptation rates (additive/subtractive) seems to perform the best. The reinforcement learning – great deluge hyper-heuristic with the settings {lower bound=0, upper bound=40, heuristic selection strategy=*max*, positive adaptation rate=*additive*, negative adaptation rate=*subtractive*} improves the performance of the simple random – great deluge hyper-heuristic for solving the examination timetabling problem instances studied in this article.

The choice of reinforcement learning heuristic selection components affects the memory length which in turn also affects the intensification and diversification processes during the search. The set of low level heuristics contains neighbourhood operators that attempt to resolve conflicts due to a specific constraint type without considering whether the move will cause other constraint violations or not. The reinforcement heuristic selection chooses a constraint based heuristic as long as a given solution is improved in terms of overall quality. If the same heuristic starts generating worsening moves, the reinforcement learning heuristic selection method still supports the selection of the same heuristic until its utility value decreases to the same value as another one (or others). As soon as there is more than one low level heuristic with the same utility value, one of them is chosen randomly to give (another) chance to the other heuristic(s) for improving the candidate solution in hand. The intensification and diversification processes over the problem domain occur as a result of a dynamic interaction between heuristic selection, move acceptance and low level heuristics. The intensification process is activated whenever an improving move is accepted and continues as long as an improvement in the overall quality is achieved. Diversification arises in three ways which is mainly based on the great deluge move acceptance. Firstly, whenever there is a worsening move, the intensification phase ends. This worsening move might still be accepted by the great deluge allowing a jump to

other promising regions of the search space. Secondly, a low level heuristic is included within the hyper-heuristic that perturbs a given candidate solution randomly using a small step size. If this heuristic is selected for invocation, it acts as a diversification mechanism. Finally, the random choice, in the case of equal utility values, provides an additional diversification mechanism. The reinforcement learning – great deluge hyper-heuristic with the given low level heuristics attempts to balance intensification and diversification automatically. It turns out to be successful in this attempt as this hyper-heuristic delivers a good performance over the Yeditepe University examination timetabling problem.

REFERENCES

- Abdullah, S., Ahmadi, S., Burke, E. K., & Dror, M. (2007). Investigating Ahuja-Orlins large neighbourhood search for examination timetabling. *OR-Spektrum*, 29(2), 351–372. doi:10.1007/s00291-006-0034-7
- Alkan, A., & Özcan, E. (2003). Memetic algorithms for timetabling. In *Proceedings of 2003 IEEE Congress on Evolutionary Computation* (pp. 1796-1802).
- Anagnostopoulos, A., Michel, L., Hentenryck, P. V., & Vergados, Y. (2006). A simulated annealing approach to the traveling tournament problem. *Journal of Scheduling*, 9, 177–193. doi:10.1007/s10951-006-7187-8
- Ayob, M., & Kendall, G. (2003). A monte carlo hyper-Heuristic to optimise component placement sequencing for multi head placement machine. In *Proceedings of the International Conference on Intelligent Technologies (InTech '03)* (pp. 132-141).
- Azimi, Z. N. (2005). Hybrid heuristics for examination timetabling problem. *Applied Mathematics and Computation*, 163(2), 705–733. doi:10.1016/j.amc.2003.10.061
- Bai, R., Blazewicz, J., Burke, E., Kendall, G., & McCollum, B. (2007a). *A simulated annealing hyper-heuristic methodology for flexible decision support* (Computer Science Tech. Rep. No. NOTTCS-TR-2007-8). Nottingham, UK: University of Nottingham.

- Bai, R., Burke, E. K., Kendall, G., & McCollum, B. (2007b). Memory length in hyper-heuristics: an empirical study. In *Proceedings of 2007 IEEE Symposium on Computational Intelligence in Scheduling (CISched2007)* (pp. 173-178).
- Bai, R., & Kendall, G. (2003). An investigation of automated planograms using a simulated annealing based hyper-heuristics. In T. Ibaraki, K. Nonobe, & M. Yagiura (Eds.), *Meta-heuristics: Progress as Real Problem Solvers, selected papers from the 5th Metaheuristics International Conference (MIC'03)* (pp. 87-108). New York: Springer.
- Bilgin, B., Özcan, E., & Korkmaz, E. E. (2007). An experimental study on hyper-heuristics and exam scheduling. In *Proceedings of the 6th International Conference on the Practice and Theory of Automated Timetabling (PATAT'06)* (LNCS 3867, pp. 394-412).
- Broder, S. (1964). Final examination scheduling. *Communications of the ACM*, 7(8), 494-498. doi:10.1145/355586.364824
- Burke, E., Bykov, Y., Newall, J. P., & Petrovic, S. (2003). A time-predefined approach to course timetabling. [YUJOR]. *Yugoslav Journal of Operations Research*, 13(2), 139-151. doi:10.2298/YJOR0302139B
- Burke, E., Hyde, M., Kendall, G., Ochoa, G., Özcan, E., & Rong, Q. (2009b). *A survey of hyper-heuristics* (Computer Science Tech. Rep. No. NOTTCS-TR-SUB-0906241418-2747). Nottingham, UK: University of Nottingham.
- Burke, E., Hyde, M., Kendall, G., Ochoa, G., Özcan, E., & Woodward, J. R. (2009a). Exploring hyper-heuristic methodologies with genetic programming. In C. L. Mumford & L. C. Jain (Eds.), *Computational intelligence: Collaboration, fusion and emergence* (pp. 177-201). New York: Springer.
- Burke, E., Hyde, M., Kendall, G., Ochoa, G., Özcan, E., & Woodward, J. R. (2009c). *A classification of hyper-heuristic approaches* (Computer Science Tech. Rep. No. NOTTCS-TR-SUB-0907061259-5808). Nottingham, UK: University of Nottingham.
- Burke, E., Kendall, G., Newall, J., Hart, E., Ross, P., & Schulenburg, S. (2003a). Hyper-heuristics: An emerging direction in modern search technology. In F. W. Glover & G. A. Kochenberger (Eds.), *Handbook of Metaheuristics* (Vol. 57, pp. 457-474). Dordrecht, The Netherlands: Kluwer International Publishing.
- Burke, E., Kendall, G., & Soubeiga, E. (2003b). A tabu-search hyper-heuristic for timetabling and rostering. *Journal of Heuristics*, 9, 451-470. doi:10.1023/B:HEUR.0000012446.94732.b6
- Burke, E. K., Dror, M., Petrovic, S., & Qu, R. (2005). Hybrid graph heuristics within a hyper-heuristic approach to exam timetabling problems. In B. L. Golden, S. Raghavan, & E. A. Wasil (Eds.), *The next wave in computing, optimization, and decision technologies: Proceedings of the 9th Informatics Computing Society Conference* (pp. 79-91). Springer.
- Burke, E. K., Elliman, D. G., Ford, P. H., & Weare, R. F. (1996a). Examination timetabling in British universities - a survey. In E. K. Burke & P. Ross (Eds.), *Selected Papers from the 1st International Conference on the Practice and Theory of Automated Timetabling*, Edinburgh (LNCS 1153, pp. 76-92).
- Burke, E. K., McCollum, B., Meisels, A., Petrovic, S., & Qu, R. (2007). A graph-based hyper-heuristic for educational timetabling problems. *European Journal of Operational Research*, 176(1), 177-192. doi:10.1016/j.ejor.2005.08.012
- Burke, E. K., & Newall, J. P. (2004). Solving examination timetabling problems through adaptation of heuristic orderings: Models and algorithms for planning and scheduling problems. *Annals of Operations Research*, 129, 107-134. doi:10.1023/B:ANOR.0000030684.30824.08
- Burke, E. K., Newall, J. P., & Weare, R. F. (1996b). A memetic algorithm for university exam timetabling. In E. K. Burke & P. Ross (Eds.), *Selected Papers from the 1st International Conference on the Practice and Theory of Automated Timetabling*, Edinburgh (LNCS 1153, pp. 241-250).
- Burke, E. K., Petrovic, S., & Qu, R. (2006). Case based heuristic selection for timetabling problems. *Journal of Scheduling*, 9, 115-132. doi:10.1007/s10951-006-6775-y
- Caramia, M., & Dell'Olmo, P. (2007). Coupling stochastic and deterministic local search in examination timetabling. *Operations Research*, 55(2). doi:10.1287/opre.1060.0354
- Caramia, M., Dell'Olmo, P., & Italiano, G. F. (2001). New algorithms for examination timetabling. In S. Naher & D. Wagner (Eds.), *Algorithm Engineering 4th International Workshop (WAE'00)* (LNCS 1982, pp. 230-241).

- Carter, M. W. (1986). A survey of practical applications of examination timetabling algorithms. *Operations Research*, 34, 193–202. doi:10.1287/opre.34.2.193
- Carter, M. W., & Laporte, G. (1996a). Recent developments in practical examination timetabling. In E. K. Burke & P. Ross (Eds.), *Selected Papers from the 1st International Conference on the Practice and Theory of Automated Timetabling*, Edinburgh (LNCS 1153, pp. 3-21).
- Carter, M. W., Laporte, G., & Lee, S. (1996b). Examination timetabling: Algorithmic strategies and applications. *The Journal of the Operational Research Society*, 47(3), 373–383.
- Casey, S., & Thompson, J. (2003). GRASping the examination scheduling problem. In E. K. Burke & P. De Causmaecker (Eds.), *Practice and theory of automated timetabling: Selected Papers from the 4th International Conference* (LNCS 2740, pp. 232-244).
- Chakhlevitch, K., & Cowling, P. I. (2008). Hyperheuristics: Recent developments. In *Adaptive and Multilevel Metaheuristics* (pp. 3-29).
- Chen, P.-C., Kendall, G., & Berghe, G. V. (2007). An ant based hyper-heuristic for the travelling tournament problem. In *Proceedings of IEEE Symposium of Computational Intelligence in Scheduling (CISched'07)* (pp. 19-26).
- Cheong, C. Y., Tan, K. C., & Veeravalli, B. (2007). Solving the exam timetabling problem via a multi-objective evolutionary algorithm – a more general approach. In *Proceedings of the IEEE Symposium on Computational Intelligence in Scheduling (CISched'07)* (pp. 165-172).
- Cole, A. J. (1964). The preparation of examination timetables using a small-store computer. *The Computer Journal*, 7, 117–121. doi:10.1093/comjnl/7.2.117
- Corr, P. H., McCollum, B., McGreevy, M. A. J., & McMullan, P. (2006). A new neural network based construction heuristic for the examination timetabling problem. In T. P. Runarsson et al. (Eds.), *PPSN IX* (LNCS 4193, pp. 392-401).
- Cowling, P., & Chakhlevitch, K. (2003). Hyperheuristics for managing a large collection of low level heuristics to schedule personnel. In *Proceedings of the IEEE Congress on Evolutionary Computation (CEC'03)* (pp. 1214-1221).
- Cowling, P., Kendall, G., & Han, L. (2002). An investigation of a hyperheuristic genetic algorithm applied to a trainer scheduling problem. In *Proceedings of the IEEE Congress on Evolutionary Computation (CEC'02)* (pp. 1185-1190).
- Cowling, P., Kendall, G., & Soubeiga, E. (2001a). A hyperheuristic approach to scheduling a sales summit. In *Proceedings of the 3rd International Conference on Practice and Theory of Automated Timetabling (PATAT'00)* (pp. 176-190). Springer-Verlag.
- Cowling, P., Kendall, G., & Soubeiga, E. (2001b). A parameter-free hyperheuristic for scheduling a sales summit. In *Proceedings of 4th Metaheuristics International Conference (MIC'01)* (pp. 127-131).
- Cuesta-Canada, A., Garrido, L., & Terashima-Marin, H. (2005). Building hyper-heuristics through ant colony optimization for the 2d bin packing problem. In *Proceedings of the 9th International Conference on Knowledge-Based Intelligent Information and Engineering Systems (KES'05)* (LNCS 3684, pp. 654-660).
- Denzinger, J., Fuchs, M., & Fuchs, M. (1997). High performance ATP systems by combining several ai methods. In *Proceedings of the Fifteenth International Joint Conference on Artificial Intelligence (IJCAI 97)* (pp. 102-107).
- Di Gaspero, L., & Schaerf, A. (2001). Tabu search techniques for examination timetabling. In E. K. Burke & W. Erben (Eds.), *Selected Papers from the 3rd International Conference on the Practice and Theory of Automated Timetabling (PATAT'00)* (LNCS 2079, pp. 104-117).
- Dowsland, K., & Thompson, J. (2005). Ant colony optimization for the examination scheduling problem. *The Journal of the Operational Research Society*, 56(4), 426–438. doi:10.1057/palgrave.jors.2601830
- Dowsland, K. A., Soubeiga, E., & Burke, E. (2007). A simulated annealing hyper-heuristic for determining shipper sizes. *European Journal of Operational Research*, 179(3), 759–774. doi:10.1016/j.ejor.2005.03.058
- Dueck, G. (1993). New optimization heuristics: the great deluge algorithm and the record-to-record travel. *Journal of Computational Physics*, 104, 86–92. doi:10.1006/jcph.1993.1010

- Erben, W. (2001). A grouping genetic algorithm for graph colouring and exam timetabling. In K. Burke & W. Erben (Eds.), *Proceedings of the 3rd International Conference on the Practice and Theory of Automated Timetabling (PATAT'00)* (LNCS 2079, pp. 132-156).
- Ergul, A. (1996). GA-based examination scheduling experience at Middle East Technical University. In E. K. Burke & P. Ross (Eds.), *Selected Papers from the 1st International Conference on the Practice and Theory of Automated Timetabling*, Edinburgh (LNCS 1153, pp. 212-226).
- Ersoy, E., Özcan, E., & Uyar, S. (2007). Memetic algorithms and hyperhill-climbers. In *Proceedings of the 3rd Multidisciplinary International Scheduling Conference: Theory and Applications (MISTA'07)* (pp. 156-166).
- Even, S., Itai, A., & Shamir, A. (1976). On the complexity of timetable and multicommodity Flow problems. *SIAM Journal on Computing*, 5(4), 691-703. doi:10.1137/0205048
- Fisher, H., & Thompson, G. L. (1961). *Probabilistic learning combinations of local job-shop scheduling rules*. Paper presented at the *Factory Scheduling Conference*, Carnegie Institute of Technology.
- Han, L., & Kendall, G. (2003). An investigation of a tabu assisted hyper-heuristic genetic algorithm. *Proceedings of the IEEE Congress on Evolutionary Computation (CEC'03)*, 3, (pp. 2230-2237).
- Kaelbling, L. P., Littman, M., & Moore, A. (1996). Reinforcement learning: A survey. *Journal of Artificial Intelligence Research*, 4, 237-285.
- Keller, R. E., & Poli, R. (2007). Cost-benefit investigation of a genetic-programming hyper-heuristic. In *Proceedings of the 8th International Conference on Artificial Evolution (EA'07)*, Tours, France (pp. 13-24).
- Kendall, G., & Hussin, N. M. (2005). Tabu search hyper-heuristic approach to the examination timetabling problem at university of technology MARA. In E. K. Burke and M. Trick (Eds.), *Proceedings of the 5th International Conference on the Practice and Theory of Automated Timetabling (PATAT'04)* (LNCS 3616, pp. 270-293).
- Kendall, G., & Mohamad, M. (2004). Channel assignment in cellular communication using a great deluge hyper-heuristic. In *Proceedings of the 12th IEEE International Conference on Network (ICON'04)* (pp. 769-773).
- Kirkpatrick, S., Gelatt, C., & Vecchi, M. (1983). Optimization by simulated annealing. *Science*, 220, 671-680. doi:10.1126/science.220.4598.671
- Marin, H. T. (1998). *Combinations of GAs and CSP strategies for solving examination timetabling problems*. Unpublished PhD thesis, Instituto Tecnológico y de Estudios Superiores de Monterrey.
- Marín-Blázquez, J., & Schulenburg, S. (2005). A hyper-heuristic framework with XCS: Learning to create novel problem-solving algorithms constructed from simpler algorithmic ingredients. In T. Kovacs, X. Llorà, K. Takadama, P. Lanzi, W. Stolzmann, & S. Wilson (Eds.), *Proceedings of the 8th International Workshop on Learning Classifier Systems (IWLC'S'05)* (LNCS 4399, pp. 193-218).
- Merlot, L. T. G., Boland, N., Hughes, B. D., & Stuckey, P. J. (2002). A hybrid algorithm for the examination timetabling problem. In E. K. Burke & P. De Causmaecker (Eds.), *Proceedings of the 4th International Conference on the Practice and Theory of Automated Timetabling (PATAT'02)* (LNCS 1153, pp. 207-231).
- Nareyek, A. (2003). Choosing search heuristics by non-stationary reinforcement learning. In *Metaheuristics: Computer decision-making* (pp. 523-544). Dordrecht, The Netherlands: Kluwer Academic Publishers.
- Ouelhadj, D., & Petrovic, S. (2008). A cooperative distributed hyper-heuristic framework for scheduling. In *Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics (SMC'08)* (pp. 2560-2565).
- Özcan, E., Bilgin, B., & Korkmaz, E. E. (2008). A comprehensive analysis of hyper-heuristics. *Intelligent Data Analysis*, 12, 3-23.
- Özcan, E., Bykov, Y., Birben, M., & Burke, K. E. (2009). Examination timetabling using late acceptance hyper-heuristics. In *Proceedings of the 2009 IEEE Congress on Evolutionary Computation (CEC'09)* (pp. 997-1004).
- Özcan, E., & Ersoy, E. (2005). Final exam scheduler – FES. In *Proceedings of the IEEE Congress on Evolutionary Computation (CEC'05)* (Vol. 2, pp. 1356-1363).
- Paquete, L. F., & Fonseca, C. M. (2001). A study of examination timetabling with multiobjective evolutionary algorithms. In *Proceedings of the 4th Metaheuristics International Conference (MIC'01)* (pp. 149-154).

- Petrovic, S., Patel, V., & Yang, Y. (2005). Examination timetabling with fuzzy constraints. In E. K. Burke & M. Trick (Eds.), *Proceedings of the 5th International Conference on the Practice and Theory of Automated Timetabling (PATAT'05)* (LNCS 3616, pp. 313-333), Springer.
- Petrovic, S., Yang, Y., & Dror, M. (2007). Case-based selection of initialisation heuristics for metaheuristic examination timetabling. *Expert Systems with Applications: An International Journal*, 33(3), 772-785. doi:10.1016/j.eswa.2006.06.017
- Pillay, N., & Banzhaf, W. (2008). A study of heuristic combinations for hyper-heuristic systems for the uncapacitated examination timetabling problem. *European Journal of Operational Research*. doi:10.1016/j.ejor.2008.07.023
- Qu, R., Burke, E. K., & McCollum, B. (2008). Adaptive automated construction of hybrid heuristics for exam timetabling and graph colouring problems. *European Journal of Operational Research*, 198(2), 392-404. doi:10.1016/j.ejor.2008.10.001
- Qu, R., Burke, E. K., McCollum, B., Merlot, L. T., & Lee, S. Y. (2009). A survey of search methodologies and automated system development for examination timetabling. *Journal of Scheduling*, 12(1), 55-89. doi:10.1007/s10951-008-0077-5
- Rattadilok, P., Gaw, A., & Kwan, R. (2005). Distributed choice function hyper-heuristics for timetabling and scheduling. In *Proceedings of the 5th International Conference on the Practice and Theory of Automated Timetabling (PATAT'2004)* (pp. 51-67).
- Ross, P. (2005). Hyper-heuristics. In: E. K. Burke & G. Kendall (Eds.), *Search methodologies: Introductory tutorials in optimization and decision support techniques* (Ch. 17, pp. 529-556). New York: Springer.
- Sutton, R. S., & Barto, A. G. (1998). *Reinforcement learning: An introduction*. Cambridge, MA: MIT Press.
- Terashima-Marin, H. T., Moran-Saavedra, A., & Ross, P. (2005). Forming hyper-heuristics with GAs when solving 2D-regular cutting stock problems. In *Proceedings of the 2005 IEEE Congress on Evolutionary Computation* (Vol. 2, pp. 1104-1110).
- Terashima-Marin, H. T., Ross, P., & Valenzuela-Rendon, M. (1999). Evolution of constraint satisfaction strategies in examination timetabling. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO'99)* (pp. 635-642).
- Terashima-Marin, H. T., Zarate, C. J. F., Ross, P., & Valenzuela-Rendon, M. (2007). Comparing two models to generate hyper-heuristics for the 2d-regular bin-packing problem. In *Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation (GECCO'07)* (pp. 2182-2189).
- Thompson, J. M., & Dowsland, K. A. (1998). A robust simulated annealing based examination timetabling system. *Computers & Operations Research*, 25, 637-648. doi:10.1016/S0305-0548(97)00101-9
- Vazquez-Rodriguez, J. A., Petrovic, S., & Salhi, A. (2007). A combined meta-heuristic with hyper-heuristic approach to the scheduling of the hybrid flow shop with sequence dependent setup times and uniform machines. In P. Baptiste, G. Kendall, A. Munier-Kordon, & F. Sourd (Eds.), *Proceedings of the 3rd Multi-disciplinary International Scheduling Conference: Theory and Applications (MISTA'07)*, Paris (pp. 506-513).
- Wong, T., Cote, P., & Gely, P. (2002). Final exam timetabling: a practical approach. In *Proceedings of the IEEE Canadian Conference on Electrical and Computer Engineering*, 2, 726-731.

Ender Ozcan is a science and innovation lecturer with the Automated Scheduling, Optimisation and Planning (ASAP) Research Group in the School of Computer Science at the University of Nottingham, UK. He received his PhD from the Department of Computer and Information Science at Syracuse University, NY, USA in 1998. He worked as a lecturer in the Department of Computer Engineering at Yeditepe University, Istanbul, Turkey from 1998-2007. He established and led the Artificial Intelligence research group from 2002 and awarded two research grants from TUBITAK. He served as the deputy head of the Department from 2004-2007. Dr. Ozcan joined the ASAP group as a senior research fellow in 2008. He has been serving as an executive committee member for the LANCS initiative, which is one of the largest Science and Innovation Rewards given by EPSRC (Engineering and Physical Sciences Research Council, UK). His research interests and

activities lie at the interface of computer science and operational research. He has been leading studies in the field of metaheuristics focusing on evolutionary algorithms (memetic algorithms, PSO), hyper-heuristics, and their applications to the real-world and theoretical problems. Dr. Ozcan has published over 55 refereed papers. He has been a member of the program committees in major international conferences and refereeing for reputable journals. He has co-organised five workshops on hyper-heuristics and metaheuristics and he is the guest co-editor of the forthcoming first special issue on hyper-heuristics (Journal of Heuristics, 2009).

Mustafa Misir received his BSc and MSc degrees in computer engineering from Yeditepe University in 2007 and 2008, respectively. He was awarded as a student assistant during his undergraduate study. He worked as a teaching and research assistant during his graduate study in the Artificial Intelligence research group at Yeditepe University. Currently, he is studying towards PhD in the Department of Computer Science (Informatics) at Katholieke Universiteit Leuven as a research fellow. He is also working as a researcher in the CODES Research Group, Katholieke Universiteit Leuven, Campus Kortrijk and IT Research Group, KaHo Sint-Lieven. His research interests include combinatorial optimization, hyper-heuristics, meta-heuristics and reinforcement learning.

Gabriela Ochoa is a senior research fellow with the Automated Scheduling, Optimisation and Planning (ASAP) Research Group in the School of Computer Science at the University of Nottingham, since October 2006, where she coordinates a project on 'Automated Heuristic Design'. She received her PhD in computer science and artificial intelligence from the University of Sussex, UK, in 2001. Gabriela Ochoa has been involved with inter-disciplinary research, and foundations and applications of evolutionary algorithms since the mid 90s, and more recently with meta-heuristics and hyper-heuristics. She has published over 30 refereed research articles, serves on the program committees of major conferences in evolutionary computation and meta-heuristics, and has refereed for reputable journals in these fields. Ochoa has recently dictated a tutorial on hyper-heuristics, and proposed and co-organised two workshops on hyper-heuristic methodologies held as part of reputable evolutionary computation conferences. She is the guest co-editor of the first special issue on hyper-heuristics (Journal of Heuristics, 2009).

Edmund K. Burke is dean of the Faculty of Science at the University of Nottingham and he leads the Automated Scheduling, Optimisation and Planning (ASAP) Research Group in the School of Computer Science. He is a member of the EPSRC Strategic Advisory Team for Mathematics. He is a fellow of the Operational Research Society and the British Computer Society and he is a member of the UK Computing Research Committee (UKCRC). Prof. Burke is editor-in-chief of the Journal of Scheduling, area editor (for combinatorial optimisation) of the Journal of Heuristics, associate editor of the INFORMS Journal on Computing, associate editor of the IEEE Transactions on Evolutionary Computation and a member of the editorial board of Memetic Computing. He is also the research director of EventMAP Ltd. and a director of Aptia Solutions Ltd, both of which are spin out companies from the ASAP group. Prof. Burke has played a leading role in the organisation of several major international conferences in his research field in the last few years. He has edited/ authored 14 books and has published over 180 refereed papers. He has been awarded 47 externally funded grants worth over £11M from a variety of sources including EPSRC, ESRC, BBSRC, EU, Research Council of Norway, East Midlands Development Agency, HEFCE, Teaching Company Directorate, Joint Information Systems Committee of the HEFCs and commercial organisations. This funding portfolio includes being the principal investigator on a recently awarded EPSRC Science and Innovation award of £2M, an EPSRC grant of £2.6M to investigate the automation of the heuristic design process and an EPSRC platform grant worth £423K.

A Survey on Evolutionary Instance Selection and Generation

Joaquín Derrac, University of Granada, Spain

Salvador García, University of Jaén, Spain

Francisco Herrera, University of Granada, Spain

ABSTRACT

The use of Evolutionary Algorithms to perform data reduction tasks has become an effective approach to improve the performance of data mining algorithms. Many proposals in the literature have shown that Evolutionary Algorithms obtain excellent results in their application as Instance Selection and Instance Generation procedures. The purpose of this article is to present a survey on the application of Evolutionary Algorithms to Instance Selection and Generation process. It will cover approaches applied to the enhancement of the nearest neighbor rule, as well as other approaches focused on the improvement of the models extracted by some well-known data mining algorithms. Furthermore, some proposals developed to tackle two emerging problems in data mining, Scaling Up and Imbalance Data Sets, also are reviewed.

Keywords: Data Reduction, Evolutionary Algorithms, Imbalanced Data, Instance Generation, Instance Selection, Prototype Generation, Prototype Selection, Scaling Up, Training Set Selection

1. INTRODUCTION

Data reduction (Pyle, 1999) is one of the data preprocessing tasks which can be applied in a data mining process. The main objective in data reduction is to reduce the original data by selecting its most representative information. This way, it is possible to avoid excessive storage and time complexity, improving the results obtained by any data mining application, ranging from predictive processes (classification, regression)

to descriptive processes (clustering, extraction of association rules, subgroup discovery).

Data reduction processes can be performed in many ways, some of the more remarkable being:

- Selecting features (Liu & Motoda, 2007), reducing the number of columns in a data set. This process is known as Feature Selection.
- Making the feature values discrete (Liu et al., 2002), reducing the number of possible values of features. This process is known as attribute Discretization.

DOI: 10.4018/jamc.2010102604

- Generating new features (Guyon et al., 2006) which describe the data in a more suitable way. This process is known as Feature Extraction.
- Selecting instances (Liu & Motoda, 2001; Liu & Motoda, 2002), reducing the number of rows in a data set. This process is known as Instance Selection (IS)
- Generating new instances (Bezdek & Kuncheva, 2001; Lozano et al., 2006), which describes the initial data set by generating artificial examples. This process is known as Instance Generation (IG).

This article discusses a wide number of IS and IG proposals. They can be divided into two types of techniques depending on the goal followed by the reduction. If the set of selected or replaced instances will be used as the reference data to instance-based classification, then we refer to Prototype Selection (PS) and Prototype Generation (PG). On the other hand, if the set of instances obtained will be used as input or training set of any data mining algorithm for building a model, then we refer to Training Set Selection (TSS).

In spite of the differences between PS and PG (the first one finds suitable prototypes, while the second one generates them), both have been mainly employed to improve the same classifier, the Nearest Neighbor rule (Cover & Hart, 1967; see also Papadopoulos & Manolopoulos, 2004; Shakhnarovich et al., 2006). This predicts the class of a new prototype by computing a similarity measure (Cunningham, in press) between it and all prototypes from the training set. In the k-Nearest Neighbors classifier, k nearest prototypes vote to decide the class of the new instance to classify. This algorithm is the baseline of the instance based learning field (Aha et al., 1991).

On the other hand, TSS consists of the selection of reduced training sets to improve the efficiency and the results obtained by any data mining algorithm. It has been mainly applied to improve the performance of decision trees,

neural networks and subgroup discovery techniques. Although there exists a wide number of TSS approaches, no IG work on TSS has been reported yet, until our knowledge.

In recent years, the data mining community has identified some challenging problems in the area (Yang & Wu, 2006). Two of these are the *Scaling Up Problem* and the *Imbalance Data Sets Problem*. They are closely related to the data reduction field.

The *Scaling Up Problem* (Provost & Kolluri, 1999; Domingo et al., 2002) appears when an overwhelming amount of data must be processed, overcoming the capabilities of the traditional data mining algorithms. The *Imbalance Data Sets Problem* (Chawla et al., 2004; Batista et al., 2004) appears when the distribution of the class in the training data is not balanced, thus the number of instances of some classes is too low. This distribution can cause several problems in the classification of examples which belong to the minority classes.

Evolutionary Algorithms (Eiben & Smith, 2003) are general-purpose search algorithms that use principles inspired by natural genetic populations to evolve solutions to problems. The basic idea is to maintain a population of chromosomes which represent plausible solutions to the problem and evolve over time through a process of competition and controlled variation.

Evolutionary Algorithms have been successfully used in different data mining (Freitas, 2002; Ghosh & Jain, 2005; Abraham et al., 2006) and data reduction (Cano et al., 2003; Oh et al., 2004) problems. Given that the IS problem can be defined as a combinatorial problem, Evolutionary Algorithms have been used to solve it with promising results (Ho et al., 2002; García et al., 2008); these applications of Evolutionary Algorithms to tackle IS problems are usually called EIS (Evolutionary Instance Selection) methods. Furthermore, Evolutionary Algorithms have shown interesting behavior in their application to IG due to it can be defined as a parameter optimization problem (Fernández & Isasi, 2004; Nanni & Lumini, 2008).

The aim of this article is to present a review on the use of Evolutionary Algorithms for PS, PG and TSS algorithms, called EIS-PS, EPG and EIS-TSS, respectively, giving their description and main characteristics. Several evolutionary proposals developed to tackle the *Scaling Up Problem* and the *Imbalance Data Sets Problem* will be included in the review.

This article is organized as follows: Section 2 presents the definitions of the techniques and problems which will be reviewed in the rest of the article. Section 3 presents an overall analysis of several EIS-PS methods. Section 4 reviews the EPG contributions presented in recent years. Section 5 deals with EIS-TSS methods and their application to tackle different data mining problems. Section 6, concludes the survey.

2. PRELIMINARIES

This section provides some preliminary concepts and definitions about the techniques and problems shown in the rest of this article. Firstly, we describe the main characteristics of IS and IG. Secondly, we present the *Scaling Up Problem* and the *Imbalance Data Sets Problem* for classification, with reference to their relevance to data reduction. Finally, we briefly review some common features of the Evolutionary Algorithms approaches applied to tackle these problems.

Instance Selection and Generation

IS is one of the main data reduction techniques. In IS, the aim is to isolate the smallest set of instances which enable a data mining algorithm to predict the class of a query instance with the same quality as the initial data set (Liu & Motoda, 2001). By minimizing the data set size, it is possible to reduce the space complexity and decrease the computational cost of the data mining algorithms that will be applied later, improving their generalization capabilities through the elimination of noise.

More specifically, IS can be defined as follows: Let X_p be an instance where $X_p = (X_{p1},$

$X_{p2}, \dots, X_{pm}, X_{pc})$, with X_p belonging to a class c given by X_{pc} and a m -dimensional space in which X_{pi} is the value of the i -th feature of the p -th sample. Then, let us assume that there is a training set TR which consists of N instances X_p and a test set TS composed by t instances X_p . Let $S \subset TR$ be the subset of selected samples that resulted from the execution of a IS algorithm, then we classify a new pattern from TS by a data mining algorithm acting over S . The whole data set is noted as D and it is composed of the union of TR and TS .

IS methods can be classified in two categories: PS methods and TSS methods. PS methods (Liu & Motoda, 2002) are IS methods which expect to find training sets offering best classification accuracy and reduction rates by using instance based classifiers which consider a certain similarity or distance measure. Recently, PS methods have increased in popularity within the data reduction field. Various approaches to PS algorithms have been proposed in the literature (see (Wilson & Martinez, 2000; Grochowski & Jankowski, 2004) for review). Figure 1 shows the basic steps of the PS process.

TSS methods are defined in a similar way. They are known as the application of IS methods over the training set used to build any predictive model (e.g. decision trees, neural networks ...) Thus, TSS can be employed as a way to improve the behavior of predictive models, precision and interpretability (Riquelme et al., 2003). Figure 2 shows the basic steps of processing a decision tree (C4.5) on the TSS.

IG is another important technique in data reduction. It has been mainly applied to instance-based classifiers, thus we can focus on describing PG in depth. PG can be defined as the application of instance construction algorithms (Liu & Motoda, 2001) over a data set to improve the classification accuracy of a nearest neighbor classifier.

More specifically, PG can be defined as follows: Let X_p be an instance where $X_p = (X_{p1}, X_{p2}, \dots, X_{pm}, X_{pc})$, with X_p belonging to a class c given by X_{pc} and a m -dimensional space in which X_{pi} is the value of the i -th feature of the p -th sample. Then, let us assume that there is

Figure 1. Prototype selection process

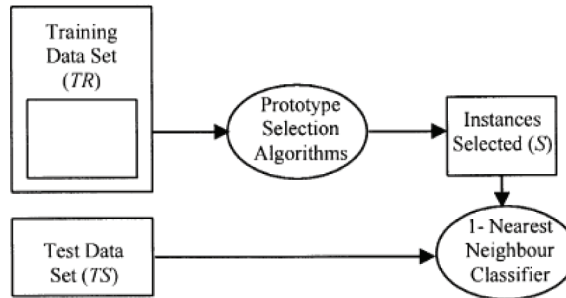
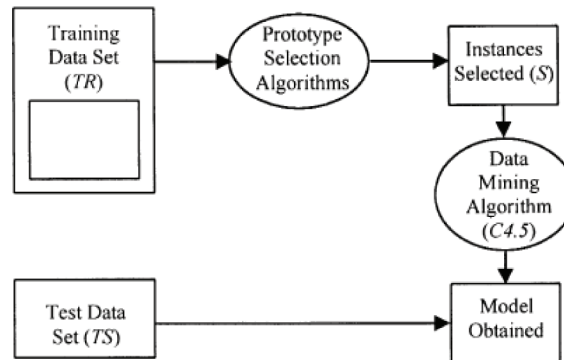


Figure 2. Training set selection process



a training set TR which consists of N instances X_p and a test set TS composed by t instances X_p . The purpose of PG is to obtain a prototype generate set, which consists of r ; $r < n$, prototypes, which are either selected or generated from the examples of X_p . The prototypes of the generated set are determined to represent efficiently the distributions of the classes and to discriminate well when used to classify the training objects. Their cardinality should be sufficiently small to reduce both the storage and evaluation time spent by a nearest neighbor classifier.

A wide number of PG methods have been designed in the specialized literature, ranging from traditional ones (Chang, 1974; Kohonen, 1990), to more modern approaches (Lozano et al., 2006). It is important to point out the fact that the research on the EPG field has started recently, being (Fernandez & Isasi, 2004) the first proposal in applying Evolutionary Algorithms to perform a PG task, in contrast to the research in evolutionary IS, where the first proposal was made in (Kuncheva, 1995).

The Scaling Up and the Imbalance Data Sets Problems

IS methods are considered a useful tool to perform data reduction tasks, obtaining interesting results. They have also been employed successfully to tackle two emergent challenges in data mining, the *Scaling Up Problem* and the *Imbalance Data Sets Problem* (Yang & Wu, 2006).

The *Scaling Up Problem* appears when the number of training samples increases beyond the capacity of the traditional data mining algorithms, harming their effectiveness and efficiency. Due to large size data sets, it produces excessive storage requirement, increases times complexity and affects to generalization accuracy. Usually, when the input data set size affects the execution of the algorithms, it is possible to face this situation with two different strategies:

- **Scaling up the algorithm:** Proposing faster and lower consumption algorithms that can face large size data sets. (Provost & Kolluri, 1999)
- **Scaling down the data set:** In this case, the attention is directed toward the data set. The idea consists of modifying the data set by means of reductions to make it adequate to the original algorithm (Liu & Motoda, 2002).

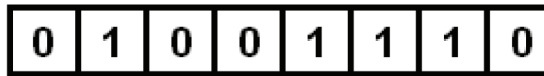
This problem has been recently addressed by many authors. An interesting example can be found in (Haro-García & García-Pedrajas, 2009), where a divide-and-conquer recursive approach to the problem is applied to very large problems, being able to match in accuracy and even improve on storage reduction the results of well-known standard IS algorithms with a very significant reduction in execution time. Another proposal, mainly adapted for use on evolutionary algorithms, is presented in (Cano et al., 2005), it will be described in the EIS-PS Section of this survey.

The *Imbalance Data Sets Problem* appears when the data contains many more examples of one class than the other and the less representative class represents the most interesting concept from the point of view of learning (Chawla et al., 2004). Imbalance in class distribution is pervasive in a variety of real-world applications, including but not limited to telecommunications (Tajbakhsh et al., 2009), web services, finance, ecology (Kubat et al., 1998), biology and medicine (Freitas et al., 2007).

Usually, in imbalanced classification problems, the instances are grouped into two types of classes: the majority or negative class, and the minority or positive class. The minority or positive class has more interest and it is also accompanied with a higher cost of misclassification. A standard classifier might ignore the importance of the minority class because its representation inside the data set is not strong enough. As a classic example, if the ratio of imbalance presented in the data is 1:100 (that is, there is one positive instance versus one hundred negatives), the error of ignoring this class is only 1%, so many classifiers could ignore it or not make any effort to learn an effective model for it.

Many approaches have been proposed to deal with the *Imbalance Data Sets Problem*. They can be divided into algorithmic approaches and data approaches. The first ones assume modifications in the operation of the algorithms, making them cost sensitive towards the minority class (Grzymala-Busse et al., 2005; Tajbakhsh et al., 2009). The data approaches modify the data distribution, conditioned on an evaluation function. Re-sampling of data could be done by means of under-sampling, by removing instances from the data (a process similar to IS) (Kubat & Matwin, 1997; Batista et al., 2004; Estabrooks et al., 2004), and over-sampling, by replicating or generating new minority examples (Chawla et al., 2002; Fernández et al., 2008).

Figure 3. A typical individual of an evolutionary IS algorithm



Basic Ideas on Evolutionary Algorithms for Instance Selection and Generation

A wide number of proposals presented on the use of Evolutionary Algorithms in IS and IG share some common characteristics regarding the two key concepts of any evolutionary algorithm: The representation of the population and the fitness function employed.

Representation: In most of the evolutionary IS algorithms, every member of the population encodes information about all the instances which are currently selected at each step of the search process.

This scheme is often used both in EIS-PS and EIS-TSS proposals. A binary representation is employed. Typically, every individual is defined as a binary string of length N , where each bit represents the current state of each instance of the training set (marked as '1' if the corresponding instance is currently selected, or '0' if not). Figure 3 shows a typical individual of an evolutionary IS algorithm.

On the other hand, in evolutionary IG algorithms every member of the population encodes information describing a new instance (or a new set of instances) to be generated. The concrete representation employed will vary depending on the codification of the problem data, although real coding is mostly preferred.

Fitness function: The majority of the evolutionary IS algorithms define a fitness function where two quality measures are employed: The accuracy of the results obtained by the subsequent data mining algorithm (e.g., a classifier), and the reduction rate achieved between the selected instances and the whole data set. Depending on the concrete method,

this reduction rate can be computed by counting the number of instances selected, or employing more sophisticated methods, e.g. valuating the reduction as a measure of the interpretability of the tree obtained, when the evolutionary IS method is applied to improve the results of a decision tree algorithm.

In contrast, in evolutionary IG algorithms the only quality measure employed to define the fitness function is the accuracy obtained by employing the actual set of instances generated (typically, the accuracy obtained with a k -Nearest Neighbors classifier). The reduction rate is not usually employed because the number of instances generated is always small (usually it is fixed to a concrete value), thus every solution is expected to achieve a high reduction rate, and the search process can be focused only on the goal of increasing the accuracy of the classification process.

3. EVOLUTIONARY PROTOTYPE SELECTION

In this section, we will present the main contributions of EIS-PS appeared in the literature in recent years. Firstly, we give a snapshot on the *state of the art* in EIS-PS. Secondly, we describe in depth the characteristics of the most representative EIS-PS methods appeared. Thirdly, we show some EIS-PS proposals dealing with the *Scaling Up* and the *Imbalance Data Sets* problems. Finally, we conclude this section presenting some EIS-PS mixed approaches.

A Snapshot on Evolutionary Prototype Selection

The necessity of the Evolutionary Algorithms in PS is discussed in (Cano et al., 2003) where the authors differentiate between the selection based in heuristics (which appears in classic non-evolutionary PS algorithms, like for example CNN, IB3 or DROP described in (Wilson & Martinez, 2000)) and the selection developed by EIS-PS algorithms. EIS-PS presents a strategy that combines inner and boundary points. It does not tend to select instances depending on their a priori position in the search space (inner class or limit ones). EIS-PS selects the instances that increase the accuracy rates independently of their a priori position.

We will review the main contributions that have included or proposed an EIS-PS model in recent years. The first appearance of the application of an evolutionary algorithm to the PS problem can be found in (Kuncheva, 1995). Kuncheva applied a genetic algorithm to select a reference set for the k- Nearest Neighbors rule. Her genetic algorithm maps the training set onto a chromosome structure composed by genes, each one with two possible states (binary representation). The computed fitness function measures the error rate by application of the k-Nearest Neighbors rule. This genetic algorithm was improved in (Kuncheva & Bezdek, 1998; Ishibuchi & Nakashima, 1999).

At this point, all EIS-PS algorithms considered above adapt a classical genetic algorithm model to the PS problem. Later, a development of EIS-PS algorithms more conditioned to the problem is made. The first example of this can be found in (Sierra et al., 2001). In this article, an Estimation of Distribution Algorithm is used. Another example can be found in (Ho et al., 2002), where a genetic algorithm design for obtaining an optimal nearest neighbor classifier based on orthogonal arrays is proposed.

The technical term EIS-PS has been adopted by Cano et al. (2003), in which they analyze the behavior of different evolutionary algorithms, generational genetic algorithms (GGAs), steady-state genetic algorithms (SS-

GAs), the CHC model (Eshelman, 1990) and Population Based Incremental Learning (PBIL) (Baluja, 1994) (which can be considered one of the basic Estimation of Distribution Algorithms). The fitness function used in these models combines two values: classification rate *clasRat* by using a 1-NN classifier and percentage reduction of prototypes of *S* with regards to *TR percRed*:

$$Fitness(S) = \alpha \cdot clasRat + (1 - \alpha) \cdot percRed$$

Where α is a weighting factor usually set to 0.5, and *perc_red* is defined as:

$$percRed = 100 \cdot (|TR| - |S| / |TR|)$$

One of the newest approaches to EIS-PS employs memetic algorithms (Ong et al., 2007). These are heuristic searches in optimization problems that combine a population-based algorithm with a local search. The memetic algorithm employed by (García et al., 2008) incorporates an *ad hoc* local search specifically designed for optimizing the search in prototype selection problem with the aim of tackling the scaling up problem. Another recent proposal (Gil-Pita & Yao, 2008), is focused in the enhancing of the fitness function and the mutation and crossover operators when applied to PS problems.

Later research has gone further, focusing its interest on other topics apart from improving the design of EIS-PS algorithms. Several efforts have been focused on developing methods which will be able to tackle new challenges in data mining. A representative example can be found in (Cano et al., 2005), where an evolutionary method to tackle the *Scaling Up Problem* on the PS process is proposed. Another challenging problem addressed recently by using EIS-PS algorithms is the *Imbalanced Data Sets Problem* (García & Herrera, 2009).

A brief review will be done regarding mixed evolutionary approaches to IS with another data preprocessing technique, e.g. Feature Selection

and Feature Weighting. Two different proposals will be reviewed, (Ros et al. 2008) being the first one. In this article, a hybrid genetic algorithm is applied to perform PS and Feature Selection simultaneously, trying to achieve three objectives simultaneously: Increasing the accuracy of the subsequent classification process, minimizing the dimensionality of the data (the number of features selected), and maximizing the number of instances selected.

The second proposal is an approach of data preprocessing with genetic algorithms in Case-Based Reasoning, which is described by the authors as an instance-based learning procedure (they also employ the k-Nearest Neighbors classifier). A basic genetic algorithm is conducted over a population of chromosomes which performs PS and Feature Weighting (which can be seen as a generalization of Feature Selection, where the weights are real valued between 0 and 1) simultaneously. This approach has been applied to two different scenarios: Customer classification and bankruptcy prediction modeling (Ahn et al., 2006; Ahn et al. 2009).

Overview on the EIS-PS Algorithms

In this subsection, we will describe in depth the characteristics of the most representative EIS-PS methods.

Generational Genetic Algorithm

Its basic idea is to maintain a population of chromosomes, which represent plausible solutions to the particular problem that evolves over successive iterations (generations) through a process of competition and controlled variation. Each chromosome in the population has an associated fitness to determine which chromosomes are to be used to form new ones in the competition process. The new chromosomes are created using genetic operators such as crossover and mutation.

In GGA, the selection mechanism produces a new population $P(t)$ with copies of chromosomes in the old population $P(t-1)$. The number of copies received for each chromosome depends on its fitness; chromosomes with higher fitness have a greater chance of contributing copies to $P(t)$. Then, the crossover and mutation operators are applied to $P(t)$.

Algorithm 1 (See Figure 4) shows a basic pseudocode of GGA:

The GGA was the first scheme developed to perform EIS-PS processes. Although its results have been overcome by most of the subsequent proposals, it is still an important milestone in the field of PS.

Figure 4. Algorithm 1: Pseudocode of the GGA

Algorithm 1: GGA basic structure.

Input: A population.

Output: An optimized population.

1: Initialize population;

2: While Termination Criterion not satisfied **do**

3: Evaluation of individual fitness;

4: Formation of a gene pool (intermediate population) through selection mechanism;

5: Recombination through crossover;

6: Mutation operator;

7: Replacement of the population;

8: end

Steady-State Genetic Algorithm

SSGA was firstly employed as an EIS-PS method in (Cano et al., 2003) In the SSGA usually one or two offspring are produced in each generation. Parents are selected to produce offspring and then a decision is made as to which individuals in the population will be selected for deletion in order to make room for the new offspring.

Algorithm 2 (See Figure 5) shows a basic pseudocode of SSGA:

In the construction of the SSGA, it is possible to select the replacement strategy (e.g., replacement of the worst, the oldest, or a randomly chosen individual) and the replacement condition (e.g., replacement if the new individual is better or unconditional replacement). A widely used combination is to replace the worst individual only if the new individual is better. Moreover, in (Goldberg & Deb, 1991), it is suggested that the deletion of the worst individuals can induce a high selective pressure, even when the parents are selected randomly.

This high selective pressure can help SSGA to improve the results of GGA in the performance of PS process. However, SSGA has been beaten also by most of the new proposals presented in recent years.

Population-Based Incremental Learning

PBIL (Baluja, 1994) is a specific Estimation of Distributions algorithm designed for binary search spaces. It attempts to explicitly maintain statistics about the search space to decide where to sample next.

The objective of the algorithm is to create a real valued probability vector V_p , which, when sampled, reveals high quality solution vectors with high probability. Initially, the values of V_p are set at 0.5. Sampling from this vector yields random solution vectors because the probability of generating a 1 or 0 for each gene is equal. As the search progresses, the values of V_p gradually shift to represent better solution vectors through the search process.

Algorithm 3 (See Figure 6) shows a basic pseudocode of PBIL algorithm.

The two basic search moves are performed in steps 6 and 8. In step 6, V_p is pushed toward S_{Best} . LR is the learning rate, which specifies how close the movement to S_{Best} is. In step 8, V_p is pushed far away from S_{Worse} . LR_{neg} is the negative learning rate, which specifies how far away the steps are from the worst solution.

PBIL can be seen as one of the most representative evolutionary proposals for performing PS process, because it is almost the only example of EIS-PS method which its search process is

Figure 5. Algorithm 2: Pseudocode of the SSGA

Algorithm 2: SSGA basic structure.

Input: A population.

Output: An optimized population.

1: Initialize population;

2: While Termination Criterion not satisfied do

3: Select two parents from the population;

4: Create one/two offspring using crossover and mutation;

5: Evaluate the offspring with the Fitness function;

6: Select one/two individuals in the population, which may be replaced by the offspring;

7: Decide if this/these individuals will be replaced;

8: end

Figure 6. Algorithm 3: Pseudocode of the PBIL algorithm

Algorithm 3: PBIL algorithm basic structure.	
Input: V_p .	
Output: Optimized V_p .	
1: Initialize V_p ;	
2: While Termination Criterion not satisfied do	
3:	Generate a fixed number of samples based upon the probabilities specified in V_p ;
4:	Select the best sample, S_{Best} , and the worst sample, S_{Worst} ;
5:	Foreach i in V_p do
6:	$V_p[i] = V_p[i] * (1-LR) + S_{Best}[i] * LR$;
7:	If $S_{Best}[i] \neq S_{Worst}[i]$ then
8:	$V_p[i] = V_p[i] * (1-LR_{neg}) + S_{Best}[i] * LR_{neg}$;
9:	end
10:	end
11:	end

not based in a genetic algorithm. In general, it obtained good results when compared with genetic-based methods in the study carried in (Cano et al., 2003).

The Evolutionary Model CHC

CHC algorithm (Eshelman, 1990) is a binary-coded genetic algorithm which involves the combination of a selection strategy with a very high selective pressure, and several components inducing a strong diversity. CHC is a robust evolutionary algorithm, which should often offer promising results in several search problems.

The four main components of the algorithm are:

- **An elitist selection:** To compose a new generation, the best individuals among parents and offspring are selected.
- **A highly disruptive crossover:** HUX, which crosses over exactly half of the non-matching alleles.
- **An incest prevention mechanism,** which only allows to cross over those pairs of individuals which has a Hamming distance higher than a difference threshold. This threshold is decreased, as

time goes by, to help the population to converge.

- **A restart process,** which is applied when the population has converged (when the threshold has dropped to zero). It generates a new population by randomly flipping a percentage (usually a 35%) of the bits of the old population individuals.

Algorithm 4 (See Figure 7) shows a basic pseudocode of CHC algorithm.

In the study carried out by (Cano et al., 2003), the CHC algorithm was selected as the best EIS-PS strategy, being able to outperform all the remaining methods of the study (the evolutionary and the non-evolutionary ones).

An interesting conclusion derived from that study was that the key feature of the CHC algorithm is its ability to select the most representative instances independently of their position in the search space, satisfying both the objectives of high accuracy and reduction rates. Due to this fact, CHC has been widely used as a baseline method to perform many evolutionary IS tasks. Some examples are shown in the next sections of this survey.

Figure 7. Algorithm 4: Pseudocode of the CHC algorithm

Algorithm 4: CHC algorithm basic structure.

Input: A population.

Output: An optimized population.

1: Initialize population;

2: While Termination Criterion not satisfied do

3: Select candidates from the population;

4: Generate offspring by crossing parents;

5: Evaluate the offspring with the fitness function;

6: Select the individuals of the new population

7: If population not changed then

8: Decrease threshold;

9: end

10: If threshold<0 then

11: Restart population and reinitialize threshold;

12: end

13: end

Intelligent Genetic Algorithm

Ho et al. (2002) propose the Intelligent Genetic Algorithm (IGA) based on Orthogonal experimental design used for PS and Feature Selection. Besides its initial definition, it can also be applied as a PS method only, without changing its initial objectives of increasing accuracy and reduction rates on the training data.

IGA is a GGA that incorporates an Intelligent Crossover operator. It builds an orthogonal array from two parents of chromosomes and searches within the array for the two best individuals according to the fitness function. It takes about $2^{\log_2 (\gamma-1)}$ fitness evaluations to perform an Intelligent Crossover operation, where γ is the number of bits that differ between both parents. Note that the application of Intelligent Crossover operator to large-size chromosomes (resulting chromosomes from large size data sets) could consume a high number of evaluations.

Algorithm 5 (See Figure 8) shows a basic pseudocode of IGA.

As their authors concluded, the employment of the Intelligent Crossover operator allows IGA to be superior to conventional genetic algorithms when applied to problems where the solution space is large and complex,

e.g. when it is composed of high dimensional overlapping patterns. Thus, it is a good EIS-PS method to apply when facing medium and large sized data sets.

Steady-State Memetic Algorithm

The steady-state memetic algorithm (SSMA) was proposed in (García et al., 2008) to cover a drawback of the conventional EIS-PS methods that had appeared before: their lack of convergence when facing large problems.

SSMA makes use of a local search or meme specifically developed for this prototype selection problem. This interweaving of the global and local search phases allows the two to influence each other; i.e. SSGA chooses good starting points, and local search provides an accurate representation of that region of the domain. This local search scheme assigns a probability value to each chromosome generated by crossover and mutation, C_{new} :

$$P_{LS} = \begin{cases} 1 & \text{if } Fitness(C_{new}) \text{ is better than } Fitness(C_{worst}) \\ 0.625 & \text{otherwise} \end{cases}$$

Figure 8. Algorithm 5. Pseudocode of the IGA

Algorithm 5: IGA basic structure.	
Input: A population.	
Output: An optimized population.	
1: Initialize population;	
2: While <i>Termination Criterion not satisfied</i> do	
3:	Evaluate all individuals using the fitness function;
4:	Use rank selection to select individuals to make a new population;
5:	Randomly cross some individuals by employing IC crossover;
6:	Select one/two individuals in the population, which may be replaced by the offspring;
7:	Apply the conventional bit inverse mutation operator to the population;
8:	end

Figure 9. Algorithm 6: Pseudocode of the SSMA

Algorithm 6: SSMA basic structure.	
Input: A population.	
Output: An optimized population.	
1: Initialize population;	
2: While <i>Termination Criterion not satisfied</i> do	
3:	Use binary tournament to select two parents;
4:	Apply crossover operator to create offspring (Off_1 , Off_2);
5:	Evaluate Off_1 and Off_2 ;
6:	Foreach Off_i do
7:	Invoke <i>Adaptive-PLS-mechanism</i> to obtain PLS_i ;
8:	If $u(0,1) < PLS_i$ then
9:	Perform meme optimization for Off_i ;
10:	end
11:	end
12:	Employ standard replacement for Off_1 and Off_2 ;
13:	end

Algorithm 6 (See Figure 9) shows a basic pseudocode of the SSMA.

Where $u(0,1)$ is a value in a uniform distribution $u[0,1]$ and the standard replacement means that a the worst individual is replaced only if the new individual is better.

The *Adaptive-PLS-mechanism* is an adaptive fitness-based method. A description of the *Adaptive-PLS-mechanism* and the meme specifically developed for the prototype selection task can be found in (García et al., 2008).

The meme optimization mechanism is a local search specifically designed for the PS problem. It tries to improve the initial chromosome by generating its neighbors by unselecting one of its current selected prototypes. These neighbors are evaluated by a special fitness function which is able to consume only partial evaluations, saving computational resources for the whole evolutionary process.

The objective of the meme optimization mechanism is adjusted dynamically in the

execution of the SSMA. Every time a certain number of evaluations have been spent, the accuracy and reduction rates achieved by the best chromosome of the population are registered. If the classification accuracy has not increased, then the meme optimization starts an *improving accuracy* stage, where only better results in accuracy are accepted through the local search. On the other hand, if the reduction rate has not increased, then the meme optimization starts an *Avoiding premature convergence* stage, where the local search accepts worse solutions in order to improve the diversity of the population.

As their authors concluded, the SSMA presents a good reduction rate and computational time. In fact, it is able to outperform the classical PS algorithms, when the accuracy and reduction rates are considered. When compared to other EIS-PS methods, SSMA is able to outperform or equal them, being particularly useful as the size of the databases increases.

Genetic Algorithm Based on Mean Square Error, Clustered Crossover and Fast Smart Mutation

A new genetic algorithm (concretely a GGA model) was proposed in (Gil-Pita & Yao, 2008) as an EIS-PS model. This algorithm (no name was provided by their authors) included the definition of a novel mean square error based fitness function, a novel clustered crossover technique, and a fast smart mutation scheme.

The fitness function employed was based on a mean square error measure:

$$F = \frac{1}{NC} \sum_{n=1}^N \sum_{i=1}^C \left(\frac{K_n[i]}{K} - d_n[i] \right)^2$$

Where N is the number of training patterns, C is the number of classes, K is the number of nearest neighbors employed, K_n is the number of K nearest neighbors belonging to class i , and d_n is 1 when the desired output for the instance n is the class I , and 0 when not. As their authors stated, the error surface defined by this function is smoother than those obtained using counting

estimator based functions, making easier the obtaining of its local minimum.

The clustering crossover operator firstly performs a k-means clustering process over the chromosomes, extracting the centroids of all the clusters found (the number of clusters is established randomly). Then, the centroids are employed with a classical random point cross operator to generate the individuals of the new generation.

The fast smart mutation procedure computes the effect of the change of one bit of the chromosome over its fitness value, testing all the possibilities. The change which produces the better fitness value is accepted as the result of the mutation operator. It is applied to every individual of the population. At the end of its application, the fitness value of the individuals is actualized, starting then a new generation of the evolutionary process.

Algorithm 7 (See Figure 10) shows a basic pseudocode of the algorithm.

The results obtained by the authors in the experimental study which was carried out suggested that the joint use of the three proposed methods could be quite interesting in the case of not very large training sets.

EIS-PS Proposals for the Scaling Up and Imbalanced Data Problems

In this subsection, we will analyze some EIS-PS proposals dealing with the *Scaling Up* and the *Imbalance Data Sets* problems.

Stratification of EIS-PS to Tackle the Scaling Up Problem

Cano et al. (2005) proposed a method to tackle the *Scaling Up Problem* in EIS-PS. The method presented consists of a stratified strategy which divides the initial data set into disjoint strata with equal class distribution. The number of strata chosen will determine their size, depending on the size of the data set. Using the proper number of strata the stratified method is able to significantly reduce the training set, avoiding the drawbacks of the *Scaling Up Problem*.

Figure 10. Algorithm 7: Pseudocode of the genetic algorithm based on mean square error, clustering crossover and fast smart mutation

Algorithm 7: Genetic algorithm based on mean square error, clustering crossover and fast smart mutation

Input: A population.

Output: An optimized population.

1: Initialize population;

2: While Termination Criterion not satisfied do

3: Evaluate population;

4: Apply clustering crossover (only one time for each 10 generations);

5: Apply Fast Smart Mutation operator;

6: end

Figure 11 shows the basic steps of the process.

Following the stratified strategy, the initial data set D is divided into t disjoint sets D_j , strata of equal size, $D_1, D_2, \dots, D_t$ maintaining class distribution within each subset. Then, PS algorithms will be applied to each D_j obtaining a selected subset DS_j . In this way, the subsets TR and TS will be obtained as follows:

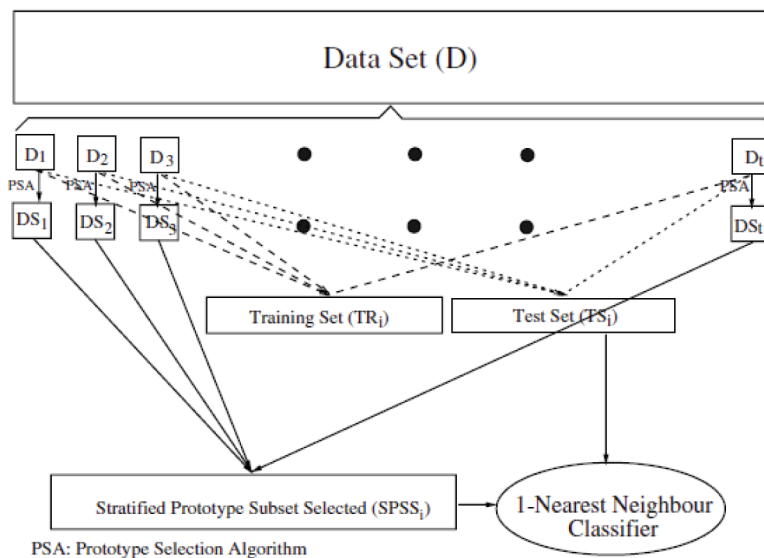
$$TR = \bigcup_{j \in J} D_j, J \subset \{1, 2, \dots, t\} \quad TS = D - TR$$

And the Stratified Prototype Subset Selected (SPSS) is defined as:

$$SPSS = \bigcup_{j \in J} DS_j, J \subset \{1, 2, \dots, t\}$$

The nearest neighbor classifier is then evaluated using as training data the $SPSS$ set, and the TS set as test data. Thus the classifica-

Figure 11. Structure of the stratification process in EIS-PS



tion process can be performed on higher size data sets, avoiding the usual drawbacks and still achieving acceptable results.

The concluding remarks of the study were that a proper choice in the number of strata makes it possible to decrease significantly execution time and resources consumption, maintaining the EIS-PS algorithm's behavior in accuracy and reduction rates. Also, the CHC was selected as the best EIS-PS algorithm when employed within the evolutionary stratified PS process.

EIS-PS Algorithms on Imbalanced Data Problems

In (García & Herrera, in press), a complete study of EUS (Evolutionary Under-Sampling) algorithms is carried out. A set of EUS methods is proposed, which take into consideration the nature of the problem and use different fitness functions to obtain a good trade-off between balance of distribution of classes and performance.

Eight different algorithms compose the set of methods proposed in the study. Furthermore every method shares the same basic structure, which is developed by using the CHC algorithms as an evolutionary model. There are three characteristics that differentiate them:

- The objective that they pursue.
 - Aiming for an optimal balancing of data without loss of effectiveness in classification accuracy. EUS models that follow this tendency will be called Evolutionary Balancing Under-Sampling.
 - Aiming for an optimal power of classification without taking into account the balancing of data, considering the latter as a sub-objective that may be an implicit process. EUS models that follow this tendency will be called Evolutionary Under-Sampling guided by Classification Measures.
- The way that they do the selection of instances.

- If the selection scheme proceeds over any kind of instance, then it is called Global Selection. That is, the chromosome contains the state of all instances belonging to the training data set and removals of minority class instances (those belonging to positive class) are allowed.
- If the selection scheme only proceeds over majority class instances then it is called Majority Selection. In this case, the chromosome saves the state of instances that belong to the negative class and a removal of a positive or minority class instance is not allowed.
- The accuracy measure used for its fitness function.
 - Geometric Mean methods, if the geometric mean is used as accuracy measure. Geometric mean was first employed in (Barandela et al., 2003} and is defined as:

$$GM = \sqrt{a^+ \cdot a^-}$$

Where a^+ denotes accuracy in examples belonging to the minority class and a^- denotes accuracy in examples belonging to the majority class.

- Area Under the ROC Curve methods, if the Area Under the Curve measure is used instead the geometric mean. Area Under the ROC Curve (Bradley, 1997) can measure the efficacy of various classifiers simultaneously, employing the True Positive and False Positive rates of a classification process.

The eight methods were tested with several imbalanced datasets, and the results obtained were contrasted by using non-parametric statistical procedures. The main conclusions drawn from the study were:

- PS algorithms must not be used for handling imbalanced problems. They are prone to gain global performance by eliminating examples belonging to the minority class considered as noisy examples.
- During the evolutionary under-sampling process, the employment of a majority selection mechanism helps to obtain more accurate subsets of instances than the use of global selection. However, the latter mechanism is necessary to achieve the highest reduction rates.
- Data sets with a low imbalance ratio should be faced with Evolutionary Under-Sampling guided by Classification Measures models, and should use in particular the model with a global mechanism of selection and evaluation through the geometric mean measure.
- Data sets with a high imbalance ratio should be faced with Evolutionary Balancing Under-Sampling models, and should use in particular the model with a majority selection mechanism and evaluation through the geometric mean measure.

Mixed EIS-PS Approaches

In recent years, some proposals have appeared performing not only the PS process with evolutionary algorithms, but also performing another data preparation process simultaneously. This subsection will review two of the most remarkable approaches.

Ros et al. (2008) proposed a hybrid genetic algorithm (HGA) which performs IS and Feature Selection simultaneously. Its objectives are to increase the accuracy of the k -Nearest Neighbors over the reference set, to minimize the number of features selected (reducing the reference data) and to maximize the number of instances selected (to retain the most information possible without harming the classification accuracy).

The HGA is divided into three phases:

- A genetic algorithm is applied in the first phase. It includes a sophisticated selection

scheme and some mechanisms to manage diversity and elitism (including an archive population and a dynamic analysis of the diversity of the population).

- By employing a histogram of the frequency with which each feature has been selected, a feature selection process is carried out, in order to simplify the problem.
- The genetic algorithm is applied again over the population. Also, some of the children generated by each generation are tuned by using local search procedures.

Despite the contradictory objectives in the number of instances and features selected, HGA is able to perform a dual IS and FS process with success, being a suitable evolutionary method to perform data reduction tasks.

A second mixed approach, GOCBR (Global Optimization of feature weighting and instance selection using genetic algorithms for Case Based Reasoning) was proposed in (Ahn et al., 2006; Ahn et al., 2009). This proposal performs a simultaneous IS and Feature Weighting process in the framework of a Case Based Reasoning system.

The search process of GOCBR consists of the application of a genetic algorithm with the common genetic operators (selection, crossover and mutation). Their individuals employ a binary representation, encoding the weights of the features by employing 14 bits to each one, and encoding the IS information in the second part of the chromosome by employing the usual binary scheme. Its fitness function measures only the accuracy obtained by employing the reference set defined by the chromosome to classify the train data in a k -Nearest Neighbors classifier.

GOCBR system has been applied successfully by the authors to various problems, such as customer classification or bankruptcy prediction modeling. Also, it is remarkable that is the only evolutionary method known of which performs a simultaneous IS and Feature Weighting process, until our knowledge.

4. EVOLUTIONARY PROTOTYPE GENERATION

In this section, we will present the main contributions of EPG appeared in the literature in recent years. In the first subsection, we give a snapshot on the *state of the art* in EPG. In the last subsection, we describe in depth the characteristics of the most representative EPG methods appeared.

A Snapshot on Evolutionary Prototype Generation

In recent years, the research efforts in the design of new PG techniques based on Evolutionary Algorithms have started to offer some interesting approaches. All of them still employ the NN rule as a reference classifier to measure the classification accuracy of the prototypes generated.

Usually, the prototypes are encoding as members of the population of the evolutionary algorithm employed to carry out the evolutionary process. A real codification scheme is used to represent them; each of its components has a concrete value for a concrete feature of the problem.

In this section, we will review the main contributions that have proposed an EPG model. The first contribution is (Fernandez & Isasi, 2004), where an evolutionary algorithm based on a two-dimensional grid is proposed, named Evolutionary Nearest Prototype Classifier. It defines a traditional evolutionary process to prepare the prototypes for its use on a 1-NN classifier, by employing a wide number of evolutionary operators.

The next two proposals are based on the Particle Swarm Optimization (PSO) Scheme (Kennedy et al., 2001). This technique is based on a set of potential solutions (particles) which evolves to find the global optimum of a real-valued function (fitness space) defined in a given space (search space). Particles represent the complete solution to the problem and move in the search space using both local information

(the particle memory) and neighbor information (the knowledge of neighbor particles).

In (Nanni & Lumini, 2008) a PSO based PG method is proposed (no name is provided for the algorithm). It can be seen as a Pittsburgh-based model, because all the components of the solution (in this case, the prototypes generated) are encoded in a single particle. The method performs a PSO search process where a reduced set of prototypes is generated to finally perform a 1-NN classification process.

In (Cervantes et al., 2007; Cervantes et al, in press), inspired by the results of (Cervantes et al., 2005), an Adaptive Michigan PSO model for PG is proposed. This model is described as a Michigan approach because every particle contains only a component of the solution, thus the complete solution is built by joining the selected particles of the swarm. This approach also improves the traditional PSO scheme, because the online generation and destruction of particles is allowed in the search process.

Finally, the last proposal which will be reviewed, (Garain, 2008), is based on the Clonar Selection Algorithm (Castro & Zuben, 2002), a representation of an Artificial Immune System model (Dasgupta, 1998). Clonar Selection Algorithms are inspired by the behavior of the immune system when performing an immune response to an antigenic stimulus. It advocates the idea that only those cells that recognize the antigens proliferate, thus being selected against those which do not. This idea is employed to develop a PG system which is able to generate suitable prototypes to perform a 1-NN classification process.

Overview on the EIG Algorithms

In this subsection, we will describe in depth the characteristics of the most representative EIG methods.

Evolutionary Nearest Prototype Classifier

In (Fernandez & Isasi, 2004), an EPG method is proposed. It employs as a basic structure

a two-dimensional matrix, where each row is associated with a prototype of the whole classifier, and each column is associated with a class to define regions where the prototypes are mapped.

On its initialization, the algorithm only defines one prototype. Then a whole evolutionary process starts: It carries out sequentially a set of evolutionary operations with the aim of generating a robust set of prototypes which will be able to correctly generalize the instances of the train set. The evolutionary operators defined are:

- **Mutation:** This operator is used to label each prototype with the most heavily populated class in each of its own regions.
- **Reproduction:** The reproduction operator function is to introduce new prototypes into the classifier, splitting the instances assigned to a prototype into two sets, where the second set is assigned to a new prototype.
- **Fight:** This operator allows prototypes to exchange their assigned instances. The fight can be performed in a cooperative or a competitive way, and it is ruled by the quality of the prototypes involved.
- **Movement:** The movement operator re-locates a prototype on the centroid of its assigned instances.

- **Die:** The current prototypes have a chance of being erased from the matrix, which is inversely proportional to its quality.

In the whole process, the quality of each prototype is defined by the number of prototypes which it has currently assigned and its classification accuracy. When the evolutionary process is finished, the generated set of prototypes is employed to classify the tests set, by means of the 1-NN classifier.

Algorithm 8 (See Figure 12) shows a basic pseudocode of Evolutionary Nearest Prototype Classifier algorithm.

The Evolutionary Nearest Prototype Classifier algorithm has shown good overall results, when compared against well-known classical methods in PG. Moreover, the results obtained when employing the prototypes generated with a 1-NN classifier were very competitive when compared with many classical instance-based classifiers as C4.5, Naïve Bayes or PART.

Particle Swarm Optimization for Prototype Generation

In (Nanni & Lumini, 2008) a PG method based on PSO is presented. This method defines the particles of the swarm as sets of a fixed number of prototypes, which are modified as the particle is moved in the search space.

Figure 12. Algorithm 8: Pseudocode of the Evolutionary Nearest Prototype Classifier algorithm

Algorithm 8: Evolutionary Nearest Prototype Classifier algorithm.

Input: Training set.

Output: A set of prototypes.

```

1: Initialization;
2: While Termination Criterion not satisfied do
3:   Collect information describing the prototypes;
4:   Mutation phase;
5:   Reproduction phase;
6:   Fight phase;
7:   Movement phase;
8:   Die phase;
9: end

```

The usual operators of PSO are employed by this proposal. The representation of each particle consists of a vector of length $S = K \cdot M$ given by the concatenation of K prototypes (dealing with M -dimensional data). The fitness function employed is defined as the classification error of the set of K prototypes over the training data.

Several runs (N) of the PSO process are carried out before finishing the PG stage (the authors recommend $N=5$). Each execution gives as result a reference set of K prototypes, being the result of the PG stage a collection of N reference sets. To classify a test instance, it is evaluated by each of the N reference sets, obtaining the final output as the result of a majority vote above all the reference sets. Thus, the proposed model can be seen as an ensemble of PSO-based classifiers.

Algorithm 9 (See Figure 13) shows a basic pseudocode of PSO-based PG algorithm.

The employment of the ensemble structure allows this proposal to obtain high accuracy rates. Furthermore, the authors suggested some ways to improve the model (like the employment of feature weighting methods). This proposal confirms that PSO is a very suitable model to perform PG processes.

An Adaptive Michigan Approach PSO for Nearest Prototype Classification

In (Cervantes et al., 2007; Cervantes et al., in press), an Adaptive Michigan Approach PSO is proposed. Its particles encode a prototype, each one being the generated train data represented as the whole particle swarm. This method does not have a fixed number of particles. On the contrary, some new operations are defined to allow the PSO search procedure to increase or decrease dynamically the number of particles.

The algorithm employs two different fitness functions: The global fitness function, defined by the standard classification accuracy on a 1-NN classifier, which is used to find the best swarm over the whole PSO procedure; and a local fitness function valued in each particle, defined by using the number of prototypes correctly classified and misclassified by itself. This secondary fitness function is used to evaluate the quality of each particle, in order to judge if it must be erased from the swarm, or if it can be employed as a parent of a new particle.

When the whole PSO process has finished, a cleaning process is carried out on the best swarm found. This swarm is the final output of the algorithm.

Algorithm 10 (See Figure 14) shows a basic pseudocode of the Adaptive Michigan PSO algorithm:

Figure 13. Algorithm 9: Pseudocode of the PSO-Based PG algorithm

Algorithm 9: PSO-based PG algorithm.

Input: Training set and test set.

Output: Classification output for the test set.

```

1: Initialization;
2: For j=1:N do
3:   PG(j)= PSO-PG(Train setj);
4: end
5: Foreach xi in test set do
6:   For j=1:N do
7:     PartialOutput(j)= Classify(xi, PG(j))
8:   end
9:   Output(xi)= VoteRule(PartialOutput);
10: end

```

This second PSO-based approach is focused in obtaining very high accuracy rates. The employment of a Michigan representation allows the definition of a local fitness function, which helps the method to find quickly suitable solutions in the search space.

Prototype Reduction Using an Artificial Immune Model

In (Garain, 2008), a PG based on a Clonar Selection algorithm is proposed. This model is composed of an immune memory which stores in its cells the best antigens found in the search process.

The Clonar Selection algorithm is initialized by representing the training instances as antigens, and choosing one antigen from each class to fill the immune memory. Then the search process starts. The first stage consists of a Hyper-mutation process. For each antigen on the training set, the most stimulating antigen in the immune memory is selected. The measure of stimulation is based on how close both antigens are (by means of Hamming or Euclidean Distance). The selected antigen of the memory is used as a parent for the Hyper-mutation

process, which generates its offspring. Then, a Resource Allocation procedure is called, which balances the total number of clones present in the system by giving half of the resources to the clones of the class of the current antigen. The other half is equally divided among clones of other classes.

While the classification accuracy is improved by the generation of clones, further Mutation processes (and Resource Allocation procedures) are carried out on the surviving clones. This Mutation produces a lower number of clones which depends on the stimulation value of each parent clone. When no improvement is achieved, the best clone found is inserted into the immune memory, performing a replacement with the worst antigen present. Then a new generation starts.

Finally, when the algorithm meets a global termination criterion, the antigens contained in the immune memory are employed as the training set to classify the test instances, by using the 1-NN classifier.

Algorithm 11 (See Figure 15) shows a basic pseudocode of PG-Clonar Selection algorithm:

Figure 14. Algorithm 10: Pseudocode of the AMPSO algorithm

Algorithm 10: Adaptive Michigan PSO algorithm.

Input: Training set.

Output: A set of prototypes (best swarm).

```

1: Initialize swarm. Dimension of particles equals number of attributes;
2: Insert N particles of each class into the training patterns;
3: While iterations < MAX and accuracyRate < 100% do
4:   Check for particle reproduction and deletion;
5:   Foreach particle do
6:     Calculate local fitness;
7:     Calculate its next position;
8:   End
9:   Move the particles;
10:  Assign classes to the training patterns using the nearest particle;
11:  Evaluate the swarm classification accuracy;
12: End
13: Delete, from the best swarm found so far, the particles that can be removed
    without a reduction in the classification accuracy.

```

The Clonar Selection Algorithm is a new method for PG based on a field of the Evolutionary Computation which has started to grown recently: The immune systems. Although it has high storage requirements to allocate the clones generated, it is a first example of a new technique which can obtain promising results with further research.

5. EVOLUTIONARY TRAINING SET SELECTION

In this section, we will present the main contributions of EIS-TSS appeared in the literature in recent years. In the first subsection, we give a snapshot on the *state of the art* in EIS-TSS. The next subsections will analyze the main approaches of EIS-TSS in decision trees, neural networks and subgroup discovery, respectively.

A Snapshot on Evolutionary Training Set Selection

The advances in EIS-TSS in recent years have been directed towards improving the results of some well-known data mining algorithms, being principally focused on the enhancement of the performance of decision trees, neural networks and subgroup discovery.

A wide range of these proposals have been inspired by the good results obtained by EIS-PS in the task of improving the performance of instance-based classifiers. Thus, some of the EIS-TSS methods which will be presented in this section will share some components with EIS-PS proposed before, adapting its principles to tackle the IS problems over other data mining algorithms.

Firstly, we will review some approaches of EIS-TSS applied to the construction of decision trees with the well-known C4.5 algorithm (Quinlan, 1993). A first application of IS to improve the construction of decision trees can be found in (Cano et al., 2003), where the results of the IS conducted by four evolutionary proposals are applied to extract reduced training sets in the construction of decision trees. Another proposal (Wu & Olaffson, 2006) also presented a genetic algorithm based IS process to improve the construction of decision trees, but they focused its effort on improving, not only the accuracy of the model and the reduction of the number of instances in the training set, but also in the interpretability and size of the trees obtained.

Later, Cano et al. (2007) presented two proposals of stratification to further improve the trees extracted by the C4.5 algorithm. The aim of both proposals was to improve the accuracy and interpretability of the trees extracted

Figure 15. Algorithm 11: Pseudocode of the PG-Clonar Selection algorithm

Algorithm 11: PG- Clonar Selection algorithm.

Input: Training set.

Output: A set of prototypes (immune memory).

1: Initialization;

2: While Termination Criterion not satisfied do

3: Proliferation I (Hyper-mutation);

4: Resource Allocation;

5: While resources left do

6: Proliferation II (Mutation);

7: Resource Allocation;

8: end

9: Insert best antigen into immune memory;

10: end

by means of stratification of the training data, trying to maintain a good trade-off between both quality measures.

As a last application of EIS-TSS in decision trees, we will review a proposal to improve the performance of C4.5 over imbalanced data sets (i.e. dealing with the *Imbalance Data Sets Problem* in the construction of decision trees) (García & Herrera, 2008). In this contribution it is shown how an evolutionary undersampling method is able to increase the accuracy of the decision tree in the classification of both majority and minority classes, employing the geometric mean accuracy measure.

Two approaches of EIS-TSS applied to neural networks will be analyzed. The first approach is (Ishibuchi et al., 2001), where a basic genetic algorithm is applied to perform both IS and Feature Selection processes to improve the results of standard three layered neural networks in classification.

A second approach to perform EIS-TSS on neural networks is presented in (Kim, 2006). This approach proposes the use of a standard genetic algorithm to perform a weight adjustment on the connections between the layers of a feed-forward neural network and an IS process over the instances which are employed to train the net.

Finally, the review of two proposals of EIS-TSS applied to subgroup discovery will be covered to close this section. The first proposal of this subsection (Cano, Herrera, Lozano & García, 2008) is an enhancement of the CN2-SD algorithm to increase its efficiency over large size datasets, by employing TSS techniques.

The second proposal (Cano et al., 2008) presents two stratification strategies to increase the presence of examples from minority classes in large size data sets with imbalanced data. The benefits shown in this study from the application of stratification includes the enhancement of Apriori-SD in its application to large size problems, and the improvement in the quality of the groups discovered over the minority classes of the problem analyzed.

EIS-TSS in Decision Trees

Cano et al. (2003) performed a complete study of the use of Evolutionary Algorithms to perform IS tasks. Although this work has been analyzed above, due to the number of EIS-PS which were presented in the study, it is important to note that a second experimental study was carried out employing the same IS algorithms as EIS-TSS methods to improve the trees built by C4.5.

The results obtained in the TSS part of the study were similar to the ones reached in the PS part: Evolutionary Algorithms based IS method were able to equal or outperform non-evolutionary methods, maintaining or increasing the accuracy of the trees obtained and increasing the reduction rates obtained by measuring the number of instances selected.

Wu & Olafsson (2006) performed a wide analysis of the application of IS to the induction of decision trees. They proposed a genetic algorithm to conduct the IS process, employing an integer codification in its chromosomes. Each individual is composed of a set of integer values, where each one represents one instance of the training set. Thus the selected instances of each individual are those whose integer identifier forms part of the chromosome.

The genetic algorithm search process employs a set of usual genetic operators: roulette wheel selection, crossover operator (which interchanges members of each set of instances defined by the parents) and mutation operator (which randomly replaces an instance by another one not selected).

The fitness function is a measure of the accuracy of the tree which can be induced by the chromosome, S , and its size. It is defined as follows:

$$\text{Fitness}(S) = -\log(e(\psi(S))) - a \cdot \log\left(\frac{\text{size}(\psi(S))}{K}\right), \quad a > 1$$

Where e is an estimator of the error rate of the decision tree $\psi(S)$, K is an upper bound on the size of the tree, and a is a weighting factor.

The final tree is obtained by merging the instances selected at least one time with a fixed number of the best chromosomes in the population. Then the tree can be employed to classify new test instances.

In addition, a study of some relevant parameters is presented along with the results of the algorithm. A discussion of two additional measures, the Average Leaf Ratio (a measure of the number of instances in each leaf node of the tree), and the Instance Entropy of the training data, show some interesting conclusions, the most remarkable being:

- Genetic algorithm based IS is able to reduce the tree sizes with minimal loss in prediction accuracy
- The best results are obtained when the Average Leaf Ratio of the final model is higher
- Genetic algorithm based IS works better on low entropy data sets. Higher values of entropy make the construction of more complex decision trees necessary
- IS can be employed as a replacement for the traditional tree pruning techniques because it is able to outperform them when they are compared in terms of accuracy and tree size

A third remarkable proposal of EIS-TSS can be found in (Cano et al., 2007), where the use of stratification is proposed to tackle the *Scaling Up* problem on the induction of decision trees. The aim of the study was to perform the extraction of classification rules from large size data by keeping a good tradeoff between the precision and the interpretability of the model generated. To accomplish its objective, the authors present a stratified strategy (similar as the employed in (Cano et al., 2005).

To conduct the EIS-TSS process, the CHC algorithm is used. Moreover, two different fitness functions are employed; they are based on the usual fitness function used for EIS-PS:

$$Fitness(TSS) = \alpha \cdot clasPer + (1 - \alpha) \cdot redPer$$

Where *clasPer* denotes the percentage of correctly classified objects from TR using only TSS to find the nearest neighbor or to extract a C4.5 model, depending on the concrete fitness function used. *redPer* denotes the reduction rate between TR and TSS.

- **Reduction-precision fitness function:** This fitness function uses the 1-Nearest Neighbor classifier for measuring the classification rate
- **Interpretability-precision fitness function:** This fitness function extracts a model with C4.5 to compute the classification performance of TSS

The two fitness definitions were tested with CHC and the stratification strategy. Several TSS methods were included in the experimental framework, to compare the performance of the proposed models against them. Finally, the results obtained were contrasted by using non-parametric statistical procedures.

The main conclusions reached in the study were as follows:

- The evolutionary stratified IS offers the best model size, maintaining an acceptable accuracy. It produces the smallest set of rules, with the minimal number of rules and the smallest number of antecedents per rule.
- The stratified CHC model with Interpretability-Precision fitness function allows us to obtain models with high test accuracy rates, similar to C4.5, but with the advantage that the size of the models are reduced considerably.
- The predictive model extraction by means of evolutionary stratified training set selection (with the CHC model and any of the fitness function presented) presents a good tradeoff between accuracy and interpretability. Thus, a very good scaling up behavior is observed, which allows us to obtain good results when the size of data set grows.

The last EIS-TSS proposal to improve the construction of decision trees which will be reviewed in this survey deals with the *Imbalance Data Sets* problem.

In (García & Herrera, in press) a new method is presented to deal with imbalanced data by performing the TSS process. The aim of the method is to improve the classification accuracy obtained by C4.5 when it is used on imbalanced data sets.

The proposed approach uses the same representation as the basic EIS-TSS methods. C4.5 is used to extract a model in order to compute the accuracy of the training set selected. The accuracy rates obtained by employing this model to classify the examples of the majority and minority classes are used to compute the geometric mean metric, which is used as fitness function.

Figure 16 shows the basic stages of the EUS process in EIS-TSS.

Although C4.5, in its standard definition, incorporates a pruning mechanism to avoid overfitting, the inclusion of the induction tree process within an evolutionary cycle can direct the resulting tree to an optimal model for training data, losing the generalization ability. To avoid this drawback, a simple and effective mechanism is incorporated. It consists of providing a higher weight for the classification

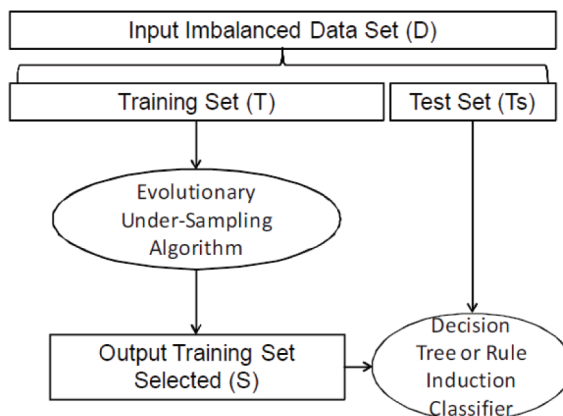
costs of the instances that are not included in TSS than to the instances that are. Therefore, the reduction ability of the selected subset is encouraged, allowing the proposed approach to avoid overfitting in the construction of the models.

To test the performance of the proposal, it was compared to a wide number of well-known re-sampling algorithms, including OSS (Kubat & Matwin, 1997), NCL (Laurikkala, 2001) and SMOTE (Chawla et al., 2002), among others. The results obtained were contrasted by using non-parametric statistical procedures, finally showing that the proposed approach is able to outperform, or, at least, to behave similarly to every method in the comparison with respect to the accuracy of the models obtained, it obtains very accurate trees with a low number of rules or leafs. Thus, the proposed approach is confirmed as a very accurate method, which is able to increase the interpretability of the models obtained.

EIS-TSS in Neural Networks

A first application of the use of EIS-TSS on neural networks can be found in (Ishibuchi et al., 2001). The aim of this proposal was to find an optimal subset of instances and features by employing a GGA.

Figure 16. The EUS process in EIS-TSS



The GGA designed to carry out the TSS task employed the usual representation of the solutions (binary coded, employing the first part of the chromosome to code the instances which are currently selected, and the second part to code the features), and a standard set of genetic operators: Random selection of parents, uniform crossover and bit flipping mutation (biased to decrease the number of instances selected). The fitness function defined by this proposal was:

$$Fitness = W_{Perf} \cdot Perf - W_f |F| - W_p \cdot |P|$$

Where *Perf* is a measure of the accuracy of a NN classifier when applied to the training data by using as a reference set the current subset selected by the chromosome, *F* and *P* are respectively the number of features and instances selected, and W_{Perf} , W_f and W_p are user-defined weights.

When the evolutionary process was finished, the final subset selected was employed to train a standard three layered neural network, being finally validated with a test data set to test its generalization capability.

Although the concrete EIS-TSS method employed has been outperformed by other EIS-TSS and EIS-PS proposals, this approach can still be considered an important milestone in the field because it was the first application of EIS-TSS to improve the performance of neural networks.

A more modern approach is proposed in (Kim, 2006). In this proposal, a GGA is also employed to optimize the performance of a three layered neural network, in the framework of an application for financial forecasting. The individuals of the genetic algorithm encode information about the instances selected and about the adjustment of the weights of the neural network, employing binary coding in both cases. Although an EIS-TSS process is performed, the genetic algorithm does not have as its objective the maintenance of a good reduction rate. Instead, the fitness function is defined as the classification accuracy of the

neural network defined by the chromosome, instead of the usual application of the neural network as a baseline classifier.

The experimental study carried out concluded that, in the context of the financial forecasting problem selected, the application of IS to improve the performance of neural networks outperformed the classical proposals of weights adjustment with genetic algorithms, highlighting the benefits of using EIS-TSS to improve the quality of the training process of neural networks.

EIS-TSS in Subgroup Discovery

In (Cano, Herrera, Lozano & García, 2008), a proposal to improve the performance of the CN2-SD algorithm for subgroup discovery in the evaluation of large size data sets is presented. Although CN2-SD is based on a divide and conquer strategy, it has to face the *Scaling Up* problem. To avoid it, the use of TSS algorithms is proposed for scaling down the data sets before the subgroup discovery task.

The study of the application of TSS algorithms, and the experiments that were carried out, was divided into two parts:

- In the first part, the effect of TSS on the subgroups discovered with CN2-SD in small data sets is studied. The objective is to analyze if the TSS process affects the descriptive qualitative measures of the subgroups (coverage, support, confidence, significance, unusualness, completeness, size and number of antecedents).

The basic IS methods applied to the TSS process were CNN (Hart, 1968), IB2, IB3 (Kibbler & Aha, 1987), DROP3 (Wilson & Martinez, 1997) ICF (Brighton & Mellish, 2002) and EIS-CHC. These methods were applied by using the stratification proposed in (Cano et al., 2005).

The results of this first part of the study were contrasted to a complete set of parametrical and non-parametrical statistical tests. Their application revealed that the use of TSS

did not negatively affect the quality indexes of the subgroup discovered. Also the measures on size and number of antecedents were improved, showing that TSS algorithms were able to discover smaller and more interpretable sets of subgroups.

- In the second part, a TSS process is combined with CN2-SD to test its behavior in large size data sets. As basic IS methods, IB2 and EIS-CHC were selected because they were the IS algorithms with the smallest subsets selected in their application to the large size data sets.

The main conclusion of this part was that the combination of the highest reduction rates of IB2 and EIS-CHC with CN2-SD makes it possible to perform a SD task on large size data sets. In particular, EIS-CHC is recommended because it shows very good results in most of the qualitative measures tested, when employed in combination with CN2-SD.

As a final conclusion of the study, the authors stated that, thanks to the application of TSS methods, CN2-SD can be executed on large data set sizes pre-processed, maintaining and improving the quality of the subgroups discovered.

A second application of EIS-TSS for subgroup discovery can be found in (Cano et al., 2008). There, a different application of the stratified strategy presented in (Cano et al., 2005) was proposed: The employment of two modified strategies of stratification to increase the presence of minority classes. The aim of the proposal was to allow a subgroup discovery algorithm, Apriori-SD (Kavsek & Lavrac, 2006), to avoid the *Scalability problem* and to face a large data set without harming its accuracy due to a poor treatment of imbalanced data.

The data set used on the experiment was the KDD Cup'99. Firstly, it is shown that the Apriori-SD could not handle the KDD Cup'99 problem because of its expensive computational cost in time. Then the TSS methods are applied to tackle the problem. ENN (Hart, 1968), IB2

(Kibbler & Aha, 1987), and EIS-CHC (Cano et al., 2005) were proposed as baseline IS methods to be used.

To preserve the number of instances of the minority classes, two different strategies of stratification were proposed:

- **Instance selection in all classes:** The instances of the majority classes are assigned randomly over the strata created. Then the whole minority classes are added to each strata. After the IS process is carried out, the subsets selected are reunited, removing duplicities.

The employment of this strategy showed a severe drawback: Its application to every TSS subset obtained after the reunion of the instances selected from the strata decreased significantly the number of instances present from the minority classes. Thus, the minority classes were not sufficiently represented for a proper subgroup discovery task, due to use of the stratified IS.

- **Instance selection in majority classes:** The selection process is applied without the minority ones, just to the majority classes. The instances which belong to the minority classes were added to the TSS subset after the reunion of the subsets selected, and then the subgroup discovery tasks were carried out.

In this case, the instances which appear in the TSS selected were the most representative of the majority classes and all the instances belonging to the minority ones. Thus the IS process was able to reduce the initial data set without affecting the presence of instances from the minority classes, making the subgroup discovery process possible in those classes.

Both strategies were tested in combination with the selected IS methods, using confidence and support as qualitative measures. The conclusions of the experiment were that the combination of the TSS algorithms with

stratification allows us to extract subgroups for most of the classes, including most of the minority ones, with high levels of confidence and support measures. Furthermore, the use of the *instance selection in majority classes* strategy of stratification was recommended to perform this task.

6. CONCLUSION

This article presents a review of PS, TSS and PG techniques performed by evolutionary algorithms. A wide number of algorithms and proposals of the *state-of-the-art* have been discussed, showing that the research in these fields have produced numerous advances in recent years to improve the quality of Instance Selection and Generation techniques in Data Mining.

Furthermore, this survey has considered the use of Evolutionary Algorithms to tackle two important issues in Data Mining: *The Scaling up Problem* and the *Imbalance Data Sets Problem*. These proposals have provided a way to improve the results obtained over large sized and imbalanced data in such fields as supervised classification and subgroup discovery, being a clear example of how Evolutionary Algorithms can be a useful tool in order to overcome these challenging problems.

ACKNOWLEDGMENT

This work was supported by TIN2008-06681-C06-01.

REFERENCES

- Abraham, A., Grosan, C., & Ramos, V. (Eds.). (2006). *Swarm intelligence in data mining*. Berlin, Germany: Springer-Verlag.
- Aha, D. W., Kibler, D., & Albert, M. K. (1991). Instance-based learning algorithms. *Machine Learning*, 6, 37–66.
- Ahn, H., & Kim, K. (2009). Bankruptcy prediction modeling with hybrid case-based reasoning and genetic algorithms approach. *Applied Soft Computing*, 9, 599–607. doi:10.1016/j.asoc.2008.08.002
- Ahn, H., Kim, K., & Han, I. (2006). Hybrid genetic algorithms and case-based reasoning systems for customer classification. *Expert Systems: International Journal of Knowledge Engineering and Neural Networks*, 23(3), 127–144. doi:10.1111/j.1468-0394.2006.00329.x
- Baluja, S. (1994). *Population-based incremental learning: A method for integrating genetic search based function optimization and competitive learning* (Tech. Rep. CMU-CS-94-163). Pittsburgh, PA: Carnegie Mellon University.
- Barandela, R., Sánchez, J. S., García, V., & Rangel, E. (2003). Strategies for learning in class imbalance problems. *Pattern Recognition*, 36(3), 849–851. doi:10.1016/S0031-3203(02)00257-1
- Batista, G. E. A. P. A., Prati, R. C., & Monard, M. C. (2004). A study of the behavior of several methods for balancing machine learning training data. *ACM Special Interest Group on Knowledge Discovery and Data Mining. SIGKDD Explorations*, 6(1), 20–29. doi:10.1145/1007730.1007735
- Bezdek, J. C., & Kuncheva, L. I. (2001). Nearest prototype classifier designs: An experimental study. *International Journal of Intelligent Systems*, 16(12), 1445–1473. doi:10.1002/int.1068
- Bradley, A. P. (1997). The use of the area under the ROC curve in the evaluation of machine learning algorithms. *Pattern Recognition*, 30(7), 1145–1159. doi:10.1016/S0031-3203(96)00142-2
- Brighton, H., & Mellish, C. (2002). Advances in instance selection for instance based learning algorithms. *Data Mining and Knowledge Discovery*, 6, 153–172. doi:10.1023/A:1014043630878
- Cano, J. R., García, S., & Herrera, F. (2008). Subgroup discovery in large size data sets preprocessed using stratified instance selection for increasing the presence of minority classes. *Pattern Recognition Letters*, 29, 2156–2164. doi:10.1016/j.patrec.2008.08.001
- Cano, J. R., Herrera, F., & Lozano, M. (2003). Using evolutionary algorithms as instance selection for data reduction in KDD: An experimental study. *IEEE Transactions on Evolutionary Computation*, 7, 561–575. doi:10.1109/TEVC.2003.819265

- Cano, J. R., Herrera, F., & Lozano, M. (2005). Stratification for scaling up evolutionary prototype selection. *Pattern Recognition Letters*, 26, 953–963. doi:10.1016/j.patrec.2004.09.043
- Cano, J. R., Herrera, F., & Lozano, M. (2007). Evolutionary stratified training set selection for extracting classification rules with trade-off precision-interpretability. *Data & Knowledge Engineering*, 60, 90–100. doi:10.1016/j.datak.2006.01.008
- Cano, J. R., Herrera, F., Lozano, M., & García, S. (2008). Making CN2-SD subgroup discovery algorithm scalable to large size data sets using instance selection. *Expert Systems with Applications*, 35, 1949–1965. doi:10.1016/j.eswa.2007.08.083
- Castro, L. N., & Zuben, F. V. J. (2002). Learning and optimization using the clonal selection principle. *IEEE Transactions on Evolutionary Computation*, 6, 239–251. doi:10.1109/TEVC.2002.1011539
- Cervantes, A., Galván, I., & Isasi, P. (2007). An adaptive michigan approach PSO for nearest prototype classification. In *Nature Inspired Problem-Solving Methods in Knowledge Engineering* (LNCS 4528, pp. 287–296).
- Cervantes, A., Galván, I., & Isasi, P. (in press). AMPSO: A new particle swarm method for nearest neighborhood classification. *IEEE Transactions on Systems, Man and Cybernetics, part B*.
- Cervantes, A., Isasi, P., & Galván, I. (2005). A comparison between the pittsburgh and michigan approaches for the binary pso algorithm. In *Proceedings of the 2005 IEEE Congress on Evolutionary Computation*, Munchen, Germany (pp. 290–297).
- Chang, C.-L. (1974). Finding prototypes for nearest neighbor classifiers. *IEEE Transactions on Computers*, 23(11), 1179–1184. doi:10.1109/T-C.1974.223827
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
- Chawla, N. V., Japkowicz, N., & Kotcz, A. (2004). Editorial: Special issue on learning from imbalanced data sets. *ACM Special Interest Group on Knowledge Discovery and Data Mining .SIGKDD Explorations*, 6(1), 1–6. doi:10.1145/1007730.1007733
- Cover, T. M., & Hart, P. E. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13, 21–27. doi:10.1109/TIT.1967.1053964
- Cunningham, P. (in press). A taxonomy of similarity mechanisms for case-based reasoning. *IEEE Transactions on Knowledge and Data Engineering*.
- Dasgupta, D. (Ed.). (1998). *Artificial immune systems and their applications*. Berlin, Germany: Springer Verlag.
- Domingo, C., Gavalda, R., & Watanabe, O. (2002). Adaptive sampling methods for scaling up knowledge discovery algorithms. *Data Mining and Knowledge Discovery*, 6(2), 131–152. doi:10.1023/A:1014091514039
- Eiben, A. E., & Smith, J. E. (2003). *Introduction to evolutionary computing*. Berlin, Germany: Springer Verlag.
- Eshelman, L. J. (1991). The CHC adaptive search algorithm: How to have safe search when engaging in nontraditional genetic recombination. In G. Rawlins (Ed.), *Foundations of genetic algorithms and classifier systems* (pp. 265–283). San Mateo, CA: Morgan Kaufmann.
- Estabrooks, A., Jo, T., & Japkowicz, N. (2004). A multiple resampling method for learning from imbalanced data sets. *Computational Intelligence*, 20(1), 18–36. doi:10.1111/j.0824-7935.2004.t01-1-00228.x
- Fernández, A., García, S., del Jesus, M. J., & Herrera, F. (2008). A study of the behaviour of linguistic fuzzy rule based classification systems in the framework of imbalanced data-sets. *Fuzzy Sets and Systems*, 159(18), 2378–2398. doi:10.1016/j.fss.2007.12.023
- Fernández, F., & Isasi, P. (2004). Evolutionary design of nearest prototype classifiers. *Journal of Heuristics*, 10, 431–454. doi:10.1023/B:HEUR.0000034715.70386.5b
- Freitas, A. A. (2002). *Data mining and knowledge discovery with evolutionary algorithms*. New York: Springer-Verlag.
- Freitas, A. A., da Costa Pereira, A., & Brazdil, P. (2007). Cost-sensitive decision trees applied to medical data. In *Data Warehousing and Knowledge Discovery* (LNCS 4654, pp. 303–312).
- Garain, U. (2008). Prototype reduction using an artificial immune model. *Pattern Analysis & Applications*, 11, 353–363. doi:10.1007/s10044-008-0106-1
- García, S., Cano, J. R., & Herrera, F. (2008). A memetic algorithm for evolutionary prototype selection: A scaling up approach. *Pattern Recognition*, 41(8), 2693–2709. doi:10.1016/j.patcog.2008.02.006

- García, S., & Herrera, F. (in press). Evolutionary under-sampling for classification with imbalanced data sets: Proposals and taxonomy. *Evolutionary Computation*.
- Ghosh, A., & Jain, L. C. (Eds.). (2005). *Evolutionary computation in data mining*. Berlin, Germany: SpringerVerlag.
- Ghosh, A., & Jain, L. C. (Eds.). (2005). *Evolutionary computation in data mining*. Berlin, Germany: Springer Verlag.
- Gil-Pita, R., & Yao, X. (2008). Evolving edited k-nearest neighbor classifiers. *International Journal of Neural Systems*, 18(6), 1–9. doi:10.1142/S0129065708001725
- Goldberg, D. E., & Deb, K. (1991). A comparative analysis of selection schemes used in genetic algorithms. In G. Rawlins (Ed.), *Foundations of genetic algorithms and classifier systems* (pp. 69-93). San Mateo, CA: Morgan Kaufmann.
- Gómez-Ballester, E., Micó, L., & Oncina, J. (2006). Some approaches to improve tree-based nearest neighbour search algorithms. *Pattern Recognition*, 39(2), 171–179. doi:10.1016/j.patcog.2005.06.007
- Grochowski, M., & Jankowski, N. (2004). Comparison of instance selection algorithms II. Results and comments. In *Proceedings of Artificial Intelligence and Soft Computing - ICAISC 2004* (LNCS 3070, pp. 580-585).
- Grzymala-Busse, J. W., Stefanowski, J., & Wilk, S. (2005). A comparison of two approaches to data mining from imbalanced data. *Journal of Intelligent Manufacturing*, 16, 565–573. doi:10.1007/s10845-005-4362-2
- Guyon, I., Gunn, S., Nikravesh, M., & Zadeh, L. (Eds.). (2006). *Feature extraction*. Heidelberg, Germany: Springer.
- Haro-García, A., & García-Pedrajas, N. (2009). A divide-and-conquer recursive approach for scaling up instance selection algorithms. *Data Mining and Knowledge Discovery*, 18, 392–418. doi:10.1007/s10618-008-0121-2
- Hart, P. E. (1968). The condensed nearest neighbour rule. *IEEE Transactions on Information Theory*, 18(3), 431–433.
- Ho, S.-Y., Liu, C. C., & Liu, S. (2002). Design of an optimal nearest neighbor classifier using an intelligent genetic algorithm. *Pattern Recognition Letters*, 23(13), 1495–1503. doi:10.1016/S0167-8655(02)00109-5
- Ishibuchi, H., & Nakashima, T. (1999). Evolution of reference sets in nearest neighbor classification. In *Selected papers from the Second Asia-Pacific Conference on Simulated Evolution and Learning on Simulated Evolution and Learning* (LNCS 1585, pp. 82-89).
- Ishibuchi, H., Nakashima, T., & Nii, M. (2001). Learning of neural networks with GA-based instance selection. In *Proceedings of the 20th North American Fuzzy Information Processing Society International Conference*, Vancouver, Canada (Vol. 4, pp. 2102-2107).
- Kavsek, B., & Luvrac, N. (2006). APRIORI-SD: Adapting association rule learning to subgroup discovery. *Applied Artificial Intelligence*, 20(7), 543–583. doi:10.1080/08839510600779688
- Kennedy, J., Eberhart, R. C., & Shi, Y. (2001). *Swarm intelligence*. San Francisco: Morgan Kaufmann Publishers.
- Kibbler, D., & Aha, D. W. (1987). Learning representative exemplars of concepts: An initial case of study. In *Proceedings of the 4th International Workshop on Machine Learning*, Irvine, CA (pp. 24-30).
- Kim, K. (2006). Artificial neural networks with evolutionary instance selection for financial forecasting. *Expert Systems with Applications*, 30, 519–526. doi:10.1016/j.eswa.2005.10.007
- Kohonen, T. (1990). The self-organizing map. *Proceedings of the IEEE*, 78, 1464–1480. doi:10.1109/5.58325
- Kubat, M., Holte, R. C., & Matwin, S. (1998). Machine learning for the detection of oil spills in satellite radar images. *Machine Learning*, 30(2-3), 195–215. doi:10.1023/A:1007452223027
- Kubat, M., & Matwin, S. (1997). Addressing the course of imbalanced training sets: One-sided selection. In *Proceedings of the 14th International Conference on Machine Learning*, Nashville, TN (pp. 179-186).
- Kuncheva, L. I. (1995). Editing for the k-nearest neighbors rule by a genetic algorithm. *Pattern Recognition Letters*, 16, 809–814. doi:10.1016/0167-8655(95)00047-K
- Kuncheva, L. I., & Bezdek, J. C. (1998). Nearest prototype classification: Clustering, genetic algorithms, or random search? *IEEE Transactions on Systems, Man, and Cybernetics*, 28(1), 160–164. doi:10.1109/5326.661099

- Laurikkala, J. (2001). Improving identification of difficult small classes by balancing class distribution. In *Proceedings of the 8th Conference on Artificial Intelligence in Medicine in Europe*, Cascais, Portugal (pp. 63-66).
- Lavrac, N., Kavsek, B., Flach, P., & Todorovski, L. (2004). Subgroup discovery with CN2-SD. *Journal of Machine Learning Research*, 5, 153-188.
- Liu, H., Hussain, F., Lim, C., & Dash, M. (2002). Discretization: An enabling technique. *Data Mining and Knowledge Discovery*, 6(4), 393-423. doi:10.1023/A:1016304305535
- Liu, H., & Motoda, H. (Eds.). (2001). *Instance selection and construction for data mining*. New York: Springer.
- Liu, H., & Motoda, H. (2002). On issues of instance selection. *Data Mining and Knowledge Discovery*, 6(2), 115-130. doi:10.1023/A:1014056429969
- Liu, H., & Motoda, H. (Eds.). (2007). *Computational methods of feature selection*. New York: Chapman & Hall.
- Lozano, M., Sotoca, J. M., Sánchez, J. S., Pla, F., Pekalska, E., & Duin, R. P. W. (2006). Experimental study on prototype optimisation algorithms for prototype-based classification in vector spaces. *Pattern Recognition*, 39(10), 1827-1838. doi:10.1016/j.patcog.2006.04.005
- Nanni, L., & Lumini, A. (2008). Particle swarm optimization for prototype reduction. *Neurocomputing*, 72, 1092-1097. doi:10.1016/j.neucom.2008.03.008
- Newman, D. J., Hettich, S., Blake, C. L., & Merz, C. J. (1998). *UCI repository of machine learning databases*. Irvine, CA: University of California, Irvine, Department of Information and Computer Sciences. Retrieved from <http://www.ics.uci.edu/~mllearn/MLRepository.html>
- Oh, I., Lee, J., & Moon, B. (2004). Hybrid genetic algorithms for feature selection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(11), 1424-1437. doi:10.1109/TPAMI.2004.105
- Ong, Y. S., Krasnogor, N., & Ishibuchi, H. (2007). Special issue on memetic algorithms. *IEEE Transactions on Systems, Man and Cybernetics . Part B*, 37(1), 2-5.
- Papadopoulos, A. N., & Manolopoulos, Y. (2004). *Nearest neighbor search: A database perspective*. Berlin, Germany: Springer-Verlag.
- Paredes, R., & Vidal, E. (2006). Learning prototypes and distances: A prototype reduction technique based on nearest neighbor error minimization. *Pattern Recognition*, 39(2), 180-188. doi:10.1016/j.patcog.2005.06.001
- Provost, F. J., & Kolluri, V. (1999). A survey of methods for scaling up inductive learning algorithms. *Data Mining and Knowledge Discovery*, 2, 131-169. doi:10.1023/A:1009876119989
- Pyle, D. (1999). *Data preparation for data mining*. San Francisco: Morgan Kaufmann.
- Quinlan, J. R. (1993). *C4.5: Programs for machine learning*. San Francisco: Morgan Kaufmann.
- Riquelme, J. C., Aguilar, J. S., & Toro, M. (2003). Finding representative patterns with ordered projections. *Pattern Recognition*, 36(4), 1009-1018. doi:10.1016/S0031-3203(02)00119-X
- Ros, F., Guillaume, S., Pintore, M., & Chretien, J. R. (2008). Hybrid genetic algorithm for dual selection. *Pattern Analysis & Applications*, 11, 179-198. doi:10.1007/s10044-007-0089-3
- Sanchez, J. S., Barandela, R., Marques, A. I., Alejo, R., & Badenas, J. (2003). Analysis of new techniques to obtain quality training sets. *Pattern Recognition Letters*, 24, 1015-1022. doi:10.1016/S0167-8655(02)00225-8
- Sebban, M., Nock, R., Chauchat, J. H., & Rakotomalala, R. (2000). Impact of learning set quality and size on decision tree performances. *International Journal of Computers . Systems and Signals*, 1(1), 85-105.
- Shakhnarovich, G., Darrel, T., & Indyk, P. (Eds.). (2006). *Nearest-neighbor methods in learning and vision: Theory and practice*. Cambridge, MA: MIT Press.
- Sierra, B., Lazkano, E., Inza, I., Merino, M., Larrañaga, P., & Quiroga, J. (2001). Prototype selection and feature subset selection by estimation of distribution algorithms. A case study in the survival of cirrhotic patients treated with TIPS. *Artificial Intelligence in Medicine (LNAI 2101)*, pp. 20-29.
- Tajbakhsh, A., Rahmati, M., & Mirzaei, A. (2009). Intrusion detection using fuzzy association rules. *Applied Soft Computing*, 9(2), 462-469. doi:10.1016/j.asoc.2008.06.001
- Wilson, D. R., & Martinez, T. R. (1997). Instance pruning techniques. In *Proceedings of the 14th International Conference on Machine Learning*, Nashville, TN (pp. 403-411).

Wilson, D. R., & Martinez, T. R. (2000). Reduction techniques for instance-based learning algorithms. *Machine Learning*, 38, 257–286. doi:10.1023/A:1007626913721

Wu, S., & Olafsson, S. (2006). *Optimal instance selection for improved decision tree induction*. Paper presented at the 2006 IIE Annual Conference and Exhibition, Orlando, FL.

Yang, Q., & Wu, X. (2006). 10 challenging problems in data mining research. *International Journal of Information Technology and Decision Making*, 5(4), 597–604. doi:10.1142/S0219622006002258

APPENDIX: ACRONYMS TABLE

In this appendix, a table with all the acronyms employed on the text is provided (Table 1). For each acronym, it is shown its meaning and the page where it was defined:

Table 1.

Acronym	Meaning	Page
EIS	Evolutionary Instance Selection	4
EIS-PS	Evolutionary Instance Selection – Prototype Selection	4
EIS-TSS	Evolutionary Instance Selection – Training Set Selection	4
EUS	Evolutionary Under Sampling	23
EPG	Evolutionary Prototype Generation	4
GGA	Generational Genetic Algorithm	10
GOCBR	Global Optimization of feature weighting and instance election using genetic algorithms for Case Based Reasoning	26
HGA	Hybrid Genetic Algorithm	26
IG	Instance Generation	3
IGA	Intelligent Genetic Algorithm	18
IS	Instance Selection	3
PBIL	Population Based Incremental Learning	10
PG	Prototype Generation	3
PS	Prototype Selection	3
PSO	Particle Swarm Optimization	28
SSGA	Steady State Genetic Algorithm	10
SSMA	Steady State Memetic Algorithm	18
TSS	Training Set Selection	3

Joaquín Derrac received the MSc degree in computer science from the University of Granada, Granada, Spain, in 2008. He is currently a PhD student in the Department of Computer Science and Artificial Intelligence, University of Granada, Granada, Spain. His research interests include data mining, lazy learning and evolutionary algorithms.

Salvador García received the MSc and PhD degrees in computer science from the University of Granada, Granada, Spain, in 2004 and 2008, respectively. He is currently an associate professor in the Department of Computer Science, University of Jaén, Jaén, Spain. His research interests include data mining, data reduction, data complexity, imbalanced learning, statistical inference and evolutionary algorithms.

Francisco Herrera received the MSc degree in mathematics in 1988 and the PhD degree in mathematics in 1991, both from the University of Granada, Spain. He is currently a professor in the Department of Computer Science and Artificial Intelligence at the University of Granada. He has published more than 150 papers in international journals. He is coauthor of the book Genetic Fuzzy Systems: Evolutionary Tuning and Learning of Fuzzy Knowledge Bases (World Scientific, 2001). As edited activities, he has co-edited five international books and co-edited twenty special issues in international journals on different soft computing topics. He acts as associated editor of the journals: IEEE Transactions on Fuzzy Systems, Mathware and Soft Computing, Advances in Fuzzy Systems, Advances in Computational Sciences and Technology, and International Journal of Applied Metaheuristic Computing. He currently serves as area editor of the Journal Soft Computing (area of genetic algorithms and genetic fuzzy systems), and he serves as member of the editorial board of the journals: Fuzzy Sets and Systems, Applied Intelligence, Knowledge and Information Systems, Information Fusion, Evolutionary Intelligence, International Journal of Hybrid Intelligent Systems, Memetic Computation, International Journal of Computational Intelligence Research, The Open Cybernetics and Systemics Journal, Recent Patents on Computer Science, Journal of Advanced Research in Fuzzy and Uncertain Systems, International Journal of Information Technology and Intelligent Computing, and Journal of Artificial Intelligence and Soft Computing Research. His current research interests include computing with words and decision making, data mining, data preparation, instance selection, fuzzy rule based systems, genetic fuzzy systems, knowledge extraction based on evolutionary algorithms, memetic algorithms and genetic algorithms.

Theorems Supporting r-flip Search for Pseudo-Boolean Optimization

Bahram Alidaee, University of Mississippi, USA

Gary Kochenberger, University of Colorado Denver, USA

Haibo Wang, Texas A&M International University, USA

ABSTRACT

Modern metaheuristic methodologies rely on well defined neighborhood structures and efficient means for evaluating potential moves within these structures. Move mechanisms range in complexity from simple 1-flip procedures where binary variables are “flipped” one at a time, to more expensive, but more powerful, r-flip approaches where “r” variables are simultaneously flipped. These multi-exchange neighborhood search strategies have proven to be effective approaches for solving a variety of combinatorial optimization problems. In this article, we present a series of theorems based on partial derivatives that can be readily adopted to form the essential part of r-flip heuristic search methods for Pseudo-Boolean optimization. To illustrate the use of these results, we present preliminary results obtained from four simple heuristics designed to solve a set of Max 3-SAT problems.

Keywords: *Combinatorial Optimization, Discrete Optimization, Integer Programming, Pseudo Boolean Optimization, Satisfiability*

INTRODUCTION

Pseudo Boolean formulations are widely known for their ability to represent a rich variety of important discrete problems. While special cases exist for which provably optimal solutions can be found in reasonable time, often the problems are NP-hard and heuristic methods must be employed to produce solutions in a reasonable amount of computer time. In their important early work (Hammer & Rudeanu,

1968), Hammer and Rudeanu discussed Pseudo-Boolean optimization and gave a dynamic programming procedure for solving certain problems. Furthermore, they gave a definition of a *first-order derivative* and indicated its use in solving discrete Pseudo-Boolean optimization problems. Specifically they gave the necessary and sufficient results for any local optimal solution of the problem when one element of x is changed at a time (*1-flip* move, or *1-move*, defined later in the article). More recently, excellent surveys of Pseudo-Boolean optimization are given by Boros and Hammer (2002)

DOI: 10.4018/jamc.2010102605

and Crama and Hammer (2007). These sources provide extensive discussions of key topics as well as comprehensive bibliographies.

In this article we extend this notion of first order derivatives by defining *higher-order derivatives* for discrete Pseudo-Boolean optimization. Moreover, in the context of changing r elements of x at a time (the so-called *r-flip* moves for $r=2,3$), we present closed-form formulas that allow 'efficient' implementation of such compound moves. Then, for the important special cases of quadratic and cubic optimization, we define a general *r-flip* move that allows efficient implementation of multi-exchange neighborhood search process for solving such problems. Finally, we illustrate the use of such moves by applying variants of simple search processes based on *r-flip* moves (with $r = 1$ and 2) to a test bed of max 3-sat problems. The article then concludes with summary and a look ahead to future research.

We note that the use of simple exchange procedures such as *r-opt* for binary optimization have been reported by several authors (Ahuja, Ergun, Orlin, & Punnen, 2002; Ahuja, Orlin, Pallottino, Scaparra, & Scutellà, 2004; Deineko & Woeginger, 2000; Frangioni, Necciari, & Scutellà, 2004; Glover, 1996; Li & Alidaee, 2002; Magazine, Polak, & Sharma, 2002; Vredeveld & Lenstra, 2003; Yaguiura & Ibaraki, 1999; Yaguiura & Ibaraki, 2001). However, in all such applications the method presented are highly problem specific. The *r-flip* rules we present in this article are quite general and can be applied to a variety of problem classes.

PSEUDO-BOOLEAN OPTIMIZATION

Let R be the set of reals, Z the set of integers, and $B = \{0,1\}$. For a positive integer n let $V = \{1,2,\dots,n\}$. Let $x = (x_1,\dots,x_n) \in B^n$ be a binary vector and $\bar{x}_i = 1 - x_i$ complement of x_i for $i \in V$. Define the set of literals to be $L = \{x_i, \bar{x}_1, \dots, x_n, \bar{x}_n\}$. Mappings $f : B^n \rightarrow R$ are called *Pseudo-Boolean functions*. Since

there is a one-to-one correspondence between subsets of V and the set of binary vectors B^n , these functions are *set functions*. All Pseudo-Boolean functions can be uniquely represented as multi-linear polynomials of the form given below (see (Boros & Hammer, 2002), for a comprehensive discussion) where q_s is a weight associated with the set $S \subseteq V$:

$$f(x_1,\dots,x_n) = \sum_{S \subseteq V} q_S \prod_{j \in S} x_j$$

The Pseudo-Boolean optimization problem can be stated as $\underset{x}{Max} f(x)$.

Definition 1: Hammer and Rudeanu (1968) defined the first derivative of f with respect to x_i as,

$$\Delta_i(x) = \frac{\partial f(x)}{\partial x_i} = f(x_1,\dots,x_{i-1},1,x_{i+1},\dots,x_n) - f(x_1,\dots,x_{i-1},0,x_{i+1},\dots,x_n), \quad \forall i \in V.$$

Definition 2: For a given $x = (x_1,\dots,x_{i-1},x_i,x_{i+1},\dots,x_n)$, define $x' = (x_1,\dots,x_{i-1},\bar{x}_i,x_{i+1},\dots,x_n)$ to be *1-flip move with respect to the i th position of x* .

Based on Definition 2 an *1-flip* local search process can be defined as follows:

Algorithm (1-flip Local Search):

Start with a solution x

Do

With some pre-specified order choose each $i \in V$,

change x_i to $\bar{x}_i$ if it improves $f(x)$

Undo

Stop if no *1-flip* move is possible.

Theorem 1. (Boros & Hammer, 2002): Given a Pseudo-Boolean function the first derivative is given by

$$\Delta_i(x) = \sum_{S \subseteq V \setminus \{i\}} q_{S \cup \{i\}} \prod_{j \in S} x_j$$

Furthermore, in any local optimal solution with respect to 1-flip search we have

$$\begin{aligned} \text{either } (x_j = 0 \text{ iff } \Delta_j(x) \geq 0) \\ \text{or } (x_j = 1 \text{ iff } \Delta_j(x) \leq 0), \quad \text{for } j \in V. \end{aligned}$$

Since values for each variable is 0 or 1 we have $(x_i)^2 = x_i$ for $i \in V$. Thus, we assume in f all $(x_i)^2$ are presented as x_i . In the following we give a definition that applies to both first as well as higher order derivatives. The definitions and the results associated with them permit an efficient implementation of r -flip search strategies.

Definition 3: For a given solution x and given $S \subseteq V$ where $|S| = r$, let x' be a solution obtained from x by complementing all elements in S , i.e., x' is the same as x except that for all $i \in S$ where x_i is changed to $\bar{x}_i$. Such a move is called an r -flip move with respect to elements of S . Now, consider f in continuous form and let

$$f^{[T]}(x) = \Delta_T(x) = \frac{\partial^{[T]} f(x)}{\partial x_1 \partial x_2 \dots \partial x_{|T|}}$$

be the $|T|^{th}$ partial derivative of f with respect to x_i and $i \in T$, $T \subseteq S$. Thus, if $T = \{i_1, \dots, i_k\}$ and $|T| = k$, then we have

$$f^k(x) = \Delta_{i_1 i_2 \dots i_k}(x) = \frac{\partial^k f(x)}{\partial x_1 \partial x_2 \dots \partial x_k}$$

Now, restrict domain of function $\Delta_T(x)$ to binary vectors. We call this the k^{th} partial derivative of the Pseudo-Boolean function f with respect to elements of T . Note that the value of $\Delta_T(x)$ is independent of the order of partial

derivative of elements in T . Also, note that the first derivative in this case is equal to the first derivative as shown in Theorem 1.

With respect to 1-flip local search, Hammer and Rudeanu (1968) prove the following:

Theorem 2. For a Pseudo-Boolean optimization problem, given $x = (x_1, \dots, x_{i-1}, x_i, x_{i+1}, \dots, x_n)$ and $x' = (x_1, \dots, x_{i-1}, \bar{x}_i, x_{i+1}, \dots, x_n)$ we have

$$f(x') - f(x) = (\bar{x}_i - x_i) \Delta_i(x),$$

Furthermore, in any local optimal solution with respect to 1-flip search we have

$$\begin{aligned} \text{either } (x_j = 0 \text{ iff } \Delta_j(x) \geq 0) \\ \text{or } (x_j = 1 \text{ iff } \Delta_j(x) \leq 0), \quad \text{for } j \in V. \end{aligned}$$

Proof: Given $x = (x_1, \dots, x_{i-1}, x_i, x_{i+1}, \dots, x_n)$. By definition 3 we have

$$\Delta_i(x) = \sum_{S \subseteq V \setminus \{i\}} q_{S \cup \{i\}} \prod_{j \in S} x_j = q_{\{i\}} + \sum_{\substack{S \subseteq V \setminus \{i\} \\ S \neq \emptyset}} q_{S \cup \{i\}} \prod_{j \in S} x_j$$

Let $x' = (x_1, \dots, x_{i-1}, \bar{x}_i, x_{i+1}, \dots, x_n)$. We then have

$$\begin{aligned} f(x) &= x_i \left[q_{\{i\}} + \sum_{\substack{S \subseteq V \setminus \{i\} \\ S \neq \emptyset}} q_{S \cup \{i\}} \prod_{j \in S} x_j \right] + \dots \\ f(x) &= x_i [\Delta_i(x)] + \dots \\ f(x') &= \bar{x}_i [\Delta_i(x)] + \dots \\ f(x') - f(x) &= (\bar{x}_i - x_i) \Delta_i(x) \end{aligned}$$

From this, the necessary and sufficient condition of local optimality follows immediately.

Similar to the definition of an 1-flip local search, using Definition 3 we define an r -flip local search process by:

Algorithm (r-flip Local Search):

Start with a solution x

Do

With some pre-specified mechanism choose an $S \subseteq V$ and $|S|=r$,

For all $i \in S$ change x_i to $\bar{x}_i$ if it improves $f(x)$

undo

Stop if no r -flip move is possible.

With respect to r -flip search, Theorem 2 can easily be extended to address the cases when $r = 2$ and $r = 3$ as shown below in Theorem 3 (involving second derivatives) and Theorem 4 (involving third derivatives), respectively.

Theorem 3: (2-flip local search, 2nd derivatives). For a Pseudo-Boolean optimization problem, given $x = (x_1, \dots, x_i, \dots, x_j, \dots, x_n)$ and $x' = (x_1, \dots, \bar{x}_i, \dots, \bar{x}_j, \dots, x_n)$ we have

$$f(x') - f(x) = (\bar{x}_i - x_i)\Delta_i(x) + (\bar{x}_j - x_j)\Delta_j(x) + (\bar{x}_i - x_i)(\bar{x}_j - x_j)\Delta_{ij}(x)$$

Proof: (see appendix A)

Extending to 3-flip local search and third derivatives we have:

Theorem 4: (3-flip local search, 3rd derivatives). For a Pseudo-Boolean optimization problem, given $x = (x_1, \dots, x_i, \dots, x_j, \dots, x_k, \dots, x_n)$ and $x' = (x_1, \dots, \bar{x}_i, \dots, \bar{x}_j, \dots, \bar{x}_k, \dots, x_n)$ we have

$$f(x') - f(x) = (\bar{x}_i - x_i)\Delta_i(x) + (\bar{x}_j - x_j)\Delta_j(x) + (\bar{x}_k - x_k)\Delta_k(x) + (\bar{x}_i - x_i)(\bar{x}_j - x_j)\Delta_{ij}(x) + (\bar{x}_i - x_i)(\bar{x}_k - x_k)\Delta_{ik}(x) + (\bar{x}_j - x_j)(\bar{x}_k - x_k)\Delta_{jk}(x) + (\bar{x}_i - x_i)(\bar{x}_j - x_j)(\bar{x}_k - x_k)\Delta_{ijk}(x)$$

Proof: (see appendix A)

In the next section we will give a general r -flip search strategy for two special classes of Pseudo Boolean optimization models that have many important applications.

SPECIAL CASES

The application potential of quadratic and cubic Pseudo Boolean optimization is well known. This prominence leads us to single them out for special attention in light of theorems 3 and 4. These special cases are considered below:

Case #1: Quadratic pseudo boolean optimization:

Quadratic Unconstrained Binary Optimization (QUBO) can be stated as $\max_x f(x) = xQx$ where Q is a symmetric matrix^x. QUBO has numerous applications in optimization including binary linear program and maximum weighted 2-SAT. In fact any Pseudo-Boolean optimization can be transferred to QUBO (refer to (Kochenberger, Glover, Alidaee, & Rego, 2004), for a recent survey of applications of QUBO).

Glover, Kochenberger, Alidaee, and Amini (1999) used an efficient 1 -flip search implementation of QUBO and tested it in the context of Tabu Search and Scatter Search. We first give Glover's result and then extend it to the more general r -flip case for QUBO.

Definition 4: Let $x = (x_1, \dots, x_i, \dots, x_n)$ be a solution to QUBO and let $x' = (x_1, \dots, \bar{x}_i, \dots, x_n)$ be a solution obtained from x by complementing x_i . Since Q is upper triangular matrix then (ij)th element, q_{ij} , is equal to the second derivative of $f(x)$, i.e., $\Delta_{ij}(x) = q_{ij}$, with respect to x_i and x_j .

Theorem 5. (Glover, et al., 1999): For x and x' as given in Definition 4, the change in the value of the objective function is

$$f(x') - f(x) = (x'_i - x_i) \Delta_i(x).$$

Moreover, in any local optimal solution of the QUBO with respect to *1-flip* search we have

$$\begin{aligned} \text{either } (x_i = 0 \text{ iff } \Delta_i(x) \geq 0) \\ \text{or } (x_i = 1 \text{ iff } \Delta_i(x) \leq 0), \quad \forall i = 1, \dots, n. \end{aligned}$$

Furthermore, after changing x_i to $\bar{x}_i$ the update for all $\Delta_i(x)$ can be calculated as follows

$$\begin{aligned} \forall j < i, \quad \Delta_j(x) &= \Delta_j(x) + q_{ji}(x'_i - x_i), \\ \forall j > i, \quad \Delta_j(x) &= \Delta_j(x) + q_{ij}(x'_i - x_i), \\ j = i, \quad \Delta_j(x) &= \Delta_i(x). \end{aligned}$$

In Theorem 6, we extend the above result to general case for *r-flip* search for QUBO.

Theorem 6. *Let x be a given solution of QUBO and x' obtained from x by an *r-flip* move where $S \subseteq V$, and $|S| = r$. Now, the change in the value of the objective function is*

$$\begin{aligned} f(x') - f(x) = \\ \sum_{i \in S} (\bar{x}_i - x_i) \Delta_i(x) + \\ \sum_{\forall i, j \in S} (\bar{x}_i - x_i)(\bar{x}_j - x_j) \Delta_{ij}(x). \end{aligned}$$

Furthermore, after changing x to x' the update for all $\Delta_i(x)$ can be calculated as follows,

$$\begin{aligned} \forall j \in V \setminus S \quad \Delta_j(x) &= \Delta_j(x) + \\ &\sum_{i \in S} (\bar{x}_i - x_i) \Delta_{ij}(x), \text{ and} \\ \forall j \in S \quad \Delta_j(x) &= \Delta_j(x) + \\ &\sum_{i \in S \setminus \{j\}} (\bar{x}_i - x_i) \Delta_{ij}(x). \end{aligned}$$

Proof: (see Appendix A)

Remarks: Theorem 6 extends theorem 3 to the case of *r-flip* search specialized for QUBO. As such, the theorem facilitates extensive neighborhood searches to be carried out when solving QUBO. Given a solution x , *r-flip* moves are made until no such move results to a better solution, yielding a local optimal solution with respect to the *r-flip* neighborhood. Such local search mechanisms can be embedded in a metaheuristic framework designed to escape the trap of local optimality and effectively search challenging solution spaces. From computational point of view the calculation of the impact of such an *r-flip* moves depends only on calculating first derivatives, $\Delta_i(x)$, for all $i \in V$ since $\Delta_{ij}(x) = q_{ij}$ is a constant. Thus, evaluation of change in the objective function can be done in $O(r^2)$ while the updating of $\Delta_i(x)$ can be done in $O(r)$ for $i = 1, \dots, n$. Note also that depending on how $S \subseteq V$ is chosen, the operations indicated can be carried out in a parallel environment. Thus, appropriate implementation of theorem 6 can also result in a very strong parallel implementation.

Case 2: Cubic pseudo boolean optimization:

The Max 3-SAT problem, as well as other important problems, can be formulated as a cubic Pseudo-Boolean optimization problem. This case is taken up here where in Theorem 7 (below) we extend the previous result to the case when $f(x)$ has up to cubic terms. Note that in this case for every two elements $\{i, j\} \subseteq V$ we have

$$\Delta_{ij}(x) = q_{ij} + \sum_{l \in V \setminus \{i, j\}} q_{ijl} x_l$$

and for every three elements $\{i, j, k\} \subset V$ we have $\Delta_{ijk}(x) = q_{ijk}$ which is a constant. We use the following equations in our proof.

$$\text{For } i \in S, \quad \Delta_i(x) = q_i + \sum_{j \in S \setminus \{i\}} q_{ij}x_j + \sum_{j \notin S} q_{ij}x_j + \sum_{\substack{j \in S \setminus \{i\} \\ k \notin S}} q_{ijk}x_jx_k + \sum_{\substack{j, k \in S \setminus \{i\}}} q_{ijk}x_jx_k + \sum_{j, k \notin S} q_{ijk}x_jx_k$$

$$\text{For } i, j \in S, \quad \Delta_{ij}(x) = q_{ij} + \sum_{k \in S \setminus \{i, j\}} q_{ijk}x_k + \sum_{k \notin S} q_{ijk}x_k$$

Theorem 7. Let x be a given solution to a cubic Pseudo-Boolean function $f(x)$ and x' obtained from x by an r -flip move where $S \subseteq V$ and $|S| = r$. The change to the value of the objective function is

$$f(x') - f(x) = \sum_{i \in S} (\bar{x}_i - x_i) \Delta_i(x) + \sum_{i, j \in S} (\bar{x}_i - x_i)(\bar{x}_j - x_j) \Delta_{ij}(x) + \sum_{i, j, k \in S} (\bar{x}_i - x_i)(\bar{x}_j - x_j)(\bar{x}_k - x_k) \Delta_{ijk}(x)$$

Furthermore, after changing x to x' the update for all $\Delta_i(x)$ can be calculated as follows,

$$\forall j \in V \setminus S \quad \Delta_j(x) = \Delta_j(x) + \sum_{i \in S} (\bar{x}_i - x_i) q_{ij} + \sum_{i, k \in S} (\bar{x}_i \bar{x}_k - x_i x_k) q_{ijk}$$

$$\forall j \in S \quad \Delta_j(x) = \Delta_j(x) + \sum_{i, j \in S(j)} (\bar{x}_i - x_i) q_{ij} + \sum_{i, k \in S \setminus (j)} (\bar{x}_i \bar{x}_k - x_i x_k) q_{ijk},$$

and the update for $\Delta_{ij}(x)$ for all i and j can be calculated as follows,

$$\forall i \notin S, j \notin S \quad \Delta_{ij}(x) = \Delta_{ij}(x) + \sum_{l \in S} (\bar{x}_l - x_l) q_{ijl},$$

$$\forall i \in S, j \in S \quad \Delta_{ij}(x) = \Delta_{ij}(x) + \sum_{l \in S \setminus \{i, j\}} (\bar{x}_l - x_l) q_{ijl},$$

$$\forall i \in S, j \notin S \quad \Delta_{ij}(x) = \Delta_{ij}(x) + \sum_{l \in S \setminus (j)} (\bar{x}_l - x_l) q_{ijl},$$

Proof: (see Appendix A)

Remarks: Theorem 7 extends theorem 4 to the case of r -flip search specialized for Cubic Pseudo Boolean optimization. As such it has significant implications for implementations

aimed at solving cubic problems. Note that evaluating the r -flip change in the objective function can be done in $O(r^3)$ time. The calculation of each $\Delta_i(x)$ can be done in $O(r^2)$ while the update for all $\Delta_{ij}(x)$ and $i, j \in V$ can be done in $O(r)$.

The results of theorems 1-7 can be used to construct effective heuristic search processes for a variety of Pseudo Boolean optimization problems. In the computational section below we illustrate such use by solving some fairly large max 3-sat problems.

COMPUTATIONAL EXPERIENCE

It is well known (see, for instance, (Boros & Hammer, 2002; Hammer & Rudeanu, 1968; Hansen, Brunswick, & Jaumard, 1990)) that max 3-sat problems can be formulated as Pseudo Boolean optimization problems taking the form of minimizing a cubic pseudo Boolean function. The formulation procedure is as follows:

Denote the literals in a given clause by $u^\alpha = u$ if $\alpha = 1$ and $\bar{u}$ if $\alpha = 0$. Letting $\bar{\alpha} = 1 - \alpha$ and representing literals by binary variables, x_i , the max 3-Sat problem is equivalent to the unconstrained Pseudo Boolean optimization problem

$$\text{Min } f(x) = \sum_{j=1}^m \prod_{i \in I_j} x_i^{\bar{\alpha}_{ij}} \quad (1)$$

where m = the number of clauses in the problem and I_j is the index set denoting the variables in clause j . Denote by x^* a solution to the above problem. If $f(x^*) = 0$, x^* satisfies all m clauses. Otherwise, x^* is a solution to the max 3-sat problem and $f(x^*)$ is the number of clauses not satisfied by x^* .

As an illustration, consider the 3-sat example with 12 clauses and 5 variables shown below:

$$\begin{array}{ll}
1 & x_1 \vee x_2 \vee x_3 \\
2 & x_1 \vee \bar{x}_2 \vee x_3 \\
3 & \bar{x}_1 \vee x_2 \vee \bar{x}_3 \\
4 & x_2 \vee \bar{x}_3 \vee x_4 \\
5 & \bar{x}_2 \vee x_3 \vee x_4 \\
6 & \bar{x}_2 \vee \bar{x}_3 \vee \bar{x}_4 \\
7 & x_2 \vee x_4 \vee x_5 \\
8 & \bar{x}_2 \vee x_3 \vee x_5 \\
9 & x_2 \vee \bar{x}_3 \vee x_5 \\
10 & x_3 \vee x_4 \vee x_5 \\
11 & x_3 \vee \bar{x}_4 \vee \bar{x}_5 \\
12 & \bar{x}_3 \vee \bar{x}_4 \vee \bar{x}_5
\end{array}$$

By the procedure given above, the corresponding optimization problem is:

$$\begin{aligned}
\text{Min} \quad f(x) = & 3 - x_1 + x_2 - 2x_4 - 2x_5 + 2x_1x_3 - \\
& 4x_2x_3 + 3x_4x_5 - x_1x_2x_3 + 3x_2x_3x_4 - \\
& x_2x_4x_5 - x_3x_4x_5 + 2x_2x_3x_5
\end{aligned}$$

It is easy to see that $f(x^*) = 0$ for $x^* = (1, 1, 1, 0, 0)$ for this example implying that all 12 clauses are satisfied. More generally, any max 3-sat problem can be solved by minimizing $f(x)$ defined by (1).

The results given in sections 2 and 3 of this article were embedded in simple search strategies used to solve some fairly large instances of max 3-sat problems taken from the literature. In our computational experiments, we considered four simple variations of r -flip search applied to the test problems. To provide a benchmark for comparison, we also include in the results reported the performance of a standard metaheuristic method (GRASP) on these same problems. Below we give a brief overview of the methods tested followed by a table of results:

- **Method 1: (l -flip)** A simple search process as described in Section 2.

In this method we start with all variables equal to zero and make simple l -flip moves. Several passes through the variables (taken in the order 1 through n) are made until additional moves fail to yield improvement.

- **Method 2: (2 -flip)** Here we used a simple implementation of r -flip local search, as described in section 2, for $r = 2$. In our

implementation we used a double loop where one variable is flipped and temporarily fixed while a search over all other variables is made to identify the first improving flip to complete the 2 -flip move. The process continues until no improving 2 -flip move can be found.

- **Method 3: (2 - l -flip)** Here we implemented a slight extension of method 2 where the search process augments an initial 2 -flip search with an additional l -flip improvement procedure.

Our 2 -flip procedure of Method 2 was run until no further improvement was realized. Then, starting with the best solution at hand we applied the l -flip rule of Method 1.

- **Method 4: (F&F) Filtration and Sequential Fan** (Glover, 1998) is a search process utilizing compound move strategies that have proven to be effective for a variety of combinatorial problems. Here we employ a simple version of F&F utilizing a combination of r -flip moves. In the paragraph below we give a brief overview of our implementation. Additional details are given in the appendix B of this article.

F&F organizes an aggressive search process utilizing a variety of different r -flip moves. The version we tested here starts with a set of locally optimal solutions found using l -flip search. Then, for each solution in this set a second round of l -flip moves is executed. If an improving move is found we accept that move and initiate a complete new round of l -flip moves. If an improving second flip move can not be found, we initiate a series of 3 -flip moves starting with the best local optimal solutions found so far. This continues until a pre-set stopping criterion is satisfied or until no further improvement is realized.

Note that all four of the methods tested here are straightforward search procedures that

make use of the theorems presented earlier in the article.

- **Method 5:** (GRASP) As a benchmark for comparison we used the well known GRASP implementation for solving max 3-sat problems as described in the articles (Resende, 1998; Resende & Feo, 1996). The code is available via <http://www.research.att.com/~mgcr/>.

All methods were coded in FORTRAN and all runs were made on a Cray Super Computer J916 at the University of Mississippi. The test problems considered here were taken from the DIMACS test bed of 3-sat problems. To illustrate performance across a range of problem sizes, we randomly chose instances ranging in size from 100 variables and 403 clauses up to 2000 variables and 8500 clauses. We note that the problems chosen are known to be satisfiable and thus optimal solutions are known to exist that satisfy all the clauses in each instance. The GRASP code was run utilizing all default parameters and our methods 1-4 were run until no further improvement was realized or a limit of 3600 seconds was reached. The results of our runs are shown in Table 1.

In Table 1, n denotes the number of variables and m denotes the number of clauses in a given problem. #Unsat gives the number of clauses not satisfied by the best solution found during the search process and “time in seconds” gives the total solution time. Note that for the F&F method and for GRASP, we list the “time to best” solution as well as the total time.

An examination of Table 1 clearly indicates that Method 4 (F&F) gives the best performance in terms of solution quality for these problems. F&F quickly found optimal solutions for all but the two largest problems and for these two it outperformed GRASP, the second best performer, by a wide margin. The time performance of F&F is also greatly preferred to that of GRASP. While GRASP is competitive in terms of run time on the 100 variable problems, it degrades substantially in terms of run times on the larger

problems. For all of the problems instances with 250 variables or more, GRASP produced inferior solutions in greatly inflated run times compared to F&F. The most extreme example of this is given by the last problem where GRASP took more than 40 hours to produce a solution with twice as many unsatisfied clauses as F&F found in 1 hour of search time.

As expected, our simple search methods improve in solution quality as we go from method 1 to method 4 (F&F). While methods 1-3 are very fast, the extra time spent in method 4 (F&F) pays considerable dividends in terms of solution quality. The problems considered here are known to be computationally challenging and thus it is not surprising that these simpler forms of search fall short of the more aggressive searches in term of solution quality.

CONCLUSION

Organizing an effective search process requires an efficient assessment of the objective function response to a potential move. In this article we presented derivative-based results for such assessment, intended to facilitate *r-flip* strategies for Pseudo-Boolean optimization. Closed form expressions for these derivatives were given permitting efficient implementation to a wide variety of combinatorial optimization problems. Four simple variations of *r-flip* search along with the well-known method, GRASP, were applied to a set of DIMACS 3-SAT test problems to illustrate the use of the ideas presented. Our results showed that the search processes organized by these methods could produce high quality solutions very quickly. Our method 4 in particular (F&F) was very successful giving better performance than the other methods, including GRASP, in terms of both solution quality and solution time.

In the testing undertaken here, we did not apply *r-flip* for $r > 3$. We intend to explore this as part of our on-going work. We also note the results of Theorems 6 and 7 are highly parallelizable opening the possibility for efficient implementation aimed at addressing large scale

problems with very aggressive neighborhood search. These research avenues, along with more sophisticated versions of F&F, are part of our work in progress that we intend to report on in future articles.

REFERENCES

- Ahuja, R., Ergun, O., Orlin, J., & Punnen, A. (2002). A survey of very large-scale neighborhood search techniques. *Discrete Applied Mathematics*, 123, 75–102. doi:10.1016/S0166-218X(01)00338-9
- Ahuja, R., Orlin, J., Pallottino, S., Scaparra, M., & Scutellà, M. (2004). A multi-exchange heuristic for the single-source capacitated facility location problem. *Management Science*, 50, 749–760. doi:10.1287/mnsc.1030.0193
- Boros, E., & Hammer, P. (2002). Pseudo-Boolean optimization. *Discrete Applied Mathematics*, 123(1-3), 155–225. doi:10.1016/S0166-218X(01)00336-5
- Crama, Y., & Hammer, P. (2007). *Boolean functions: Theory, algorithms and applications*. Cambridge, UK: Cambridge University Press.
- Deincko, V., & Woeginger, G. (2000). A study of exponential neighborhoods for the travelling salesman problem and for the quadratic assignment problem. *Mathematical Programming . Ser. A*, 87, 519–542.
- Frangioni, A., Necciari, E., & Scutellà, M. (2004). A multi-exchange neighborhood for minimum makespan parallel machine scheduling problems. *Journal of Combinatorial Optimization*, 8, 195–220. doi:10.1023/B:JOCO.00000031420.05971.29
- Glover, F. (1996). Finding the best traveling salesman 4-opt move in the same time as a best 2-opt move. *Journal of Heuristics*, 2, 169–179. doi:10.1007/BF00247211
- Glover, F. (1998). A template for scatter search and path relinking. In J. K. Hao, E. Lutton, E. Ronald, M. Schoenauer, & D. Snyers (Eds.), *Artificial evolution* (Vol. 1363, pp. 13-54).
- Glover, F., Kochenberger, G., Alidaee, B., & Amini, M. (1999). Tabu search with critical event memory: An enhanced application for binary quadratic programs. In S. M. S. Voss, I. Osman, & C. Roucairol (Eds.), *Meta-heuristics, Advances and trends in local search paradigms for optimization* (pp. 93-109). Dordrecht, The Netherlands, Kluwer Academic Publishers.
- Hammer, P., & Rudeanu, S. (1968). *Boolean methods in operations research and related areas*. New York: Springer.
- Hansen, P., Brunswick, N., & Jaumard, B. (1990). Algorithms for the maximum satisfiability problem. *Computing*, 44, 279–303. doi:10.1007/BF02241270
- Kochenberger, G., Glover, F., Alidaee, B., & Rego, C. (2004). A unified modeling and solution framework for combinatorial optimization problems. *Operations Research Spectrum*, 26(2), 237–250.
- Li, W., & Alidaee, B. (2002). Dynamics of local search heuristics for the traveling salesman problem. *IEEE Transactions on Systems, Man, and Cybernetics - Part A*, 32, 173-184.
- Magazine, M., Polak, G., & Sharma, D. (2002). A multi-exchange neighborhood search for integrated clustering and machine setup model for PCB manufacturing. *Journal of Electronics Manufacturing*, 11, 107–119. doi:10.1142/S0960313102000448
- Resende, M. G. C. (1998). Computing approximate solutions of the maximum covering problem using GRASP. *Journal of Heuristics*, 4, 161–171. doi:10.1023/A:1009677613792
- Resende, M. G. C., & Feo, T. A. (Eds.). (1996). *A GRASP for satisfiability, cliques, coloring, and satisfiability: Second DIMACS implementation challenge* (Vol. 26). Providence, RI: American Mathematical Society.
- Vredevel, T., & Lenstra, J. (2003). On local search for the generalized graph coloring problem. *Operations Research Letters*, 31, 28–34. doi:10.1016/S0167-6377(02)00165-7
- Yaguiura, M., & Ibaraki, T. (1999). Analyses on the 2 and 3-flip neighborhoods for the MAX SAT. *Journal of Combinatorial Optimization*, 3, 95–114. doi:10.1023/A:1009873324187
- Yaguiura, M., & Ibaraki, T. (2001). Efficient 2 and 3-flip neighborhood search algorithms for the MAX SAT: Experimental evaluation. *Journal of Heuristics*, 7, 423–442. doi:10.1023/A:1011306011437

APPENDIX A: THEOREM PROOFS

Proof of Theorem 3:

For $x = (x_1, \dots, x_i, \dots, x_j, \dots, x_n)$ and $x' = (x_1, \dots, \bar{x}_i, \dots, \bar{x}_j, \dots, x_n)$ we have

$$\begin{aligned}
 f(x) &= x_i \left[q_{\{i\}} + \sum_{l \neq ij} q_{\{i,l\}} x_l + \sum_{lm \neq ij} q_{\{i,l,m\}} x_l x_m + \sum_{lmh \neq ij} q_{\{i,l,m,h\}} x_l x_m x_h + \dots \right] + \\
 &\quad x_j \left[q_{\{j\}} + \sum_{l \neq ij} q_{\{j,l\}} x_l + \sum_{lm \neq ij} q_{\{j,l,m\}} x_l x_m + \sum_{lmh \neq ij} q_{\{j,l,m,h\}} x_l x_m x_h + \dots \right] + \\
 &\quad x_i x_j (q_{\{i,j\}} + \sum_{l \neq ij} q_{\{i,j,l\}} x_l + \sum_{lm \neq ij} q_{\{i,j,l,m\}} x_l x_m + \sum_{lmh \neq ij} q_{\{i,j,l,m,h\}} x_l x_m x_h + \dots) + \dots \\
 f(x') &= \bar{x}_i \left[q_{\{i\}} + \sum_{l \neq ij} q_{\{i,l\}} x_l + \sum_{lm \neq ij} q_{\{i,l,m\}} x_l x_m + \sum_{lmh \neq ij} q_{\{i,l,m,h\}} x_l x_m x_h + \dots \right] + \\
 &\quad \bar{x}_j \left[q_{\{j\}} + \sum_{l \neq ij} q_{\{j,l\}} x_l + \sum_{lm \neq ij} q_{\{j,l,m\}} x_l x_m + \sum_{lmh \neq ij} q_{\{j,l,m,h\}} x_l x_m x_h + \dots \right] + \\
 &\quad \bar{x}_i \bar{x}_j [q_{\{i,j\}} + \sum_{l \neq ij} q_{\{i,j,l\}} x_l + \sum_{lm \neq ij} q_{\{i,j,l,m\}} x_l x_m + \sum_{lmh \neq ij} q_{\{i,j,l,m,h\}} x_l x_m x_h + \dots] + \dots
 \end{aligned}$$

For each bracket in $f(x)$ and $f(x')$ we can add and subtract appropriate terms to get the following:

$$\begin{aligned}
 f(x) &= x_i \left[\Delta_i(x) - x_j (q_{\{i,j\}} + \sum_{l \neq i,j} q_{\{i,j,l\}} x_l + \sum_{l,m \neq i,j} q_{\{i,j,l,m\}} x_l x_m + \dots) \right] + \\
 &\quad x_j \left[\Delta_j(x) - x_i (q_{\{i,j\}} + \sum_{l \neq i,j} q_{\{i,j,l\}} x_l + \sum_{l,m \neq i,j} q_{\{i,j,l,m\}} x_l x_m + \dots) \right] + \\
 &\quad x_i x_j \left[(q_{\{i,j\}} + \sum_{l \neq i,j} q_{\{i,j,l\}} x_l + \sum_{l,m \neq i,j} q_{\{i,j,l,m\}} x_l x_m + \dots) \right] + \dots
 \end{aligned}$$

Which we can write as

$$f(x) = x_i [\Delta_i(x) - x_j \Delta_{ij}(x)] + x_j [\Delta_j(x) - x_i \Delta_{ij}(x)] + x_i x_j [\Delta_{ij}(x)] + \dots$$

Likewise, we have

$$\begin{aligned}
 f(x') &= \bar{x}_i \left[\Delta_i(x) - x_j (q_{\{i,j\}} + \sum_{l \neq i,j} q_{\{i,j,l\}} x_l + \sum_{l,m \neq i,j} q_{\{i,j,l,m\}} x_l x_m + \dots) \right] + \\
 &\quad \bar{x}_j \left[\Delta_j(x) - x_i (q_{\{i,j\}} + \sum_{l \neq i,j} q_{\{i,j,l\}} x_l + \sum_{l,m \neq i,j} q_{\{i,j,l,m\}} x_l x_m + \dots) \right] + \\
 &\quad \bar{x}_i \bar{x}_j \left[(q_{\{i,j\}} + \sum_{l \neq i,j} q_{\{i,j,l\}} x_l + \sum_{l,m \neq i,j} q_{\{i,j,l,m\}} x_l x_m + \dots) \right] + \dots
 \end{aligned}$$

Which can be written as

$$f(x') = \bar{x}_i [\Delta_i(x) - x_j \Delta_{ij}(x)] + \bar{x}_j [\Delta_j(x) - x_i \Delta_{ij}(x)] + \bar{x}_i \bar{x}_j [\Delta_{ij}(x)] + \dots$$

So that

$$f(x') - f(x) = (\bar{x}_i - x_i) [\Delta_i(x) - x_j \Delta_{ij}(x)] + (\bar{x}_j - x_j) [\Delta_j(x) - x_i \Delta_{ij}(x)] + (\bar{x}_i \bar{x}_j - x_i x_j) \Delta_{ij}(x)$$

or

$$f(x') - f(x) = (\bar{x}_i - x_i) \Delta_i(x) + (\bar{x}_j - x_j) \Delta_j(x) + (\bar{x}_i - x_i)(\bar{x}_j - x_j) \Delta_{ij}(x)$$

Proof of Theorem 4:

Let $x = (x_1, \dots, x_i, \dots, x_j, \dots, x_k, \dots, x_n)$ and $x' = (x_1, \dots, \bar{x}_i, \dots, \bar{x}_j, \dots, \bar{x}_k, \dots, x_n)$ then we have

$$\begin{aligned} f(x) = & x_i [q_i + \sum_{l \neq i, j, k} q_{il} x_l + \sum_{l, h \neq i, j, k} q_{ilh} x_l x_h + \sum_{\substack{T \subseteq V \setminus \{i, j, k\} \\ |T| \geq 3}} q_{iT} \prod_{t \in T} x_t] + \\ & x_j [q_j + \sum_{l \neq i, j, k} q_{jl} x_l + \sum_{l, h \neq i, j, k} q_{jhl} x_l x_h + \sum_{\substack{T \subseteq V \setminus \{i, j, k\} \\ |T| \geq 3}} q_{jT} \prod_{t \in T} x_t] + \\ & x_k [q_k + \sum_{l \neq i, j, k} q_{kl} x_l + \sum_{l, h \neq i, j, k} q_{klh} x_l x_h + \sum_{\substack{T \subseteq V \setminus \{i, j, k\} \\ |T| \geq 3}} q_{kT} \prod_{t \in T} x_t] + \\ & x_i x_j \sum_{T \subseteq V \setminus \{i, j, k\}} q_{ijT} \prod_{t \in T} x_t + x_i x_k \sum_{T \subseteq V \setminus \{i, j, k\}} q_{ikT} \prod_{t \in T} x_t + x_j x_k \sum_{T \subseteq V \setminus \{i, j, k\}} q_{jkT} \prod_{t \in T} x_t + \\ & x_i x_j x_k \sum_{T \subseteq V \setminus \{i, j, k\}} q_{ijkT} \prod_{t \in T} x_t + \text{Terms not involving elements of } S \end{aligned}$$

Consider the first bracket of terms. By adding and subtracting appropriate terms we get:

$$\begin{aligned} & x_i [q_i + \sum_{l \neq i, j, k} q_{il} x_l + \sum_{l, h \neq i, j, k} q_{ilh} x_l x_h + \sum_{\substack{T \subseteq V \setminus \{i, j, k\} \\ |T| \geq 3}} q_{iT} \prod_{t \in T} x_t] + \\ & x_j \sum_{T \subseteq V \setminus \{i, j, k\}} q_{ijT} \prod_{t \in T} x_t + x_k \sum_{T \subseteq V \setminus \{i, j, k\}} q_{ikT} \prod_{t \in T} x_t + x_j x_k \sum_{T \subseteq V \setminus \{i, j, k\}} q_{ijkT} \prod_{t \in T} x_t - \\ & x_j \sum_{T \subseteq V \setminus \{i, j, k\}} q_{ijT} \prod_{t \in T} x_t - x_k \sum_{T \subseteq V \setminus \{i, j, k\}} q_{ikT} \prod_{t \in T} x_t - x_j x_k \sum_{T \subseteq V \setminus \{i, j, k\}} q_{ijkT} \prod_{t \in T} x_t] = \\ & x_i [\Delta_i(x) - x_j \sum_{T \subseteq V \setminus \{i, j, k\}} q_{ijT} \prod_{t \in T} x_t - x_k \sum_{T \subseteq V \setminus \{i, j, k\}} q_{ikT} \prod_{t \in T} x_t - x_j x_k \sum_{T \subseteq V \setminus \{i, j, k\}} q_{ijkT} \prod_{t \in T} x_t] = \\ & x_i [\Delta_i(x) - x_j \sum_{T \subseteq V \setminus \{i, j, k\}} q_{ijT} \prod_{t \in T} x_t - x_k \sum_{T \subseteq V \setminus \{i, j, k\}} q_{ikT} \prod_{t \in T} x_t - x_j x_k \sum_{T \subseteq V \setminus \{i, j, k\}} q_{ijkT} \prod_{t \in T} x_t \\ & - x_j x_k \sum_{T \subseteq V \setminus \{i, j, k\}} q_{ijkT} \prod_{t \in T} x_t + x_j x_k \sum_{T \subseteq V \setminus \{i, j, k\}} q_{ijkT} \prod_{t \in T} x_t] = \\ & x_i [\Delta_i(x) - x_j \Delta_{ij}(x) - x_k \Delta_{ik}(x) + x_j x_k \sum_{T \subseteq V \setminus \{i, j, k\}} q_{ijkT} \prod_{t \in T} x_t] = \\ & x_i [\Delta_i(x) - x_j \Delta_{ij}(x) - x_k \Delta_{ik}(x) + x_j x_k \Delta_{ijk}(x)] \end{aligned}$$

Similar pattern exists for each of the second and third brackets. Furthermore, the terms outside of the brackets can be written as:

$$\begin{aligned}
& x_i x_j \sum_{T \subseteq V \setminus \{i,j,k\}} q_{ijT} \prod_{t \in T} x_t + x_i x_k \sum_{T \subseteq V \setminus \{i,j,k\}} q_{ikT} \prod_{t \in T} x_t + x_j x_k \sum_{T \subseteq V \setminus \{i,j,k\}} q_{jkT} \prod_{t \in T} x_t + \\
& x_i x_j x_k \sum_{T \subseteq V \setminus \{i,j,k\}} q_{ijkT} \prod_{t \in T} x_t + \text{Terms not involving elements of } S = \\
& x_i x_j (\Delta_{ij}(x) - x_k \Delta_{ijk}(x)) + x_i x_k (\Delta_{ik}(x) - x_j \Delta_{ijk}(x)) + x_j x_k (\Delta_{jk}(x) - x_i \Delta_{ijk}(x)) + \\
& x_i x_j x_k \Delta_{ijk}(x) + \text{Terms not involving elements of } S
\end{aligned}$$

Grouping and rearranging gives the result:

$$\begin{aligned}
f(x) = & x_i [\Delta_i(x) - x_j \Delta_{ij}(x) - x_k \Delta_{ik}(x) + x_j x_k \Delta_{ijk}(x)] + \\
& x_j [\Delta_j(x) - x_i \Delta_{ij}(x) - x_k \Delta_{jk}(x) + x_i x_k \Delta_{ijk}(x)] + \\
& x_k [\Delta_k(x) - x_i \Delta_{ik}(x) - x_j \Delta_{jk}(x) + x_i x_j \Delta_{ijk}(x)] + \\
& x_i x_j (\Delta_{ij}(x) - x_k \Delta_{ijk}(x)) + x_i x_k (\Delta_{ik}(x) - x_j \Delta_{ijk}(x)) + x_j x_k (\Delta_{jk}(x) - x_i \Delta_{ijk}(x)) + \\
& x_i x_j x_k \Delta_{ijk}(x) + \text{Terms not involving elements of } S
\end{aligned}$$

Similarly for $f(x')$ we get

$$\begin{aligned}
f(x') = & \bar{x}_i [\Delta_i(x) - x_j \Delta_{ij}(x) - x_k \Delta_{ik}(x) + x_j x_k \Delta_{ijk}(x)] + \\
& \bar{x}_j [\Delta_j(x) - x_i \Delta_{ij}(x) - x_k \Delta_{jk}(x) + x_i x_k \Delta_{ijk}(x)] + \\
& \bar{x}_k [\Delta_k(x) - x_i \Delta_{ik}(x) - x_j \Delta_{jk}(x) + x_i x_j \Delta_{ijk}(x)] + \\
& \bar{x}_i \bar{x}_j (\Delta_{ij}(x) - x_k \Delta_{ijk}(x)) + \bar{x}_i \bar{x}_k (\Delta_{ik}(x) - x_j \Delta_{ijk}(x)) + \bar{x}_j \bar{x}_k (\Delta_{jk}(x) - x_i \Delta_{ijk}(x)) + \\
& \bar{x}_i \bar{x}_j \bar{x}_k \Delta_{ijk}(x) + \text{Terms not involving elements of } S
\end{aligned}$$

Note that in $f(x')$ all elements in brackets and parentheses are related to the solution x . Using the above results we get

$$\begin{aligned}
f(x') - f(x) = & (\bar{x}_i - x_i) [\Delta_i(x) - x_j \Delta_{ij}(x) - x_k \Delta_{ik}(x) + x_j x_k \Delta_{ijk}(x)] + \\
& (\bar{x}_j - x_j) [\Delta_j(x) - x_i \Delta_{ij}(x) - x_k \Delta_{jk}(x) + x_i x_k \Delta_{ijk}(x)] + \\
& (\bar{x}_k - x_k) [\Delta_k(x) - x_i \Delta_{ik}(x) - x_j \Delta_{jk}(x) + x_i x_j \Delta_{ijk}(x)] + \\
& (\bar{x}_i \bar{x}_j - x_i x_j) (\Delta_{ij}(x) - x_k \Delta_{ijk}(x)) + \\
& (\bar{x}_i \bar{x}_k - x_i x_k) (\Delta_{ik}(x) - x_j \Delta_{ijk}(x)) + \\
& (\bar{x}_j \bar{x}_k - x_j x_k) (\Delta_{jk}(x) - x_i \Delta_{ijk}(x)) + \\
& (\bar{x}_i \bar{x}_j \bar{x}_k - x_i x_j x_k) \Delta_{ijk}(x).
\end{aligned}$$

Rearranging yields the desired result.

Proof of Theorem 6:

For two vectors x' and x where component x_i for all $i \in S$ are complemented we have $f(x)$ and $f(x')$ as follows:

$$\begin{aligned}
f(x) &= \sum_{i \in S} x_i [q_{ij} + \sum_{j \notin S} x_j q_{ij}] + \sum_{ij \in S} x_i x_j q_{ij} + \dots \\
f(x) &= \sum_{i \in S} x_i [\Delta_i(x) - \sum_{j \notin S \setminus \{i\}} x_j q_{ij}] + \sum_{ij \in S} x_i x_j q_{ij} + \dots \\
f(x') &= \sum_{i \in S} \bar{x}_i [q_{ij} + \sum_{j \notin S} x_j q_{ij}] + \sum_{ij \in S} \bar{x}_i \bar{x}_j q_{ij} + \dots \\
f(x') &= \sum_{i \in S} \bar{x}_i [\Delta_i(x) - \sum_{j \notin S \setminus \{i\}} x_j q_{ij}] + \sum_{ij \in S} \bar{x}_i \bar{x}_j q_{ij} + \dots \\
f(x') - f(x) &= \sum_{i \in S} (\bar{x}_i - x_i) [\Delta_i(x) - \sum_{j \notin S \setminus \{i\}} x_j q_{ij}] + \sum_{ij \in S} (\bar{x}_i \bar{x}_j - x_i x_j) q_{ij} \\
f(x') - f(x) &= \sum_{i \in S} (\bar{x}_i - x_i) \Delta_i(x) - \sum_{i \in S} \sum_{j \notin S \setminus \{i\}} (\bar{x}_i - x_i) x_j q_{ij} + \sum_{ij \in S} (\bar{x}_i \bar{x}_j - x_i x_j) q_{ij}
\end{aligned}$$

For each pair of elements i, j in the second and the third terms we have

$$\begin{aligned}
\bar{x}_i \bar{x}_j + x_i x_j - \bar{x}_i x_j - x_i \bar{x}_j &= (\bar{x}_i - x_i)(\bar{x}_j - x_j) \\
f(x') - f(x) &= \sum_{i \in S} (\bar{x}_i - x_i) \Delta_i(x) + \sum_{\forall i, j \in S} (\bar{x}_i - x_i)(\bar{x}_j - x_j) \Delta_{ij}(x)
\end{aligned}$$

When the elements of the set S are flipped we need to update elements of vector $\Delta(x) = (\Delta_1(x), \dots, \Delta_n(x))$. Since each element of the vector, i.e., the j^{th} element, is the first derivative of f with respect to the j^{th} component, updating this vector after complementing elements of S only involves the addition or subtraction of q_{ij} depending on the present value of x_i . In fact we have

$$\begin{aligned}
\forall j \notin S, \quad \Delta_j(x) &= q_j + \sum_{i \notin S \setminus \{j\}} q_{ij} x_i + \sum_{i \in S} q_{ij} x_i \quad \text{and} \quad \Delta_j(x') = q_j + \sum_{i \notin S \setminus \{j\}} q_{ij} x_i + \sum_{i \in S} q_{ij} \bar{x}_i \\
\forall j \in S, \quad \Delta_j(x) &= q_j + \sum_{i \notin S} q_{ij} x_i + \sum_{i \in S \setminus \{j\}} q_{ij} x_i \quad \text{and} \quad \Delta_j(x') = q_j + \sum_{i \notin S} q_{ij} x_i + \sum_{i \in S \setminus \{j\}} q_{ij} \bar{x}_i
\end{aligned}$$

Note that we can substitute $\Delta_{ij}(x)$ for q_{ij} . From this the desired result follows immediately.

Proof of theorem 7:

Given x , let x' be a solution obtained from x by complementing x_i for all $i \in S$ and $|S|=r$. Values of $f(x)$ and $f(x')$ can be calculated as follows,

$$\begin{aligned}
f(x) &= \sum_{i \in S} x_i [q_i + \sum_{j \notin S} q_{ij} x_j + \sum_{j, k \notin S} q_{ijk} x_j x_k] + \\
&\quad \sum_{i, j \in S} q_{ij} x_i x_j + \sum_{\substack{i, j \in S \\ k \notin S}} q_{ijk} x_i x_j x_k + \sum_{i, j, k \in S} q_{ijk} x_i x_j x_k +
\end{aligned}$$

Terms not involving elements of S

$$f(x) = \sum_{i \in S} x_i [\Delta_i(x) - \sum_{j \in S \setminus \{i\}} q_{ij} x_j - \sum_{\substack{j \in S \setminus \{i\} \\ k \notin S}} q_{ijk} x_j x_k - \sum_{j, k \in S \setminus \{i\}} q_{ijk} x_j x_k] + \\ \sum_{i, j \in S} x_i x_j (q_{ij} + \sum_{k \notin S} q_{ijk} x_k) + \sum_{i, j, k \in S} q_{ijk} x_i x_j x_k + \\ \text{Terms not involving elements of } S$$

$$f(x) = \sum_{i \in S} x_i [\Delta_i(x) - \sum_{j \in S \setminus \{i\}} \Delta_{ij}(x) x_j + \sum_{j, k \in S \setminus \{i\}} q_{ijk} x_j x_k] + \\ \sum_{i, j \in S} x_i x_j (\Delta_{ij}(x) - \sum_{k \in S \setminus \{i, j\}} q_{ijk} x_k) + \sum_{i, j, k \in S} q_{ijk} x_i x_j x_k + \\ \text{Terms not involving elements of } S$$

$$f(x) = \sum_{i \in S} x_i [\Delta_i(x) - \sum_{j \in S \setminus \{i\}} \Delta_{ij}(x) x_j + \sum_{j, k \in S \setminus \{i\}} \Delta_{ijk}(x) x_j x_k] + \\ \sum_{i, j \in S} x_i x_j (\Delta_{ij}(x) - \sum_{k \in S \setminus \{i, j\}} \Delta_{ijk}(x) x_k) + \sum_{i, j, k \in S} \Delta_{ijk}(x) x_i x_j x_k + \\ \text{Terms not involving elements of } S$$

Similarly, we can calculate $f(x')$ as follows.

$$f(x') = \sum_{i \in S} \bar{x}_i [q_i + \sum_{j \notin S} q_{ij} x_j + \sum_{\substack{j, k \notin S}} q_{ijk} x_j x_k] + \\ \sum_{i, j \in S} q_{ij} \bar{x}_i \bar{x}_j + \sum_{\substack{i, j \in S \\ k \notin S}} q_{ijk} \bar{x}_i \bar{x}_j x_k + \sum_{i, j, k \in S} q_{ijk} \bar{x}_i \bar{x}_j \bar{x}_k + \\ \text{Terms not involving elements of } S$$

$$f(x') = \sum_{i \in S} \bar{x}_i [\Delta_i(x) - \sum_{j \in S \setminus \{i\}} q_{ij} x_j - \sum_{\substack{j \in S \setminus \{i\} \\ k \notin S}} q_{ijk} x_j x_k - \sum_{j, k \in S \setminus \{i\}} q_{ijk} x_j x_k] + \\ \sum_{i, j \in S} \bar{x}_i \bar{x}_j (q_{ij} + \sum_{k \notin S} q_{ijk} x_k) + \sum_{i, j, k \in S} q_{ijk} \bar{x}_i \bar{x}_j \bar{x}_k + \\ \text{Terms not involving elements of } S$$

or,

$$f(x') = \sum_{i \in S} \bar{x}_i [\Delta_i(x) - \sum_{j \in S \setminus \{i\}} \Delta_{ij}(x) x_j + \sum_{j, k \in S \setminus \{i\}} \Delta_{ijk}(x) x_j x_k] + \\ \sum_{i, j \in S} \bar{x}_i \bar{x}_j (\Delta_{ij}(x) - \sum_{k \in S \setminus \{i, j\}} \Delta_{ijk}(x) x_k) + \sum_{i, j, k \in S} \Delta_{ijk}(x) \bar{x}_i \bar{x}_j \bar{x}_k + \\ \text{Terms not involving elements of } S$$

Given these expressions for $f(x)$ and $f(x')$ we then have

$$f(x') - f(x) = \sum_{i \in S} (\bar{x}_i - x_i) [\Delta_i(x) - \sum_{j \in S \setminus \{i\}} \Delta_{ij}(x) x_j + \sum_{j, k \in S \setminus \{i\}} \Delta_{ijk}(x) x_j x_k] + \\ \sum_{i, j \in S} (\bar{x}_i \bar{x}_j - x_i x_j) (\Delta_{ij}(x) - \sum_{k \in S \setminus \{i, j\}} \Delta_{ijk}(x) x_k) + \sum_{i, j, k \in S} \Delta_{ijk}(x) (\bar{x}_i \bar{x}_j \bar{x}_k - x_i x_j x_k)$$

$$-(\bar{x}_i - x_i)\Delta_{ij}(x)x_j - (\bar{x}_j - x_j)\Delta_{ij}(x)x_i + (\bar{x}_i\bar{x}_j - x_ix_j)\Delta_{ij}(x) = \\ \Delta_{ij}(x)(x_ix_j - \bar{x}_ix_j + x_ix_j - x_i\bar{x}_j + \bar{x}_i\bar{x}_j - x_ix_j) = (\bar{x}_i - x_i)(\bar{x}_j - x_j)\Delta_{ij}(x),$$

$$\left[(\bar{x}_i - x_i)x_jx_k + (\bar{x}_j - x_j)x_ix_k + (\bar{x}_k - x_k)x_ix_j \right. \\ \left. - (\bar{x}_i\bar{x}_j - x_ix_j)x_k - (\bar{x}_i\bar{x}_k - x_ix_k)x_j - (\bar{x}_j\bar{x}_k - x_jx_k)x_i \right. \\ \left. + \bar{x}_j\bar{x}_kx_i - x_ix_jx_k \right] \Delta_{ijk}(x) = (\bar{x}_i - x_i)(\bar{x}_j - x_j)(\bar{x}_k - x_k)\Delta_{ijk}(x)$$

Regrouping yields the desired result

$$f(x') - f(x) = \sum_{i \in S} (\bar{x}_i - x_i)\Delta_i(x) + \sum_{i,j \in S} (\bar{x}_i - x_i)(\bar{x}_j - x_j)\Delta_{ij}(x) + \\ \sum_{i,j,k \in S} (\bar{x}_i - x_i)(\bar{x}_j - x_j)(\bar{x}_k - x_k)\Delta_{ijk}(x)$$

Which completes the first part of the proof.

To prove the second part of the theorem we consider

$$\begin{aligned} \forall j \in V \setminus S \quad \Delta_j(x) &= q_j + \sum_{i \notin S \setminus \{j\}} q_{ij}x_i + \sum_{i,k \notin S \setminus \{j\}} q_{ijk}x_ix_k + \sum_{i \in S} q_{ij}x_i + \sum_{i,k \in S} q_{ijk}x_ix_k \quad \text{and} \\ \forall j \in S \quad \Delta_j(x) &= q_j + \sum_{i \notin S} q_{ij}x_i + \sum_{i,k \notin S} q_{ijk}x_ix_k + \sum_{i \in S \setminus \{j\}} q_{ij}x_i + \sum_{i,k \in S \setminus \{j\}} q_{ijk}x_ix_k. \\ \forall j \in V \setminus S \quad \Delta_j(x') &= q_j + \sum_{i \notin S \setminus \{j\}} q_{ij}x_i + \sum_{i,k \notin S \setminus \{j\}} q_{ijk}x_ix_k + \sum_{i \in S} q_{ij}\bar{x}_i + \sum_{i,k \in S} q_{ijk}\bar{x}_i\bar{x}_k \quad \text{and} \\ \forall j \in S \quad \Delta_j(x') &= q_j + \sum_{i \notin S} q_{ij}x_i + \sum_{i,k \notin S} q_{ijk}x_ix_k + \sum_{i \in S \setminus \{j\}} q_{ij}\bar{x}_i + \sum_{i,k \in S \setminus \{j\}} q_{ijk}\bar{x}_i\bar{x}_k. \end{aligned}$$

Upon applying $\Delta_j(x') - \Delta_j(x)$ the desired result follows immediately.

To prove the third part of the theorem we consider

$$\begin{aligned} \forall i \notin S, j \notin S \quad \Delta_{ij}(x) &= q_{ij} + \sum_{l \in (V \setminus S \cup \{i,j\})} q_{ijl}x_l + \sum_{l \in S} q_{ijl}x_l, \text{ and} \\ \Delta_{ij}(x') &= q_{ij} + \sum_{l \in (V \setminus S \cup \{i,j\})} q_{ijl}x_l + \sum_{l \in S} q_{ijl}\bar{x}_l \\ \forall i \in S, j \in S \quad \Delta_{ij}(x) &= q_{ij} + \sum_{l \in V \setminus S} q_{ijl}x_l + \sum_{l \in S \setminus \{i,j\}} q_{ijl}x_l, \text{ and} \\ \Delta_{ij}(x') &= q_{ij} + \sum_{l \in V \setminus S} q_{ijl}x_l + \sum_{l \in S \setminus \{i,j\}} q_{ijl}\bar{x}_l \\ \forall i \in S, j \notin S \quad \Delta_{ij}(x) &= \Delta_{ij}(x) = q_{ij} + \sum_{l \in (V \setminus S \cup \{i\})} q_{ijl}x_l + \sum_{l \in S \setminus \{j\}} q_{ijl}x_l, \text{ and} \\ \Delta_{ij}(x') &= q_{ij} + \sum_{l \in (V \setminus S \cup \{i\})} q_{ijl}x_l + \sum_{l \in S \setminus \{j\}} q_{ijl}\bar{x}_l. \end{aligned}$$

Upon applying $\Delta_{ij}(x') - \Delta_{ij}(x)$ the desired result follows.

APPENDIX B: A FILTER AND FAN PROCEDURE

The Filter and Fan (F&F) method, originally proposed by Glover (1998) as an improvement method for Scatter Search, can be used as a standalone search method that organizes a search process that dynamically cycles through alternative neighborhoods depending on conditions encountered. This form of strategic oscillation, employing a variety of neighborhood structures and r-flip moves, is intended to produce a robust yet economical search of the solution space. While F&F is rather general and can be implemented in a variety of ways, our work here employs a basic version known as “Strict Improvement F&F.” We summarize this method below as given by Glover (1998).

1. **Generate a candidate list of 1-moves for the current solution x:**

- a. If any of the 1-moves are improving: choose the best member from the list and execute it to create a new current solution x . (The best member may be the only member if the list terminates with the first improving move encountered.) Then return to the start of step 1.
- b. If none of the 1-moves is improving: Identify the set M of the n_0 best 1-moves examined. Let $M(1)$ be a subset of the n_1 best moves from M , and let $X(1)$ be the set of solutions produced by these moves. Set $L=1$ and proceed to step 2.

2. **For each L-move m in $M(L)$:** Identify the associated set $A(m)$ of the n_2 best compatible moves m' derived from M , and evaluate each solution that results by applying move m' to the corresponding member of $X(L)$. When fewer than n_2 moves of M are compatible with move m , restrict consideration to this smaller set of moves in composing $A(m)$.

- a. If an improving move is found during the foregoing process: Select the best such move generated (by the point where the process is elected to terminate), and execute the move to create a new current solution x . Then return to step 1.

If no improving move is found by the time all moves in $M(L)$ are examined: Stop if L has reached a chosen upper limit $MaxL$. Otherwise, identify the set $M(L+1)$ of the n_1 best $(L+1)$ -moves evaluated (and/or identify the associated set $X(L+1)$). (If fewer than n_1 distinct $(L+1)$ -moves in $M(L+1)$.) Then set $L=L+1$ and return to the start of Step 2.

In Step 1(a) of the procedure, 1-flip (1-move) local search is applied to a starting solution to produce a local optimal solution. When the 1-move process fails to yield further improvement, a set of the best moves considered, M , is maintained and a subset of these moves, $M(1)$, is used to create a set of solutions $X(1)$ in Step 1(b). The moves carried out here are called the Level-1 moves. Then in Step2, for each move m in $M(1)$, the set $A(m)$ of compatible m' move(s) is constructed from M and compound moves are executed. During the process if an improving move is created the procedure goes to Step 1(a), i.e., to create a new local optimal solution.

When creating compound moves if no improving moves are found by the compound moves in step2(a), choose a set of high quality moves from the compound moves to create $M(2)$ and an associated set of solutions $X(2)$. (Level-2 moves). Continue the process to even higher levels of moves until a stopping criteria, $MaxL$ is reached. In our implementation we chose parameters as follows, $|M|=40$, $|M(L)|=4$, $|A(m)|=4$, and $MaxL=20$. We used simple short term Tabu implementation by making a tabu list for the set $M(L)$ equal to $\min\{50, 0.25n\}$. (See Table 1)

Table 1. Max 3-Sat results from five different methods

<i>n, m</i>	Method 1: 1-flip		Method 2: 2-flip		Method 3: 2-1-flip		Method 4: F&F			GRASP		
	#Unsat	Time seconds	#Unsat	Time seconds	#Unsat	Time seconds	#Unsat	Time to Best seconds	Total Time Seconds	#Unsat	Time to Best seconds	Total Time seconds
100, 403	13	.01	7	.02	5	0.02	0	3.10	3.10	0	36.48	36.48
100, 403	12	.01	6	.02	4	0.02	0	195.10	195.10	0	641.89	641.89
100, 403	9	.02	9	.03	6	0.03	0	4.22	4.22	0	30.44	30.44
100, 403	7	.01	4	.02	5	0.02	0	11.01	11.01	0	10.75	10.75
250,1065	27	.60	14	1.74	14	1.74	0	161.25	161.25	4	2234.59	6227.66
250,1065	33	.59	20	1.58	18	1.58	0	189.34	189.34	8	342.32	4212.46
250,1065	28	.34	22	.99	13	0.99	0	119.12	119.12	5	5598.00	65605.40
250,1065	22	.32	16	1.01	15	1.01	0	385.32	385.32	8	1037.07	6483.66
250,1065	31	.31	21	1.04	11	1.04	0	184.82	184.82	5	1698.40	6386.63
600,2550	55	9.25	30	34.64	24	34.85	0	534.39	534.39	23	4467.11	27431.72
1000,4250	109	55.35	60	308.89	49	309.99	7	338.04	3600.00	49	35260.85	45359.88
2000,8500	202	259.02	126	649.38	107	661.44	49	982.21	3600.00	107	129157.39	159670.70

A Sociopsychological Perspective on Collective Intelligence in Metaheuristic Computing

Yingxu Wang, University of Calgary, Canada

ABSTRACT

In studies of genetic algorithms, evolutionary computing, and ant colony mechanisms, it is recognized that the higher-order forms of collective intelligence play an important role in metaheuristic computing and computational intelligence. Collective intelligence is an integration of collective behaviors of individuals in social groups or collective functions of components in computational intelligent systems. This article presents the properties of collective intelligence and their applications in metaheuristic computing. A social psychological perspective on collected intelligence is elaborated toward the studies on the structure, organization, operation, and development of collective intelligence. The collective behaviors underpinning collective intelligence in groups and societies are analyzed via the fundamental phenomenon of the basic human needs. A key question on how collective intelligence is constrained by social environment and group settings is explained by a formal motivation/attitude-driven behavioral model. Then, a metaheuristic computational model for a generic cognitive process of human problem solving is developed. This work helps to explain the cognitive and collective intelligent foundations of metaheuristic computing and its engineering applications.

Keywords: Behavior Models, Cognitive Computing, Cognitive Informatics, Cognitive Processes, Collective Behaviors, Collective Intelligence, Computational Intelligence, Metaheuristic Computing, Social Psychology

INTRODUCTION

Metaheuristic computing is an emerging computing methodology and technology (Geem et al., 2001; Blum & Roli; 2003; Glover & Gary; 2003; Talbi, 2009) developed in the field of genetic, evolutionary, autonomous, and search-based computing (Holland, 1975; Goldberg,

1989; Wang, 2008a, 2009c). The advances of metaheuristic computing are “higher-level problem solving without the hassle to design problem-specific operations each time a new application appears” (Yin, 2010).

In order to rigorously convey the concept of metaheuristic computing, the fundamental terms in metaheuristic computing are analyzed below.

DOI: 10.4018/jamc.2010102606

Definition 1. *A heuristic is a reasoning methodology in problem solving that enables a solution to a problem is derived by trial-and-error and/or rule of thumb.*

Definition 2. *A metaheuristic is a generic or higher-level heuristics that is more general in problem solving.*

Definition 3. *Computing in a narrow sense is an application of computers to solve a given problem by imperative instructions; while in a broad sense, it is a process to implement the instructive intelligence by a system that transfers a set of given information or instructions into expected intelligent behaviors.*

On the basis of Definitions 1 through 3, the concept of metaheuristic computing can be described as follows.

Definition 4. *Metaheuristic computing is an adaptive and/or autonomous methodology for computing that applies general heuristic rules, algorithms, and processes in solving a category of computational problems.*

It is recognized in abstract intelligence (αI) (Wang, 2009a) and cognitive informatics (Wang, 2002a, 2003, 2007b, 2009b; Wang et al., 2009; Wang, Zadeh & Yao, 2009), there are three categories of intelligent behaviors known as the *imperative*, *autonomic (adaptive)*, and *cognitive* intelligent behaviors. In a certain extent, metaheuristic computing intends to implement autonomic/adaptive intelligent behaviors beyond those of imperative ones in computational intelligence.

In order to explain the cognitive and sociopsychological properties of metaheuristic computing and its engineering applications, a transdisciplinary study between metaheuristic computing and collective intelligence are presented in this article. Parallel with studies in genetic algorithms, evolutionary computing, and ant colony mechanisms in metaheuristic

computing and computational intelligence, the higher-order forms of collective intelligence are human groups and societies. This work turns the attention to the collective intelligence of human societies and its applications in metaheuristic computing. This article presents a rigorous treatment of human social mechanisms in the context of collective intelligence. Collective behaviors in groups and societies underpinning collective intelligence are explored via analyses of the basic human needs. The constraints of collective intelligence are explained by social environment and group settings by a formal motivation/attitude-driven behavioral model. Then, the cognitive process of generic problem solving is formally described as a metaheuristic computing model for cognitive computing and computational intelligence.

COMPUTATIONAL INTELLIGENCE FOUNDATIONS OF METAHEURISTIC COMPUTING

As a preparation, the concept of metaheuristic computing is modeled using concept algebra. A formal model of genetic algorithms for metaheuristic computing is described via Real-Time Process Algebra (RTPA). A computational intelligence perspective on metaheuristic computing is presented, which synergizes metaheuristic computing with studies in collective intelligence.

The Concept Algebra Model of Metaheuristic Computing

Major metaheuristic computing concepts and techniques, according to Wikipedia, are such as genetic algorithms, evolutionary algorithms, random optimizations, local search, reactive search, greedy algorithm, hill-climbing, best-first search, simulated annealing, ant colony optimization, stochastic diffusion search, harmony search, and variable neighborhood search (Wikipedia, 2009). Therefore, the intension of metaheuristic computing is a set of generic

and meta algorithms, processes, and rules of computational problem solving. The extension of metaheuristic computing is a set of adaptive/autonomic, trial-and-error, and heuristic computing methodologies.

According to concept algebra (Wang, 2008b), a formal definition of metaheuristic computing based on Definition 4 can be rigorously described as follows.

Definition 5. *Metaheuristic computing, MHC, is formally defined as a 5-tuple, i.e.:*

$$MHC \triangleq (O, A, R^c, R^i, R^o) \quad (1)$$

where

- O is a finite nonempty set of objects, i.e., a set of MHC methodologies, $O = \{\text{meta-heuristic, adaptive, autonomic, trial-and-error, cognitive}\}$;
- A is a finite nonempty set of attributes, i.e., a set of generic algorithms, processes, and rules, where $A = \{\text{genetic algorithms, evolutionary algorithms, random optimizations, local search, reactive search, greedy algorithm, hill-climbing, best-first search, simulated annealing, ant colony optimization, stochastic diffusion search, harmony search, variable neighborhood search}\}$;
- $R^c = O \times A$ is a set of internal relations;
- $R^i \subseteq A \times A$, $A' \sqsubseteq C' \wedge A \sqsubseteq c$, is a set of input relations, where C' is a set of external concepts, $\delta(J, x', c)$ known as the concept environment. For convenience, $R^i = A \times A$ may be simply denoted as $R^i = C' \times c$.
- $R^o \subseteq c \times C'$ is a set of output relations.

The algebraic model of the concept of metaheuristic computing, MHC, provides an abstract mathematical structure for the formal treatment of MHC and its algebraic relations, operations, and associations (Wang, 2008d).

The Formal Model of Genetic Algorithms for Metaheuristic Computing

Genetic algorithms are proposed by John Holland in 1975 based on studies of cellular automata (Holland, 1975). It is extended by work of Goldberg (1989), Davis (1991), and Buckles and Petry (1992).

Definition 6. *A genetic algorithm is a recursive-search-based process that adopts adaptive selection in each generation of the recursive computation via crossovers and mutations.*

A mathematical model of genetic algorithm can be formally described as follows using RTPA (Wang, 2002b, 2007a, 2008c). (See Figure 1)

The natural-evolutionary-mechanism inspired methodology of genetic algorithm provides a new approach to design and implement adaptive and autonomic systems in computing and computational intelligence. Two technologies known as *crossover* and *mutation* play a central role in the genetic algorithm. The former is a means that allows a new generation of computing entity and its mechanism to be inherently different based on the combinatory properties of the population pool. The latter is a dynamics that creates a new generation of computing entity and its algorithm with extended new features and properties beyond its earlier generations based on heuristic selections. Genetic algorithms form the most important metaheuristic model for evolutionary and metaheuristic computing.

Computational Intelligence Perspectives on Metaheuristic Computing

Intelligence is a driving force or an ability to acquire and use knowledge and skills, or to reason in problem solving. Therefore, computational intelligence is a paradigm of the generic intelligence known as *abstract intelligence* (αI), which is the universal mathematical

Figure 1. The formal model of genetic algorithms

```

GeneticAlgorithm(<I:: (>; <O:: (>; <UDM:: (G(N)ST)>)PC  $\triangleq$ 
{ N := 0
  → GeneratePC (G(N)ST)
  → SolutionFoundBL:= F

  →  $\begin{matrix} \text{I} \\ \text{R} \end{matrix}$  (N := N + 1
    SolutionFoundBL:= F
    → GeneratePC (G(N)ST)
    → EvaluatePC((G(N)ST)
    → ( ♦ FitnessN((G(N)ST) = Satisfied
      → SolutionFoundBL:= T
      | ♦ ~
      → SolutionFoundBL:= F
      → CrossoverPC((G(N)ST)
      → MutationPC((G(N)ST)
    )
  }

```

form of intelligence that transfers information into knowledge and behaviors (Wang, 2009a). Wang's theory of *ai* states there are four types of intelligence known as the *reflective*, *perspective*, *instructive*, and *cognitive* intelligence. It is also recognized in *ai* that there are three categories of intelligent behaviors known as the *imperative*, *autonomic (adaptive)*, and *cognitive* intelligent behaviors (Wang, 2009a; Wang, Kinsner & Zhang, 2009).

Parallel with studies in ant colony mechanisms in metaheuristic computing and computational intelligence, the higher-order forms of collective intelligence, i.e., the human groups and societies, are recognized. Collective intelligence is a special phenomenon in sociology that natural or machine intelligence may coordinately work together in a formal or contingent structure toward a common goal by group intelligence. It is recognized that a new approach may be taken to study the mechanisms of metaheuristic computing focusing on collective intelligence of human societies and its interactive and cumulative properties. This approach creates synergies between metaheuristic computing and collective intelligence in sociology. As a system science at the most complicated level of human social organizations

(Wang, 2007a). Sociology studies structures and behaviors of human societies and how a human society may be organized efficiently and effectively on certain constraints of resources and environments (Macionis et al., 1997). In which, social psychology studies social interactions and their effects on human behaviors in collective intelligence (Wiggins et al., 1994).

Definition 7. *Collective intelligence is an integration of collective behaviors of individuals in social groups or collective functions of components in computational intelligent systems.*

A transdisciplinary study on metaheuristic computing and social psychology may explain a set of key mechanisms of collected intelligence. In this context, a society is a huge organized human system in which people are grouped, coordinated, interconnected, and interacted by a variety of organizations in order to form and enable collective intelligence. A society as a whole consists of individuals, groups, organizations, and sectors from the bottom up. Because the basic objects under study in sociology are individual human beings and their interactions, social psychology is the key to understand a wide range of complicated social phenomena

and collective intelligence, as well as the driving forces underpinning them.

In collective intelligence, a group is the basic social unit formed by two or more persons working toward a particular purpose. Groups are needed because of the *interdependency* among members when a given work cannot be carried out by a single individual limited by the scarcity of physical or intelligent capabilities and resources. The study on organizational psychology and collective social behaviors within groups and organizations helps to explain how structures of groups and organizations may impact people's behaviors, productivity, and performance in collective intelligence.

COLLECTIVE BEHAVIORS OF COLLECTIVE INTELLIGENCE

From a sociological point of view, collective intelligence is an integration of collective behaviors of individuals in a group or society. This section explores the fundamental human traits and the basic needs of individuals in a society in order to understand the natural drives and constraints of human collective behaviors.

The Hierarchical Model of Basic Human Needs

As an individual, the bottom-level basic biological need of humans is a stable inner environment regulated by a mechanism known as homeostasis.

Definition 8. *Homeostasis is an adaptive biological mechanism of the human body that maintains a relatively constant state in order to live and function.*

In psychology, Sigmund Freud perceived that humans are motivated by internal tension states known as drives that build up until they are released. The basic drives that Freud identified are self-preservation, sex, and aggression. However, he focused only on the last two drives

later in his theory (Freud, 1895; Leahey, 1997). Clark Hull proposed a drive-reduction theory that states motivation stems from a combination of drive and reinforcement of unfulfilled needs (Hull, 1943). The primary drives are innate drives such as hunger, thirst, and sex; the secondary drives are acquired drives such as studying, socializing, and earning money.

The hierarchical nature of human needs is identified by Abraham Maslow at five levels known as the needs of physiological, safety, social, esteem, and self-actualization from the bottom up (Maslow, 1962, 1970). The five basic levels of human needs are described in Table 1. Except those at Level 5, most needs identified by Maslow are deficiency needs, which are a need generated by a lack of something. The Level 5 needs for self-actualization can be classified as growth needs.

On the basis of the needs taxonomies of Maslow, Hull, and Freud, a formal human needs hierarchy model is developed below.

Definition 9. *The Human Needs Hierarchy (HNN) model is a hierarchical model that encompasses five-level fundamental human needs known from the bottom-up as N_0 – physiological needs, N_1 – psychological needs, N_2 – cognitive needs, N_3 – social needs, and N_4 – self-expressive needs.*

The HNN model can be illustrated as shown in Figure 2. Detailed explanations of each of the basic needs are provided in Table 2.

Lemma 1. *The lower the level of a need in the HNN hierarchy, the more concrete or material-oriented the need. In other words, the higher the level of a need, the more virtualized or perception-oriented the need.*

Definition 10. *The predominant need of an individual is the need at the lowest unsatisfied level of the HNN model.*

Table 1. Maslow’s hierarchy of needs

Level	Category	Needs	Description
1	Lower order needs	Physiological	Needs for biological maintenance such as food, water, sex, sleep etc.
2		Safety	Needs for physical and social security, protection, and stability such as shelter
3		Belongingness	Needs for love, affection, socialization
4	Higher order needs	Esteem	Needs for respect, prestige, recognition, and self-satisfaction
5		Self-actualization	Need to express oneself, grow, and to fulfill one’s maximum potential toward success

Figure 2. The human needs hierarchy (HNN) model



Table 2. The Human needs hierarchy (HNN) model

Level	Basic Needs	Description
N ₀	Physiological	Needs for maintaining homeostasis, such as food, water, clothes, sex, sleep, and shelter
N ₁	Psychological	Needs for feeling safe, comfortable, and wellbeing
N ₂	Cognitive	Needs for satisfaction of curiosity, knowledge, pleasure, and interaction with the environment
N ₃	Social	Needs for work, socialization, respect, prestige, esteem, and recognition
N ₄	Self-expressive	Need to express oneself, grow, and to fulfill one’s maximum potential toward success

Maslow suggests that human needs should be satisfied level by level. That is, the lower level needs should be satisfied before any higher level need comes into play (Maslow, 1970). This observation leads to the following principle.

Theorem 1. *When multiple needs of a person are unsatisfied at a given time, satisfaction of the most predominant need is most pressing.*

Understanding of the nature of basic human needs is not only useful in predicating motivations of human beings in collective intelligence, but also important in identifying the driving forces for the approach of group and society organization.

Collective Behaviors Underpinning Collective Intelligence

Studies on collective behaviors within groups and organizations may help to explain how structures of them impact people's behaviors, productivity, and performance, as well as the formation of collective intelligence. Psychological experiments indicate that individual's behavior may vary in a group influenced by the interactions with other members of the group, which is identified as collective behaviors (Zander, 1979; Wiggins et al., 1994).

Definition 11. *A collective behavior is an integrated behavior of a group in which individuals' behaviors are influenced in different ways by the group.*

Collective behaviors are one of the most important social properties of groups and organizations. It is perceived in sociology that any human social behavior may be compared and analyzed against the social norms, which forms a qualitative or quantitative standard for the behavior (Wiggins et al., 1994). Therefore, *individual behaviors* in the social context are measurable and analytical in term of performance against the social norms.

A set of observable phenomena of collective behaviors is formally described below, such as social conformity, social synchronization, coaction, coordination, groupthink, group polarization, social dilemmas, and social loafing.

Individuals intend to adjust their behavior or actions, which reflect their thought, to the

common goal and norms of a group that they involve and think belong to. This social phenomenon is called conformity.

Definition 12. *Social conformity is a social phenomenon in which an individual's behavior is approached to a social norm or standard in forms of ethical values, role expectations, and laws.*

Conformity may be explained by the principle of minimum energy consumption, especially when there is no obvious or intuitive best choice.

Individuals intend to set their behavior or actions to the timing of the group. This social phenomenon is called synchronization (Wang, 2007a).

Definition 13. *Social synchronization is a social phenomenon in which an individual's behavior is timed to a social norm of a group.*

Synchronization is a special type of social conformity. Synchronization may be explained by the principles of system synchronization and minimum energy consumption, because synchronization contributes to the maximum output of a group.

It is found that in temporary social situations and informal groups where no or little coordination is required, people still influence each other when their actions or tasks are identical or have similarity.

Definition 14. *A coaction is a social phenomenon in which the identical or similar actions or tasks are carried out by different individuals with little interaction.*

Coaction influences the performance of individuals because it puts the individual in a social context. The phenomena of coaction indicate there is a natural law, as described below, which constrains collective social behaviors of human beings even in a noncohesive social

context and a highly temporary and random social relation.

Lemma 2. *An autonomous synchronization tendency between individuals exists in any permanent or temporary social context where people automatically adjust to conjunctive goals and cooperative timing.*

The coercion influences on individual's performance can be positive or negative. The former can be a higher expectation, an awareness of difference, and a learning of better practice; while the latter can be a distraction or disappointment.

Coaction discussed above is an *ad hoc* cooperation in an informal group where there is no common goal as well as predefined means of cooperation and communication. In contrary to coaction, coordination happens in a formal group where common goals as well as means of cooperation and communications exist.

Definition 15. *A coordination is a social phenomenon in which the identical or similar action or task is carried out via intensive interactions between different individuals.*

Coordination may influence the performance of individuals dramatically in a group context. Organizational theories of work coordination and efficiencies of group coordination have been extensively studied in engineering sciences, system science, and management science, respectively, particularly the generic work coordination theory (Wang, 2007c).

In collective social behaviors, groupthink and group polarization are two undesirable phenomena identified in social psychology (Janis, 1971).

Definition 16. *Groupthink is a social phenomenon in which the decision-making process within a highly cohesive group is dominated by group consensus that restrains critical thinking of members in the group.*

Groupthink may occur in a highly cohesive group where decisions are made by the group in which individuals lose their ability to critically evaluate situations or information. Groupthink symptoms identified by Irving Janis in 1971 include illusion of invulnerability, illusion of morality, stereotypes of outsiders, pressure for conformity, self-censorship, and illusion of unanimity (Janis, 1971).

Groupthink acting as a filter of critical ideas may result in another social phenomenon known as group polarization that turns a group to a positive-feedback or instable system.

Definition 17. *Group polarization is a social phenomenon in which group members intend to shift toward the extreme of an already preferred position of the group.*

The tendency of group polarization is a powerful positive-feedback mechanism that may result in an instable state of a social unit or system. Behaviors of a positive-feedback system are sometime unpredictable even destructive. Therefore, the art of leadership for groups, to some extent, is to prevent the polarization situation in a group from happening.

Lemma 3. *A weighting system that encourages and appreciates negative or hesitant feedback towards the current group's position is a stable system.*

The rule of thumb is that, in a group polarization situation, the one who hesitates in the group is perhaps the wiser one. Therefore, Lemma 3 indicates that the negative-feedback mechanism is not only suitable for a natural system, but also applicable to social systems and collective intelligence.

Definition 18. *The social dilemma is a social phenomenon in which members of a group face a conflict choice between the maximization of group's interests by cooperative actions and*

the maximization of own individual's interests by noncooperative actions.

The collective behaviors of social dilemmas have been identified by many sociologists and social psychologists since 1985 (Komorita & Barth, 1985; Coleman, 1990). If only egoism is adopted in a society, the social dilemma may exist forever. However, when altruism is recognized to balance egoism, social dilemmas may be resolved systematically.

The collective behavior known as social loafing was first identified in Max Ringelmann's experiments on rope-pulling before World War I (Kravits & Martin, 1986). The same experiment was replicated by Alan Ingham et al. in 1974. This collective phenomenon is then termed as social loafing by Latane and his colleagues in 1979 based on extended studies (Latané et al., 1979; Hardy & Latane, 1986).

Definition 19. *Social loafing is a social phenomenon in which there exists the tendency for people to work less hard on a cooperative task in a group than they do individually.*

Three independent experiments on the efficiency of coordinated group tasks as shown in Table 3 reveal similar patterns of efficiency decreasing when more persons are involved in collective group tasks. These are the main evidences of social loafing. However, it can also be scientifically explained by coordinate

overhead and efficiency of systems according to Wang's abstract work coordination theory (Wang, 2007c).

A typical collective behavior of social loafing is the free-rider effect (Kerr, 1983).

Definition 20. *The free-rider effect is a social phenomenon in which there exists the tendency for a member of a group to act noncooperatively based on the assumption that one's individual cooperative action may not be necessary because others will do for the interests of the group.*

Another social loafing phenomenon is identified by Jackson and Harkins (1985) known as the sucker effect.

Definition 21. *The sucker effect is a social phenomenon in which exists the tendency for a member of a group to act noncooperatively based on the assumption that others may take advantage of one's individual cooperative contribution to the group.*

Social loafing may happen in a group where tasks are parallel allocated and the sum of all parallel capacity is much larger than the workload of the group, for instance, a group of porters and a team of programmers. The studies on collective behaviors indicate that rational and suitable organizational forms may

Table 3. Experiments on efficiency of coordinated group work

Collective tasks		An individual	A group			
			2 persons	3 persons	2-6 persons	8 persons
Rope-pulling ¹	Force (lb.)	130		352		546
	Efficiency	100%		90.3%		52.5%
Rope-pulling ²		100%	90%		85%	
Cheerleaders ³		100%			92%	

Note: Experiment 1 is based on Max Ringelmann [Kravits and Martin, 1986]
Experiment 2 is based on Alan Ingham et al. in 1974.
Experiment 3 is based on Hardy and Latane in 1986.

enable efficient coordinative work and reduce any negative effect.

COGNITIVE
SOCIOPSYCHOLOGY OF
COLLECTIVE INTELLIGENCE

On the basis of sociological foundations of collective intelligence, this section presents a rigorous treatment of human perceptual processes such as emotions, motivations, and attitudes, and their influences on human behaviors in collective intelligence. The interactions and relationships between motivation and attitude are formally described.

Emotions

Emotions are a set of states or results of perception that interprets the feelings of human beings on external stimuli or events in the binary categories of pleasant or unpleasant.

Definition 22. *An emotion is a subjective feeling derived from one's current internal status, mood, circumstances, historical context, and external stimuli.*

Emotions are closely related to desires and willingness. A desire is a personal feeling to possess an object, to conduct an interaction with the external world, or to prepare for an event to happen. A willingness is the faculty of conscious, deliberate, and voluntary choice of actions.

According to the study of Fischer and his colleagues (Fischer et al., 1990), the taxonomy

of emotions can be described as shown in Table 4.

It can be observed that human emotions at the perceptual layer may be classified into only two opposite categories: pleasant and unpleasant. Various emotions in the two categories can be classified at five levels according to its strengths of subjective feelings as shown in Table 5, where each level encompasses a pair of positive/negative or pleasant/unpleasant emotions.

Definition 23. *The strength of emotion $|E_m|$ is a normalized measure of how strong a person's emotion on a scale of 0 through 4, i.e.:*

$$0 \leq |E_m| \leq 4 \tag{2}$$

An organ known as the hypothalamus in the brain is supposed to interpret the properties or types of emotions in terms of pleasant or unpleasant (Smith, 1993; Leahey, 1997; Sternberg, 1998).

Definition 24. *Letting T_e be a type of emotion, ES the external stimulus, IS the internal perceptual status, and BL the Boolean values true or false, the perceptual mechanism of hypothalamus can be described as a function, i.e.:*

$$T_e: ES \times IS \rightarrow BL \tag{3}$$

It is interesting that sometime the same event or stimulus ES may be explained in different types due to the difference of the real-time context of the perceptual status IS of

Table 4. Taxonomy of emotions

Level	Description				
Supper level	Positive (pleasant)		Negative (unpleasant)		
Basic level	Joy	Love	Anger	Sadness	Fear
Sub-category level	Bliss, pride, contentment	Fondness, infatuation	Annoyance, hostility, contempt, jealousy	Agony, grief, guilt, loneliness	Horror, worry

Table 5. The hierarchy of emotions

Level (Positive/Negative)			Description
0	No emotion		-
1	Weak emotion	Comfort	Safeness, contentment, fulfillment, trust
		Fear	Worry, horror, jealousy, frightening, threatening
2	Mediate emotion	Joy	Delight, fun, interest, pride
		Sadness	Anxiety, loneliness, regret, guilt, grief, sorrow, agony
3	Strong emotion	Pleasure	Happiness, bliss, excitement, ecstasy
		Anger	Annoyance, hostility, contempt, infuriated, enraged
4	Strongest emotion	Love	Intimacy, passion, amorousness, fondness, infatuation
		Hate	Disgust, detestation, abhorrence, bitter

the brain. For instance, walking from home to office may be interpreted as a pleasant activity for one who likes physical exercises, but the same walk due to a car breakdown will be interpreted as unpleasant.

Corollary 1. *The human emotional system is a binary system that interprets or perceives an external stimulus and/or internal status as pleasant or unpleasant.*

Although there are various emotional categories in different levels, the binary emotional system of the brain provides a set of pairwise universal solutions to express human feelings. For example, anger may be explained as a default solution or generic reaction for an emotional event when there was no better solution available; otherwise, delight will be the default emotional reaction.

Motivations

Motivation is an innate potential power of human beings that energizes behavior. It is motivation that transforms thought (information) into action (energy). In other words, human behaviors are the embodiment of motivations.

Therefore, any cognitive behavior is driven by an individual motivation.

Definition 25. *A motivation is a willingness or desire triggered by an emotion to pursue a goal or a reason for triggering an action.*

As described in the LRMB model (Wang et al., 2006), motivation is a cognitive process of the brain at the perception layer that explains the initiation, persistence, and intensity of personal emotions and desires, which are the faculty of conscious, deliberate, and voluntary choices of actions.

Motivation is a psychological and social modulating and coordinating influence on the direction, vigor, and composition of behavior. This influence arises from a wide variety of internal, environmental, and social sources, and is manifested at many levels of behavioral and neural organizations.

The taxonomy of motives can be classified into two categories known as learned and unlearned (Wittig, 2001). The latter is the primary motives such as the *survival motives* (hunger, thirst, breathing, shelter, sleep, eliminating), and pain. The former is the secondary motives such as the need for achievement, friendship,

affiliation, dominance of power, and relief from anxiety.

Definition 26. *The strength of motivation M is a normalized measure of how strong a person's motivation is on a scale of 0 through 100, i.e.:*

$$0 \leq M \leq 100 \quad (4)$$

where $M = 100$ is the strongest motivation and $M = 0$ is the weakest motivation.

It is observed that the strength of a motivation is determined by multiple factors (Westen, 1999; Wang, 2007d) such as:

- The absolute motivation $|E_m|$:** The strength of the emotion.
- The relative motivation $E - S$:** A relative difference or inequity between the expectancy of a person E for an object or an action towards a certain goal and the current status S of the person.
- The cost to fulfill the motivation C :** A subjective assessment of the effort needed to accomplish the expected goal.

Therefore, the strength of a motivation can be quantitatively analyzed and estimated by the subjective and objective motivations and their cost as described in the following theorem.

Theorem 2. *The strength of motivations states that a motivation M is proportional to both the strength of emotion $|E_m|$ and the difference between the expectancy of desire E and the current status S , of a person, and is inversely proportional to the cost to accomplish the expected motivation C , i.e.:*

$$M = \frac{2.5 \bullet |E_m| \bullet (E-S)}{C} \quad (5)$$

where $0 \leq |E_m| \leq 4$, $0 \leq (E-S) \leq 10$, and $1 \leq C \leq 10$.

In Theorem 2, the strength of a motivation is measured in the scope of $[0 \dots 100]$, i.e., $0 \leq M \leq 100$. When $M > 1$, the motivation is considered being a desired motivation. The higher the value of M , the stronger the motivation.

It is noteworthy that motivation is only a potential mental power of human beings, and a strong motivation will not necessarily result in a behavior or action. The condition for transforming a motivation into a real behavior or action is dependent on multiple factors, such as values, social norms, expected difficulties, availability of resources, and the existence of alternative goals.

Attitudes

As described in the previous section, motivation is the potential power that may trigger an observable behavior or action. Before the behavior is performed, it is judged by an internal regulation system known as the attitude.

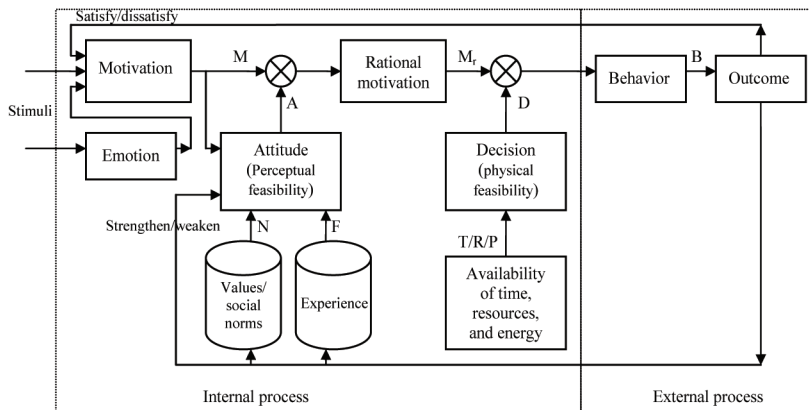
Attitudes are perceived in various ways in social psychology. Fazio describes an *attitude* as an association between an act or object and an evaluation (Fazio, 1986). Eagly and Chaiken define attitude as a tendency of a human to evaluate a person, concept, or group positively or negatively in a given context (1992). More recently, Arno Wittig describes attitude as a learned evaluative reaction to people, objects, events, and other stimuli (Wittig, 2001).

Definition 27. *An attitude is a subjective tendency towards a motivation, an object, a goal, or an action based on an intuitive evaluation of its feasibility.*

The modes of attitudes can be positive or negative, which can be quantitatively analyzed using the following definition.

Definition 28. *The mode of an attitude A is determined by both an objective judgment of its conformance to the social norm N and a subjective judgment of its empirical feasibility F , i.e.:*

Figure 3. The model of Motivation/Attitude-Driven Behavior (MADB) in collective intelligence



$$A = \begin{cases} 1, & N = \mathbf{T} \wedge F = \mathbf{T} \\ 0, & N = \mathbf{F} \vee F = \mathbf{F} \end{cases} \quad (6)$$

where $A = 1$ indicates a positive attitude; otherwise, it indicates a negative attitude.

THE INTERACTIVE BEHAVIORAL MODEL OF COLLECTIVE INTELLIGENCE

Based on the preceding subsections, the relationship between a set of interlinked perceptual psychological processes such as emotions, motivations, attitudes, decisions, and behaviors can be integrated into a coherent framework. A motivation/attitude-driven behavioral model will be developed for formally describing the cognitive processes of motivation and attitude in collective intelligence.

It is observed that motivation and attitude have considerable impact on behavior and influence the way a person thinks and feels (Westen, 1999). A reasoned action model is proposed by Martin Fishbein and Icek Ajzen in 1975 that suggests human behavior is directly generated by behavioral intentions, which are controlled by the attitude and social norms (Fishbein &

Ajzen, 1975). An initial motivation before the judgment by an attitude is only a temporal idea; with the judgment of the attitude, it becomes a rational motivation, also known as the behavioral intention.

The relationship between an emotion, motivation, attitude, and behavior can be formally and quantitatively described by the Motivation/Attitude-Driven Behavioral (MADB) model as illustrated in Figure 3. In the MADB model, motivation and attitude have been defined in Eqs. 5 and 6, respectively. It is noteworthy that, as shown in Figure 3, a motivation is triggered by an emotion or desire. The rational motivation, decision, and behavior can be quantitatively analyzed according to the following definitions.

Definition 29. A rational motivation M_r is a motivation regulated by an attitude A with a positive or negative judgment, i.e.:

$$M_r = M \cdot A = \frac{2.5 \cdot |E_m| \cdot (E-S)}{C} \cdot A \quad (7)$$

Definition 30. A decision for confirming an attitude, D_a , for executing a motivated behavior is a binary choice on the basis of the availability of time T , resources R , and energy P , i.e.:

$$D_a = \begin{cases} 1, & T \wedge R \wedge P = \mathbf{T} \\ 0, & T \vee R \vee P = \mathbf{F} \end{cases} \quad (8)$$

Therefore, the formal model of MADB can be described as follows, where a behavior is determined by a product of the strength of motivation and the approval of the decision by a positive attitude.

Lemma 4. *A behavior B driven by a motivation M_r and an attitude is a realized action initiated by a motivation M and supported by a positive attitude A and a positive decision D_a toward the action, i.e.:*

$$B = \begin{cases} \mathbf{T}, & M_r \cdot D_a = \frac{2.5 \cdot |E_m| \cdot (E-S)}{C} \cdot A \cdot D_a > 1 \\ \mathbf{F}, & \text{otherwise} \end{cases} \quad (9)$$

The MADB model presented in Lemma 4 and Figure 3 provides a rigorous explanation of the mechanisms and relationship between motivation, attitude, and behavior. The model can be used to explain how the motivation process drives human behaviors and actions, and how the attitude as well as the decision making process help to regulate the motivation and determines whether the motivation should be implemented.

THE COGNITIVE PROCESS OF PROBLEM SOLVING FOR METAHEURISTIC COMPUTING

The studies in cognitive informatics and collective intelligence as well as their denotational mathematical means can be widely applied in metaheuristic computing such as the theoretical model of Abstract Intelligence (AI) (Wang, 2009a), the Layered Reference Model of the Brain (LRMB) (Wang et al., 2006), and the Cognitive Reference Model of Autonomous Agents (Wang, 2009c).

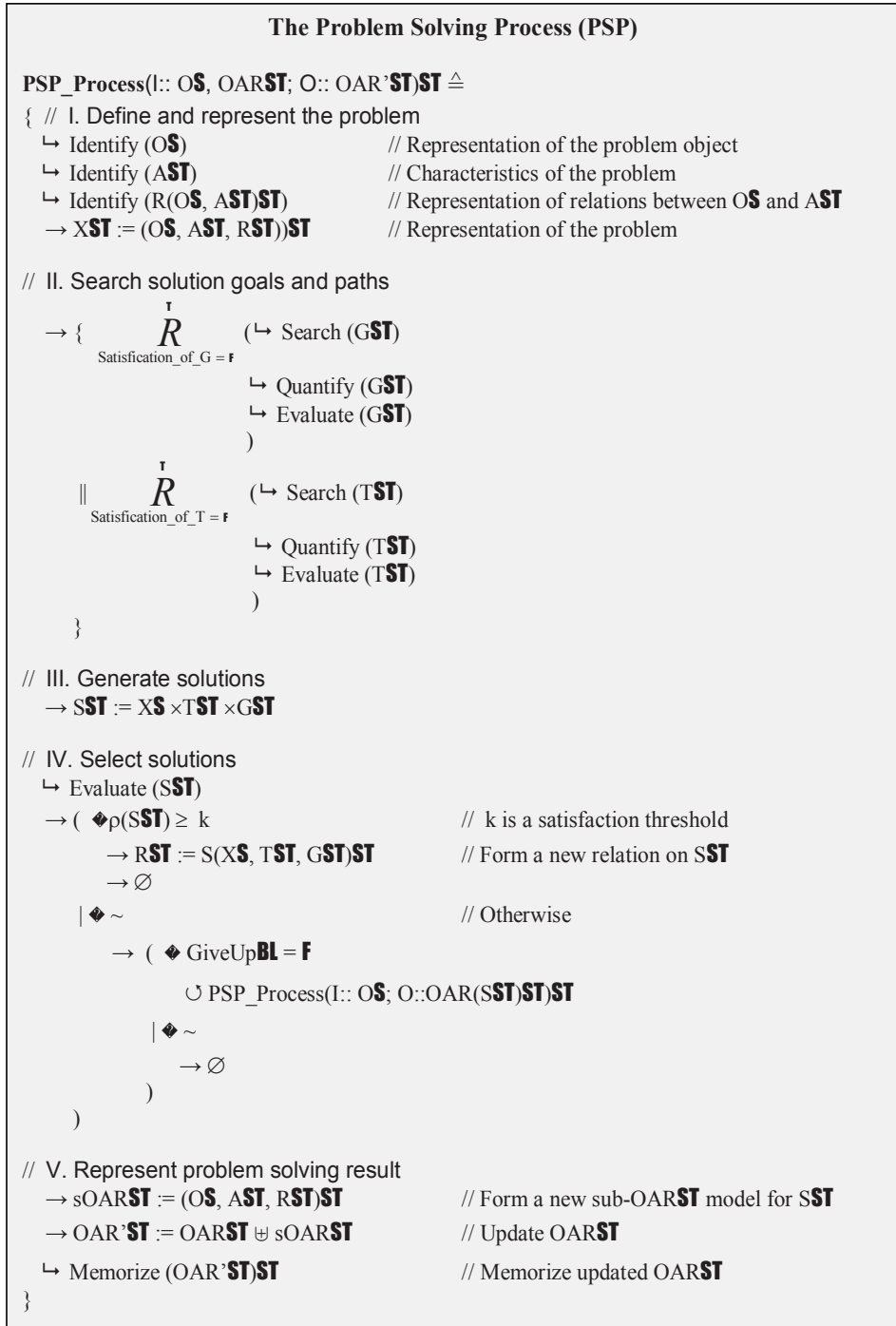
An interesting metaheuristic study in cognitive informatics and collective intelligence is the generic algorithm of problem solving as a fundamental cognitive process of the brain (Newell & Simon, 1972; Goldstein, 1978; Robertson, 2001; Wang & Chiew, 2009), which may be simulated and implemented by computational intelligence in metaheuristic computing.

Based on RTPA, concept algebra, and the Object-Attribute-Relation (OAR) model (Wang, 2007e) of internal knowledge representation in the brain, the cognitive process of problem solving can be formally elaborated in Figure 4, which can be divided into the following five steps:

- To define the problem:** This step describes the problem and its input layout X by identifying its object O_x and attributes A_x in a sub-OAR model.
- To search the solution goals and paths:** In this step, the brain performs a parallel search for possible goals G and paths T for the solution. External memory and resources may be searched if there is no available or sufficient G or T in the internal knowledge of the problem solver.
- To generate solutions:** This step forms a set of possible solutions by a Cartesian product $S = X \times T \times G$.
- To select suitable solutions:** This step evaluates each possible solutions $s \in S = X \times T \times G$ as obtained in Step (c). Recursive searching actions may be executed if the obtained solution(s) could not satisfy the problem solver's expectation.
- To represent the problem solving result:** This step incorporates and memorizes the solution(s), $s \in S$, as the result of problem solving into the entire OAR model in the long-term memory of the problem solver. Before memorization, the solution S is represented as a part of the relations, R , in the sub-OAR model.

It is noteworthy in Figure 4 that a number of lower layer cognitive processes, as represented

Figure 4. Formal description of the cognitive process of problem solving in RTPA



by double-ended boxes, are adopted to carry out the problem solving process. The relationships and interactions between the problem solving process and other cognitive processes can be explained according to the LRMB model (Wang et al., 2006) as illustrated in Figure 5. As a top layer process of the conscious life functions of the brain in LRMB, the problem solving process interacts with other higher cognitive processes at Layer 7, such as the learning and comprehension processes, it also involves lower layer processes such as those of abstraction, search, and memorization.

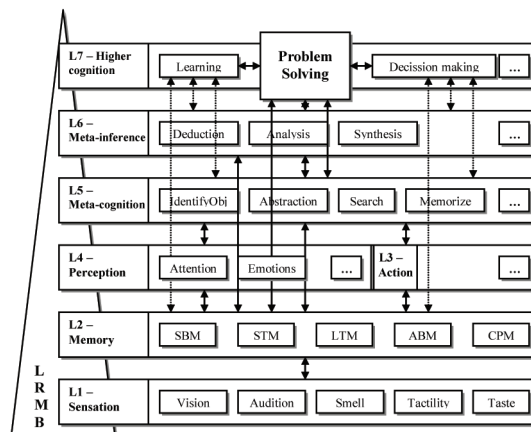
The generic cognitive process of problem solving provides a rigorous metaheuristic computing model for explaining a part of the functions of the natural intelligence from a social psychological point of view. On the basis of the metaheuristic model of problem solving, the cognitive process can be simulated by computational intelligence toward the mimic of the brain in metaheuristic computing.

CONCLUSION

A social psychological perspective on collected intelligence in metaheuristic computing has been presented toward the studies on the

cognitive and social psychological foundations of metaheuristic computing. This article has presented the properties of collective intelligence of human societies and its applications in metaheuristic computing. The fundamental human traits and collective behaviors have been described that form a foundation for explaining the human factors in collective intelligence. The fundamental problem on how collective intelligence is constrained by social environment and group settings has been explained with the formal motivation/attitude-driven behavioral model. The metaheuristic computational model of a generic cognitive process of problem solving has been developed. As a result, a coherent framework of sociological perspectives on collective intelligence has been established. A synergy between studies of genetic algorithms, evolutionary computing, and ant colony mechanisms in metaheuristic computing and the sociopsychological properties of collective intelligence has been created. Applications of collective intelligence in metaheuristic computing have identified in the studies of abstract intelligence, the denotational mathematical model of genetic algorithms, and the formal description of generic problem solving in natural and computational intelligence.

Figure 5. Interaction between problem solving and other cognitive processes in LRMB



ACKNOWLEDGMENT

The author would like to acknowledge the Natural Science and Engineering Council of Canada (NSERC) for its partial support to this work. The author would like to thank Prof. Peng-Yeng Yin's support and the anonymous reviewers' comments and suggestions on this invited article.

REFERENCES

- Blum, C., & Roli, A. (2003). Metaheuristics in combinatorial optimization: Overview and conceptual comparison. *ACM Computing Surveys*, 35(3), 268–308. doi:10.1145/937503.937505
- Buckles, B. P., & Petry, F. E. (1992). *Genetic algorithms*. Los Alamitos, CA: IEEE CS Press.
- Coleman, J. S. (1990). *Fundamentals of social theory*. Cambridge, MA: Belknap Press of Harvard Univ Press.
- Davis, L. (Ed.). (1991). *Handbook of genetic algorithms*. Reinhold, NY: Van Nostrand.
- Eagly, A. H., & Chaiken, S. (1992). *The psychology of attitudes*. San Diego, CA: Harcourt Brace.
- Fazio, R. H. (1986). How do attitudes guide behavior? In R. M. Sorrentino & E. T. Higgins (Eds.), *The handbook of motivation and cognition: Foundations of social behavior*. New York: Guilford Press.
- Fishbein, M., & Ajzen, I. (1975). *Belief, attitude, intention, and behavior: An introduction to theory and research*. Reading, MA: Addison-Wesley.
- Freud, S. (1895). Project for a scientific psychology. In J. Strachey (Ed.), *The standard edition the complete psychological works of Sigmund Freud* (Vol. 1). London: Hogarth Press.
- Geem, Z. W., Kim, J. H., & Loganathan, G. V. (2001). A new heuristic optimization algorithm: Harmony search. *Simulation*, 76(2), 60–68. doi:10.1177/003754970107600201
- Glover, F., & Gary, A. (Eds.). (2003). *Handbook of metaheuristics: International series in operations research & management science* (Vol. 57). New York: Springer.
- Goldberg, D. E. (1989). *Genetic algorithms in search, optimization, and machine learning*. Reading, MA: Addison-Wesley.
- Goldstein, I. (1978). *Developing a computational representation for problem solving skills* (AI Memo 495). Cambridge, MA: MIT.
- Hardy, C., & Latane, B. (1986). Social loafing on a cheering task. *Social Science*, 71, 165–172.
- Holland, J. H. (1975). *Adaptation in natural and artificial systems*. Ann Arbor, MI: University of Michigan Press.
- Hull, C. L. (1943). *Principles of behavior: An introduction to behavior theory*. New York: Oxford University Press.
- Jackson, J. M., & Harkins, S. G. (1985). Equity in effort: An explanation of the social loafing effect. *Journal of Personality and Social Psychology*, 49, 1199–1206. doi:10.1037/0022-3514.49.5.1199
- Janis, I. (1971). Groupthink. *Psychology Today*, 43–46.
- Kerr, N. L. (1983). Motivation losses in small groups: A social dilemma analysis. *Journal of Personality and Social Psychology*, 45, 819–828. doi:10.1037/0022-3514.45.4.819
- Komorita, S. S., & Barth, J. M. (1985). Components of reward in social bargaining. *Journal of Personality and Social Psychology*, 48, 364–373. doi:10.1037/0022-3514.48.2.364
- Kravits, D. A., & Martin, B. (1986). Ringelmann Rediscovered: The Original Article. *Journal of Personality and Social Psychology*, 50, 936–941. doi:10.1037/0022-3514.50.5.936
- Latane, B., Williams, K. D., & Harkins, S. G. (1979). Many hands make light the work: The cause and consequences of social loafing. *Journal of Personality and Social Psychology*, 37, 822–832. doi:10.1037/0022-3514.37.6.822
- Leahey, T. H. (1997). *A history of psychology: Main currents in psychological thought* (4th ed.). Upper Saddle River, NJ: Prentice-Hall.
- Macionis, J. J., Clarke, J. N., & Gerber, L. M. (1997). *Sociology* (2nd ed.). Scarborough, Ontario, Canada: Prentice Hall Allyn and Bacon.
- Maslow, A. H. (1962). *Towards a psychology of being*. Princeton, NJ: Van Nostrand.
- Maslow, A. H. (1970). *Motivation and personality* (2nd ed.). New York: Harper & Row.
- Newell, A., & Simon, H. A. (1972). *Human problem solving*. Upper Saddle River, NJ: Prentice-Hall.

- Robertson, I. (2001). *Problem solving*. Philadelphia: Psychology Press.
- Smith, R. E. (1993). *Psychology*. St. Paul, MN: West Publishing Co.
- Sternberg, R. J. (1998). *In search of the human mind* (2nd ed.). New York: Harcourt Brace.
- Talbi, E.-G. (2009). *Metaheuristics: From design to implementation*. New York: Wiley & Sons.
- Wang, Y. (2002a). Keynote: On cognitive informatics. In *Proceedings of the 1st IEEE International Conference on Cognitive Informatics (ICCI'02)*, Calgary, Canada (pp. 34-42). Los Alamitos, CA: IEEE CS Press.
- Wang, Y. (2002b). The real-time process algebra (RTPA). *Annals of Software Engineering*, 14, 235–274. doi:10.1023/A:1020561826073
- Wang, Y. (2003). On cognitive informatics. *Brain and Mind: A Transdisciplinary Journal of Neuroscience and Neuropsychology*, 4(3), 151-167.
- Wang, Y. (2007a). *Software engineering foundations: A software science perspective: CRC series in software engineering II*. Boca Raton, FL: Auerbach Publications.
- Wang, Y. (2007b). The theoretical framework of cognitive informatics. *International Journal of Cognitive Informatics and Natural Intelligence*, 1(1), 1–27.
- Wang, Y. (2007c). On laws of work organization in human cooperation. *International Journal of Cognitive Informatics and Natural Intelligence*, 1(2), 1–15.
- Wang, Y. (2007d). On the cognitive processes of perception with emotions, motivations, and attitudes. *International Journal of Cognitive Informatics and Natural Intelligence*, 1(4), 1–13.
- Wang, Y. (2007e). The OAR model of neural informatics for internal knowledge representation in the brain. *International Journal of Cognitive Informatics and Natural Intelligence*, 1(3), 68–82.
- Wang, Y. (2008a). On contemporary denotational mathematics for computational intelligence. *Transactions of Computational Science*, 2, 6–29. doi:10.1007/978-3-540-87563-5_2
- Wang, Y. (2008b). On concept algebra: A denotational mathematical structure for knowledge and software modeling. *International Journal of Cognitive Informatics and Natural Intelligence*, 2(2), 1–19.
- Wang, Y. (2008c). RTPA: A denotational mathematics for manipulating intelligent and computational behaviors. *International Journal of Cognitive Informatics and Natural Intelligence*, 2(2), 44–62.
- Wang, Y. (2009a). On abstract intelligence: Toward a unified theory of natural, artificial, machinable, and computational intelligence. *International Journal of Software Science and Computational Intelligence*, 1(1), 1–17.
- Wang, Y. (2009b). On cognitive computing. *International Journal of Software Science and Computational Intelligence*, 1(3), 1–15.
- Wang, Y. (2009c). A cognitive informatics reference model of autonomous agent systems (AAS). *International Journal of Cognitive Informatics and Natural Intelligence*, 3(1), 1–16.
- Wang, Y., & Chiew, V. (2009). On the cognitive process of human problem solving. *Cognitive Systems Research: An International Journal*, 10(4), 81-92.
- Wang, Y., Kinsner, W., Anderson, J. A., Zhang, D., Yao, Y., & Sheu, P. (2009). A doctrine of cognitive informatics. *Fundamenta Informaticae*, 90(3), 203–228.
- Wang, Y., Kinsner, W., & Zhang, D. (2009). Contemporary cybernetics and its faces of cognitive informatics and computational intelligence. *IEEE Transactions on System, Man, and Cybernetics (B)*, 39(4), 823–833. doi:10.1109/TSMCB.2009.2013721
- Wang, Y., Wang, Y., Patel, S., & Patel, D. (2006). A layered reference model of the brain (LRMB). *IEEE Transactions on Systems, Man and Cybernetics. Part C, Applications and Reviews*, 36(2), 124–133. doi:10.1109/TSMCC.2006.871126
- Wang, Y., Zadeh, L. A., & Yao, Y. (2009). On the system algebra foundations for granular computing. *International Journal of Software Science and Computational Intelligence*, 1(1), 64–86.
- Westen, D. (1999). *Psychology: Mind, brain, and culture* (2nd ed.). New York: John Wiley & Sons.
- Wiggins, J. A., Eiggins, B. B., & Zanden, J. V. (1994). *Social psychology* (5th ed.). New York: McGraw-Hill.
- Wikipedia. (2009). *Metaheuristic*. Retrieved from <http://en.wikipedia.org/wiki/Metaheuristic>.
- Wittig, A. F. (2001). *Schaum's outlines of theory and problems of introduction to psychology* (2nd ed.). New York: McGraw-Hill.

- Yin, P.-Y. (2010). Editorial. *International Journal of Applied Metaheuristic Computing*, 1(1), i-ii.
- Zander, A. (1979). The psychology of the group process. *Annual Review of Psychology*, 418.

Yingxu Wang is professor of cognitive informatics and software engineering, director of International Center for Cognitive Informatics (ICfCI), and director of Theoretical and Empirical Software Engineering Research Center (TESERC) at the University of Calgary. He is a Fellow of WIF, a P.Eng of Canada, a senior member of IEEE and ACM, and a member of ISO/IEC JTC1 and the Canadian Advisory Committee (CAC) for ISO. He received a PhD in software engineering from The Nottingham Trent University, UK, in 1997, and a BSc in electrical engineering from Shanghai Tiedao University in 1983. He has industrial experience since 1972 and has been a full professor since 1994. He was a visiting professor in the Computing Laboratory at Oxford University in 1995, Dept. of Computer Science at Stanford University in 2008, and the Berkeley Initiative in Soft Computing (BISC) Lab at University of California, Berkeley in 2008, respectively. He is the founder and steering committee chair of the annual IEEE International Conference on Cognitive Informatics (ICCI). He is founding editor-in-chief of International Journal of Cognitive Informatics and Natural Intelligence (IJCINI), founding editor-in-chief of International Journal of Software Science and Computational Intelligence (IJSSCI), associate editor of IEEE Trans on System, Man, and Cybernetics (A), and editor-in-chief of CRC Book Series in Software Engineering. He is the initiator of a number of cutting-edge research fields and/or subject areas such as cognitive informatics, cognitive computing, abstract intelligence, denotational mathematics, theoretical software engineering, coordinative work organization theory, cognitive complexity of software, and built-in tests. He has published over 105 peer reviewed journal papers, 193 peer reviewed conference papers, and 12 books in cognitive informatics, software engineering, and computational intelligence. He is the recipient of dozens international awards on academic leadership, outstanding contribution, research achievement, best paper, and teaching in the last 30 years.